

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ
BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ
FACULTY OF INFORMATION TECHNOLOGY

ÚSTAV POČÍTAČOVÉ GRAFIKY A MULTIMÉDIÍ
DEPARTMENT OF COMPUTER GRAPHICS AND MULTIMEDIA

INTERAKTIVNÍ MANUÁL V ROZŠÍŘENÉ REALITĚ

INTERACTIVE MANUAL IN AUGMENTED REALITY

BAKALÁŘSKÁ PRÁCE
BACHELOR'S THESIS

AUTOR PRÁCE **NIKITA SMIRNOV**
AUTHOR

VEDOUCÍ PRÁCE **Ing. MICHAL KAPINUS, Ph.D.**
SUPERVISOR

BRNO 2026

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

Zadání bakalářské práce

171546

Ústav:	Ústav počítačové grafiky a multimédií (UPGM)
Student:	**Smirnov Nikita**
Program:	Informační technologie
Název:	**Interaktivní manuál v rozšířené realitě**
Kategorie:	Uživatelská rozhraní
Akademický rok:	2025/26

Zadání:

1. Prostudujte koncept rozšířené reality a její využití při návrhu uživatelských rozhraní na mobilním zařízení. Seznamte se s existujícími aplikacemi řešící danou problematiku.
2. Seznamte se s herním enginem Unity3D a prostudujte možnosti tvorby interaktivních elementů v tomto enginu.
3. Navrhněte metodu a sadu GUI elementů umožňující efektivně používat interaktivní manuály.
4. Navržené rozhraní implementujte na mobilním zařízení (telefon, tablet, Oculus Quest 3 apod.).
5. Proveďte experimenty, demonstrujte a diskutujte vlastnosti vašeho řešení. Zaměřte se na použitelnost a uživatelskou zkušenost.
6. Vytvořte video prezentující klíčové výsledky vaší práce.

Literatura:

- Dieter Schmalstieg, Tobias Hollerer. *Augmented Reality: Principles and Practice.* Addison-Wesley, 2016. ISBN: 978-0321883575.
- Russ Unger, Carolyn Chandler. *A Project Guide to UX Design: For user experience designers in the field or in the making.* New Riders, 2012. ISBN: 0132931729.
- Dále dle pokynu vedoucího.

Při obhajobě semestrální části projektu je požadováno:
Body 1, 2, 3 a rozpracovaný bod 4.

Podrobné závazné pokyny pro vypracování práce viz https://www.fit.vut.cz/study/theses/

Vedoucí práce:	**Kapinus Michal, Ing., Ph.D.**
Vedoucí ústavu:	Černocký Jan, prof. Dr. Ing.
Datum zadání:	1.11.2025
Termín pro odevzdání:	13.5.2026
Datum schválení:	7.5.2026

Abstrakt

Práce je věnována vývoji interaktivního manuálu v rozšířené realitě, určeného k zobrazení technických instrukcí přímo v prostoru reálného objektu. Cílem je vytvořit dostupné řešení, schopné pracovat na mobilních zařízeních a podporovat současné použití velkého množství markerů. Navržený prototyp zahrnuje systém detekce a sledování markerů, uživatelské rozhraní a serverovou část pro správu instrukcí. Realizace byla provedena s využitím Unity, OpenCV a vlastního metodu detekce markerů. Provedené testování ukázalo, že použití rozšířené reality snižuje počet chyb a urychluje provádění složitých sekvenčních úloh ve srovnání s tradiční papírovou dokumentací. Vyvinuté řešení demonstruje praktickou použitelnost AR manuálů a může sloužit jako základ pro další rozvoj systémů technické podpory.

Abstract

The work is devoted to the development of an interactive augmented reality manual designed to display technical instructions directly in the space of a real-world object. The goal is to create an accessible solution that can run on mobile devices and support the simultaneous use of a large number of markers. The proposed prototype includes a marker detection and tracking system, a user interface, and a server component for managing instructions. The implementation was carried out using Unity, OpenCV, and a custom marker detection method. The tests conducted showed that using AR reduces the number of errors and speeds up the execution of complex sequential tasks compared to traditional paper documentation. The developed solution demonstrates the practical applicability of AR manuals and can serve as a foundation for further development of technical support systems.

Klíčová slova

AR, rozšířená realita, interaktivní manuál, detekce markerů, Unity, mobilní aplikace, technická dokumentace

Keywords

AR, augmented reality, interactive manual, marker detection, Unity, mobile application, technical documentation

Citace

SMIRNOV, Nikita. *Interaktivní manuál v rozšířené realitě*. Brno, 2026. Bakalářská práce. Vysoké učení technické v Brně, Fakulta informačních technologií. Vedoucí práce Ing. Michal Kapinus, Ph.D.

Interaktivní manuál v rozšířené realitě

Prohlášení

Prohlašuji, že jsem tuto bakalářskou práci vypracoval samostatně pod vedením pana Ing. Michala Kapinuse, Ph.D. Uvedl jsem všechny literární prameny, publikace a další zdroje, ze kterých jsem čerpal.

.....
Nikita Smirnov
10. května 2026

Poděkování

Tímto bych chtěl poděkovat svému vedoucímu práce panu Ing. Michalu Kapinusovi, Ph.D., za odborné vedení, cenné rady a podporu během celé realizace této bakalářské práce.

Obsah

1 Úvod **3**

2 Rozšířená realita **4**
2.1 Definice a základní pojmy 4
2.2 Zobrazovací zařízení a platformy pro AR 5
2.3 Vliv AR na efektivitu plnění úkolů 8
2.4 Existující řešení 12

3 Prostorový tracking v AR **16**
3.1 Přístupy ke sledování polohy 16
3.2 Referenční markery (Fiducial markers) 17
3.3 Detekce markerů 20
3.4 Odhad polohy markeru 28

4 Návrh systému pro práci s AR manuály **32**
4.1 Cíle systému a kontext použití 32
4.2 Uživatelský průzkum a požadavky 35
4.3 Architektura systému 36
4.4 Datový model 39
4.5 Detekce a sledování markerů 41

5 Uživatelské rozhraní a UX návrh **44**
5.1 Cíle návrhu uživatelského rozhraní 44
5.2 Návrhové principy 45
5.3 Uživatelské rozhraní aplikace 45
5.4 Rozhraní pro autory manuálů 48
5.5 Průzkum uživatelů na základě představeného konceptu 52

6 Implementace **54**
6.1 Použité technologie a nástroje 54
6.2 Struktura projektu v Unity 55
6.3 Implementace AR části v Unity 57
6.4 Nativní plugin ArUco využívající OpenCV 62
6.5 Backend 64

7 Testování **67**
7.1 Metodika testování 67
7.2 Statistické výsledky 68

7.3 Empirické výsledky (uživatelské hodnocení) 70
7.4 Stabilita fungování aplikace 71
7.5 Omezení testování 72

8 Závěr **73**

Literatura **74**

Kapitola 1

Úvod

Technická dokumentace hraje klíčovou roli v tom, jak uživatelé pracují se zařízeními, vybavením a složitými technologickými systémy. Přestože tradiční tištěné a elektronické manuály zůstávají standardem, často se ukazují jako nepraktické: vyžadují neustálé přepínání pozornosti, ztěžují orientaci v posloupnosti kroků a zvyšují pravděpodobnost chyb. S rozvojem mobilních zařízení a technologií prostorového sledování se objevila možnost zobrazovat pokyny přímo v kontextu reálného objektu, což otevírá cestu k novým formám interakce s technickou dokumentací.

Rozšířená realita (AR) se v posledních letech dynamicky rozvíjí a nachází uplatnění v průmyslu, vzdělávání i servisní podpoře. Existující řešení jsou však často omezená buď vysokými náklady na vývoj, úzkou specializací, nebo technickými limity, například nemožností pracovat s velkým množstvím markerů současně. To je obzvlášť důležité v situacích, kdy objekt obsahuje mnoho prvků, které vyžadují individuální přiřazení instrukcí. Jedním z klíčových odlišujících rysů této práce je implementace systému, který je schopen stabilně pracovat s velkým množstvím markerů současně, což rozšiřuje možnosti vytváření složitých vícekrokových návodů a zvyšuje flexibilitu použití AR manuálů.

Cílem této práce je návrh a zhodnocení prototypu interaktivního AR manuálu, který zahrnuje systém prostorového sledování, uživatelské rozhraní a serverovou část pro správu instrukcí. Navrhované řešení je zaměřeno na snížení kognitivní zátěže, omezení počtu chyb a zvýšení komfortu při plnění úkolů díky zobrazování instrukcí přímo v prostoru reálného objektu.

Práce zahrnuje několik klíčových směrů: analýzu stávajících přístupů k AR a jejich použitelnosti v kontextu technické dokumentace, návrh architektury systému, implementaci detekce a sledování markerů, vytvoření uživatelského rozhraní a provedení testování za účasti reálných uživatelů. Výsledkem je funkční prototyp mobilní aplikace, který umožňuje zobrazovat postupové instrukce v rozšířené realitě a hodnotit vliv tohoto přístupu na rychlost plnění úkolů a počet chyb.

Struktura práce je organizována následujícím způsobem. Kapitola 2 obsahuje přehled technologií rozšířené reality a jejich použití. Kapitola 3 je věnována prostorovému trackingu a metodám detekce markerů. Ve 4 kapitole je uveden systémový návrh včetně architektury a datového modelu. Kapitola 5 popisuje uživatelské rozhraní a principy interakce. Kapitola 6 se zabývá implementací aplikace a backendu. Kapitola 7 obsahuje testovací metodiku, statistické a empirické výsledky. V závěrečné kapitole jsou shrnuty výsledky práce a diskutovány směry dalšího rozvoje systému.

Kapitola 2

Rozšířená realita

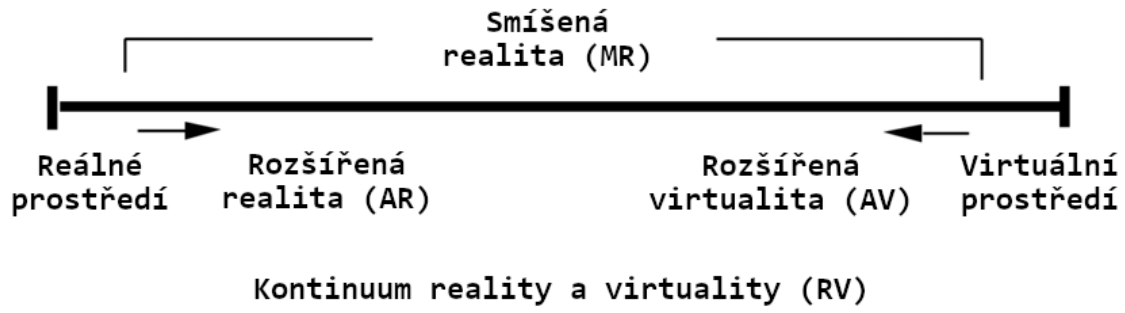
Pro hlubší porozumění terminologii užívané v této práci a pro orientaci v aktuálním stavu vývoje a průmyslových aplikací rozšířené reality (Augmented Reality, AR) je nejprve nezbytné vymezit a objasnit základ této práce.

2.1 Definice a základní pojmy

Pojem „rozšířená realita“ se dnes používá všude, jeho přesná definice však zůstává předmětem diskusí. Různé zdroje jej interpretují odlišně – jedni kladou důraz na technickou realizaci, druzí na vnímání uživateli. Aby bylo možné jasně vymezit, o čem přesně se v této práci jedná, je nutné začít od základní klasifikace.

V roce 1994 Milgram a Kishino navrhli koncepční model (viz obr. 2.1), který dostal název kontinuum reality a virtuality (Reality-Virtuality Continuum) [27]. Na jednom konci tohoto kontinua se nachází zcela reálné prostředí – to, co člověk vnímá bez jakýchkoli technologických zásahů. Na druhém konci je zcela virtuální prostředí, ve kterém je uživatel zcela izolován od reálného světa a vše, co vnímá, je generováno počítačem. Celý prostor mezi těmito dvěma extrémy autoři nazvali smíšenou realitou (Mixed Reality, MR).

V rámci smíšené reality lze rozlišit dva důležité dílčí případy. Rozšířená realita je případ, kdy je reálné prostředí doplněno virtuálními prvky: trojrozměrnými objekty, textem a animacemi. Uživatel přitom i nadále vnímá reálný svět jako základ. Opačný případ – rozšířená virtualita (Augmented Virtuality, AV) – nastává, když jsou do převážně virtuálního prostředí vkládány prvky reálného světa, například přenos videa z kamery.



Obrázek 2.1: Zjednodušené znázornění kontinua RV. Převzato a přeloženo z [27]

Kromě klasifikace Milgrama a Kishina se široce používá definice AR, kterou v roce 1997 navrhl Azuma [3]. Podle Azumy lze systém považovat za systém rozšířené reality, pokud současně splňuje tři podmínky:

1. Spojuje reálné a virtuální objekty v jednotném prostoru.
2. Funguje v reálném čase.
3. Správně registruje virtuální objekty vzhledem k reálnému trojrozměrnému prostoru.

Třetí podmínka, prostorová registrace, je klíčová a technicky nejnáročnější: právě ona odlišuje AR od pouhého překrytí grafiky přes obraz. Bez přesného přiřazení virtuálních prvků ke skutečným objektům ztrácí systém svou hlavní výhodu – pocit, že digitální obsah fyzicky existuje v prostoru.

Azumova definice zůstává aktuální i dnes, ačkoli od roku 1997 prošly technologie AR cestou od objemných laboratorních zařízení až po software, který funguje na běžném telefonu a zároveň umožňuje sledovat polohu v prostoru s vysokou přesností a zobrazovat přitom trojrozměrné modely s vysokým počtem polygonů.

Právě AR je předmětem této práce, kvůli tomu že umožňuje uživateli zobrazovat doplňkové informace, aniž by ho odtrhával od reálného světa, ve kterém provádí své činnosti. A proto je tento přístup ideální pro integraci manuálů do výrobních procesů a běžného použití.

2.2 Zobrazovací zařízení a platformy pro AR

Rozšířená realita může být implementována na různých typech hardwarových platforem, z nichž každá nabízí vlastní kompromis mezi funkčností, cenou a uživatelským komfortem. Volba platformy přímo určuje, jakým způsobem uživatel s obsahem AR interaguje – a tedy i to, jak by měl být navržen samotný systém. Bude zvažována pouze část existujících technologií [44], které jsou pro tento projekt nejvhodnější.

Head-Mounted Displays (HMD)

HMD jsou zařízení nasazovaná na hlavu, která uvolňují ruce uživatele pro práci. Z hlediska zobrazování se HMD běžně dělí na dvě zavedené kategorie: Optical See-through a Camera See-through.

- **Optical See-through** (obr. 2.2a a 2.2c) – průhledné čočky přímo kombinují reálné prostředí s promítaným digitálním obsahem. Výhodou je přirozené vnímání okolí a lepší prostorová orientace. Nevýhodou je obvykle menší zorné pole pro vykreslovaný digitální obsah.
- **Camera See-through** (obr. 2.2b) – uzavřené headsety snímají okolí kamerou a uživateli zobrazují kompozitní obraz na interním displeji. Tento přístup umožňuje větší zorné pole pro AR obsah a plnější kontrolu nad vykreslováním, avšak uživatel vnímá okolí „přes kameru“, což může omezit přirozenost vnímání a zhoršit efektivitu práce.

Pro účely této práce je důležité rozlišovat tyto kategorie, protože volba správného typu zařízení ovlivňuje předpokládaný způsob používání aplikace. V tomto kontextu se jako vhodnější jeví kategorie Optical See-through, protože zachovává přirozené vnímání reálného prostředí a snižuje vizuální rozptylování uživatele během plnění pracovních úkolů.

Mezi výhody těchto zařízení patří například:



(a) Microsoft HoloLens 2[1] (b) Meta Quest 3[2] (c) Meta Ray-Ban[3]

Obrázek 2.2: Příklady různých typů zařízení HMD. Optical See-through: AR brýle (a), chytré brýle (c). Camera See-through: headset s podporou smíšené reality (b).

- Možnost používat zařízení a mít přitom **volné ruce**, což je v některých výrobních situacích kriticky důležité.
- Široké **zorné pole** (oproti mobilním zařízením)
- **Zobrazení ovládacích prvků** a nápovědy přímo v zorném poli uživatele, bez nutnosti rozptylovat se sledováním samostatně umístěné obrazovky.

Samozřejmě mají i řadu nevýhod:

- **Vysoká cena**, což pro většinu malých a středních podniků představuje velký problém při integraci do výroby a pro běžného uživatele se často stává hlavním důvodem, proč je nepoužívá.
- **Značná hmotnost**, i když je u AR brýlí tento problém minimalizován, může uživatel při dlouhodobém používání přesto cítit nepohodlí.
- Omezená **výdrž baterie**, je nutné mít k dispozici několik kusů zařízení, což v kombinaci s vysokou cenou představuje rovněž problém.
- Nutnost **speciálního zaškolení** uživatelů kvůli tomu, že tato zařízení jsou pro většinu lidí novinkou.

Studie porovnávající mobilní a HMD zařízení v kontextu AR aplikací potvrzují, že hmotnost HMD způsobovala uživatelům nepohodlí a bolest, zatímco u mobilních zařízení tento problém nebyl zaznamenán [32].

Mobilní zařízení

Mobilní zařízení – představují dnes nejrozšířenější platformu pro AR. Podle údajů společnosti Statista počet uživatelů mobilních telefonů ve světě od roku 2026 překročil 5 miliard[4].

[1]Převzato z Synergiz. Dostupné z https://www.synergiz.com/en/produit/hololens-2-industrial-edition-2/

[2]Převzato z Amazon. Dostupné z https://www.amazon.com/Meta-Quest-512GB-Breakthrough-Reality-3/dp/B0CD1JTBSC

[3]Převzato z Meta. Dostupné z https://www.meta.com/ai-glasses/wayfarer/?srsltid=AfmBOoo47NAPeOPlY4Qq7kMNfKQoEVtIvqNjVnSSFcRpVVZZNuc1szXa

[4]Statista: number of smartphone users 2014-2029. Dostupné z https://www.statista.com/forecasts/1143723/smartphone-users-in-the-world/

Mezi mobilními zařízeními vhodnými pro rozšířenou realitu můžeme z hlediska ergonomie rozlišit dva základní typy:

- Mobilní telefony
- Tablety

I když se tato zařízení liší především svými rozměry, má to výrazný vliv na uživatelský zážitek při práci s aplikacemi AR. Mobilní telefony jsou kompaktnější a snáze se nosí s sebou, což uživateli umožňuje pracovat s AR v omezených prostorech a také rychleji přepínat pozornost mezi interakcí s reálným světem a rozšířenou realitou. Tablety naopak nabízejí větší displej, což je užitečné pro zobrazení informací uživateli, a obvykle disponují větším výkonem a kapacitou baterie, což umožňuje zvýšit přesnost při práci s rozšířenou realitou a prodloužit dobu, po kterou může uživatel nepřetržitě pracovat.

Mezi výhody těchto zařízení patří například:

- **Nízká cena**, pokud jde o soukromé použití, téměř každý dnes vlastní mobilní telefon, a tudíž nemá při používání AR aplikací žádná omezení. V případě podniků je cena těchto zařízení relativně nízká ve srovnání s náklady na jiné vybavení.
- Vysoká **mobilita a dostupnost**, tato zařízení máte vždy po ruce, případně je lze snadno přenášet s sebou téměř za jakýchkoli podmínek.
- **Snadné používání**, rozhraní a ovládání jsou srozumitelné prakticky pro každého, a proto není nutné žádné další zaškolení.

Samozřejmě mají i řadu nevýhod:

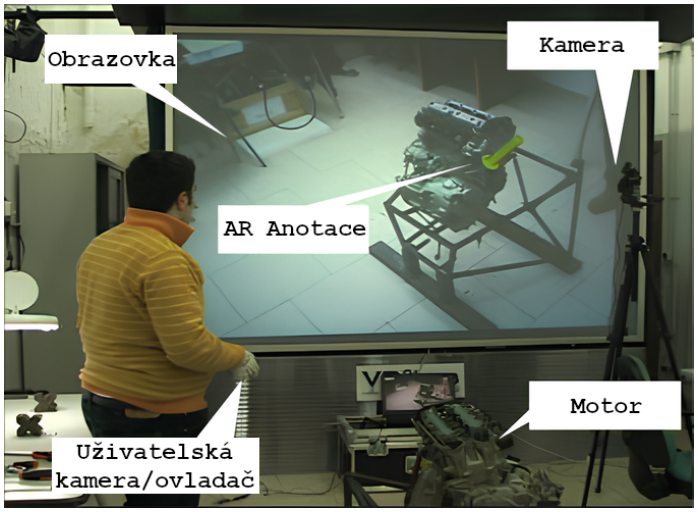
- Hlavní nevýhodou je nutnost držet **zařízení v ruce**, u tabletů často dokonce v obou rukou, což omezuje uživatele v jeho činnostech a často znemožňuje souběžnou práci a interakci s aplikací.
- Příliš staré modely nemají dostatečný **výkon** pro stabilní provoz AR aplikací, zatímco zařízení s Android API nižším než 11 často vůbec nepodporují jejich spouštění.
- **Rozptýlení pozornosti**, uživatel je nucen odvrátit pozornost od plnění úkolu a soustředit se na zvláštní displej. Což také snižuje dobu reakce a rychlost provedení práce.

Prostorové displeje

Také stojí za zmínku tento typ zařízení [10]. Obvykle se skládají z kombinace samostatných kamer a umístěných displejů jak je uvedeno na obrázku 2.3. To je pohodlné v případě implementace na statických pracovních místech, zároveň odpadají problémy s výkonem a kapacitou baterie a jejich cena také není vysoká. Nicméně objevují se takové problémy, jako je adaptace stávajících řešení z jiných platforem, statičnost a náročnost instalace.

Z těchto důvodů by měla být tato zařízení zvažována pro velmi specifické případy použití, nicméně stále mají své opodstatnění.

Pro účely této práce – vytvoření systému AR-návodů zaměřeného na široké praktické využití bez zvláštních požadavků na vybavení, jsou nejvhodnější platformou mobilní zařízení. Tato volba zajišťuje maximální dostupnost systému při zachování dostatečné funkčnosti pro zobrazení interaktivních pokynů.



Obrázek 2.3: Využití prostorových displejů na příkladu údržby motoru. Převzato a přeloženo z [10]

Moderní technologie umožňují multiplatformní vývoj, což umožňuje nasazení aplikace na různých platformách bez nutnosti kompletního přepisu kódu. Například AR Foundation, která představuje SDK[5] pro integraci s různými platformami, jako jsou:

- **Android** – Google ARCore XR
- **iOS** – Apple ARKit XR
- **Meta Quest** – Unity OpenXR: Meta
- **visionOS** – Apple visionOS XR

2.3 Vliv AR na efektivitu plnění úkolů

Samozřejmě může vyvstat otázka, zda je zavádění AR jak do výrobního procesu, tak i přímo do práce s manuály efektivní. Abychom na ni odpověděli, musíme podrobněji prostudovat stávající tradiční přístup ve výrobních procesech, nové průmyslové standardy a také příklady již realizované integrace AR.

Problém tradičních manuálů

Moderní podoby manuálů, stejně jako přístupy k jejich tvorbě a používání, obsahují mnoho nedostatků jak pro výrobce, tak i pro běžného uživatele, což vede ke zvýšení času stráveného triviálními úkoly, frustraci uživatele a snížení celkové efektivity. Jako příklad můžeme uvést několik nejrozšířenějších problémů:

- **Vizuální reprezentace** – kvůli používání 2D obrázků je pro uživatele často obtížné pochopit umístění ovládacích prvku na zařízení, což je obzvlášť aktuální u velkých strojů. Postupné provádění instrukcí je také ztíženo kvůli absenci pochopitelných přechodů mezi jednotlivými kroky, kvůli čemuž se uživatel může ztrácet.

[5]Unity: AR Foundation. Dostupné z https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@6.4/manual/index.html

- **Sjednocení různých verzí** – někdy za účelem snížení nákladů výrobce spojuje několik podobných verzí do jednoho manuálu. Kvůli tomu se jeho rozsah může zvětšit, protože v místech, kde se verze liší, je nutné opakovaně uvádět stejné ovládací prvky. To může uživateli způsobovat dodatečné časové náklady při hledání informací týkajících se jeho konkrétního zařízení nebo vést k chybám v důsledku popletení jednotlivých verzí.
- **Problém s aktualizací** – spočívá v tom, že výrobce má možnost provádět úpravy v dokumentaci pouze do okamžiku tisku, protože poté mohou být náklady na opravu chyby příliš vysoké. To s sebou nese řadu nepříjemností, riziko lidské chyby existuje vždy, výrobce je nucen věnovat další čas pečlivé kontrole materiálu, ale i přesto zůstává vysoká pravděpodobnost chyby. Může také nastat situace, kdy již po uvedení produktu do hromadného prodeje budou odhalena problematická místa, která již nepůjde opravit.
- **Velikost vícejazyčných manuálů** – čím je zařízení složitější, tím výraznější je vliv tohoto problému, protože obvykle jak samotné zařízení, tak všechny materiály k němu (krabice, manuál, popisky) jsou vydávány buď pro globální trh, nebo pro skupinu zemí, které jsou si geograficky blízké. Uživatel je nucen trávit více času hledáním svého jazyka. A efektivní rozsah informací, které člověk skutečně využije, se s rostoucím počtem jazyků blíží nule.
- **Vyhledávání informací** – i když v poslední době lze častěji najít elektronické verze papírových manuálů, kde je tento problém o něco méně patrný, uživatelé obvykle nechtějí ztrácet čas jejich hledáním a raději využijí materiály přiložené k jejich zařízení. Také i přes dobře zpracovaný obsah se při prvních použitích může uživatel setkat s problémem, že přesně neví, co vlastně hledá, a v důsledku toho je nucen přečíst znovu celou kapitolu, aby našel potřebnou část informací.
- **Dostupnost** – často je nutné trávit čas hledáním manuálu, běžní uživatelé se občas setkávají s tím, že jejich dokumentace byla zcela ztracena, nebo byl její stav natolik znehodnocen, že další používání již není možné. Ve výrobních provozech dochází ke ztrátě dokumentace méně často, nicméně čas potřebný na její vyhledání také roste s rostoucím počtem používaných zařízení.
- **Náklady na výrobu** – při vytváření nového zařízení je výrobce nucen počítat s tím, že příprava manuálu zabere nemálo času: od promyšlení struktury a tvorby infografiky až po finální kontrolu a tisk. V případě zjištění chyby nebo problematických míst po vytištění je výrobce také nucen buď nechat vše tak, jak je, nebo provést úpravy při vydání dalších sérií, aby se vyhnul dodatečným nákladům.

Interakce s tradičními instrukcemi obvykle představuje standardní sled kroků:
uživatel si instrukci přečte → odloží ji/odvrátí zrak → provede požadovanou činnost → a poté se k instrukci znovu vrátí. To vede k vyšší kognitivní zátěži, stejně jako ke zvýšení časové náročnosti a pravděpodobnosti chyb, což bylo prokázáno v řadě studií [29, 17].

Co přináší AR

Integrace rozšířené reality (AR) najednou řeší mnoho problémů tradičního přístupu, jak obecně v oblasti pracovních procesů, tak přímo při práci s manuály a jejich použití na pracovišti. Mezi výhody lze vyzdvihnout několik následujících bodů:

- **Snížení počtu chyb**, kterých se uživatel dopouští – Mnoho studií se shoduje v tom, že největší přínos rozšířené reality při integraci do pracovních procesů se projevuje právě ve srovnání počtu dopuštěných chyb [47]. Silný vliv má také *faktor složitosti* úlohy – čím je úloha složitější a čím více po sobě jdoucích kroků obsahuje, tím větší je vliv AR na konečný výsledek, a to jak na počet chyb, tak na rychlost plnění úkolů. Také v některých studiích [39] je zmiňován vliv *zkušeností uživatele* při plnění zadaného úkolu – zatímco u zkušených uživatelů přínos používání AR směřuje k nule, u ostatních má výrazně pozitivní dopad.

- **Snížení kognitivní zátěže** – tato výhoda je poměrně nejednoznačná a silně závisí jak na použité AR platformě, tak na úloze, ve které je využívána AR asistence, což potvrzují některé studie [6]. Výsledky testování se často shodují v tom, že AR systémy jsou spojeny s nižší nebo ekvivalentní kognitivní zátěží při vyšší produktivitě.

- **Zvýšení rychlosti lokalizace úkolů** – jedním z hlavních problémů tradiční dokumentace k plnění úkolů je propast mezi informací a objektem jejího použití. Místo standardní posloupnosti kroků může uživatel zobrazit informace o prvku, který ho zajímá, přímo překryté na reálném objektu. Některé studie ukazují statisticky významné zvýšení rychlosti ve srovnání s tradičními možnostmi [11, 16], což výrazně zvyšuje celkovou rychlost a efektivitu plnění úkolů.

- **Pozitivní uživatelská zkušenost** – poměrně subjektivní výhoda, která je takovou především u aplikací určených k používání pro osobní úkoly. Výzkumy však často zmiňují, že i v případech, kdy rychlost ani počet chyb při používání AR systémů nevykazovaly statisticky významné snížení, byla uživatelská zkušenost ve srovnání s tradičními analogiemi snazší a zajímavější [43].

Je také vhodné zmínit problematické aspekty používání tradičních manuálů, které se daří odstraňovat právě díky specifikům rozšířené reality při použití ve výrobních procesech:

- Díky použití grafického znázornění ve formě 3D modelů, textových nápověd, videí a dalších typů [50] přímo v bodech zájmu je pro uživatele často **snazší vnímat** informace a sled jednotlivých kroků.

- Problém s **aktualizací** také odpadá, protože pro aktualizaci informací stačí buď aktualizovat data na serveru, nebo aktualizovat samotnou aplikaci (v závislosti na způsobu, který si vybral vývojář).

- **Velikost manuálu** nehraje roli, protože všechny informace jsou uloženy v zařízení uživatele. Je také třeba poznamenat, že tento problém se nevyskytuje u dokumentací, které jsou od začátku dodávány v elektronické podobě nebo mají elektronickou verzi.

Je však také důležité zmínit, že výzkumy, které analyzují a shromažďují statistiky podle metrik používání a integrace AR, jsou často kritizovány, protože bývají velmi subjektivní hned z několika důvodů:

- **Volba AR platformy** má silný vliv na výslední uživatelský zážitek.

- Často se v podobných výzkumech používá **vlastní prototyp**, který může mít řadu svých nedostatků, přičemž analýza je přesto prezentována jako objektivní pro daný typ úloh.

- Některé studie zmiňují silný vliv „**dodatečných funkcí**“, jejichž realizace nebo naopak absence výrazně ovlivňuje konečný výsledek.
- **Oblast použití** má také silný vliv na výsledek analýzy, takže v některých oblastech pozorujeme výrazný pozitivní dopad na celkovou efektivitu, zatímco v jiných integrace AR nepřináší významné změny nebo dokonce vykazuje negativní výsledek.
- Dalším relevantním faktorem, který bývá často opomíjen, je **předchozí uživatelská zkušenost**, jak s používáním AR zařízení, tak i s danou testovací úlohou. V důsledku toho může nastat situace, kdy jedinec s rozsáhlou zkušeností s plněním konkrétního úkolu začne tento úkol vykonávat znovu za využití AR asistence. Výsledkem pak může být, že zjištěný vliv je statisticky nevýznamný, případně dochází dokonce k zhoršení výkonu.
- Často se testování účastní příliš **malý vzorek účastníků**, což posiluje vliv výše zmíněných faktorů na celkové výsledky.

Právě z těchto důvodů lze často pozorovat situaci, kdy jedna studie dochází k závěru, že efektivita využívání tradičních forem dokumentace vykazuje lepší výsledky než při integraci AR [43], zatímco jiné studie naopak ukazují významné zvýšení metrik při jejím používání [4].

Průmysl 4.0

Je třeba poznamenat, že dynamický rozvoj moderních technologií vyvolává potřebu systematické integrace nových postupů a standardů do průmyslové praxe s cílem zvýšit celkovou efektivitu výrobních procesů. Gattullo et al. [14] ve své práci řadí technologii rozšířené reality mezi klíčové prvky v kontextu nových standardů a současně identifikuje a adaptuje šest hlavních principů designu průmyslu 4.0, které lze použít v kontextu vývoje vlastního nástrojového vybavení pro řešení úloh:

- **Interoperabilita** – interakce mezi člověkem a výrobními systémy by měla probíhat prostřednictvím internetu věcí (Internet of Things, IoT) a internetu služeb (Internet of Services, IoS). `Standardizace`, stejně jako `vícejazyčnost`, v tomto kontextu představují klíčové faktory ovlivňující efektivitu a spolehlivost těchto komunikačních procesů.
- **Virtualizace** – kyber-fyzické systémy (Cyber-Physical Systems, CPS) interagují s virtuálními a simulačními modely reálného světa, díky čemuž mohou sledovat a předávat informace o fyzických procesech. K tomu je však zapotřebí `digitální verze` dokumentace.
- **Decentralizace** – s rostoucí poptávkou po podobných produktech se centralizovaný systém stává hůře škálovatelným. Proto je nutné, aby i při výpadcích zůstal zachován `lokální přístup` k dokumentaci. Systémy by také měly poskytovat `konkrétní postup` pro odstranění chyby, namísto toho, aby uživatele nutili k samostatnému hledání v celé dokumentaci.
- **Funkce v reálném čase** – v případě použití IoT musí být data poskytována a analyzována v reálném čase, aby bylo možné okamžitě reagovat v případě poruch. Technická dokumentace také musí mít možnost `aktualizace v reálném čase`.

- **Orientace na služby** – výroba by se měla opírat o servisně orientovanou architekturu, která organizuje všechny prvky jako služby dostupné prostřednictvím IoS. To zahrnuje i dokumentaci s postupy údržby, které musí být k dispozici na vyžádání. Jako jeden z příkladů autoři uvádějí – vzdálenou údržbu.
- **Modularita** – pro flexibilní přizpůsobení změnám ve výrobním cyklu musí dokumentace podporovat `snadnou rozšiřitelnost` a integraci nových přístupů a technologií.

Tyto principy je nezbytné zohlednit při koncipování vlastního souboru nástrojů pro práci s dokumentací, a to v souladu s aktuálně převládajícími průmyslovými postupy. V dalších kapitolách budou tyto principy detailně rozebrány a aplikovány z hlediska fundamentálních principů návrhu a vývoje nástrojů.

2.4 Existující řešení

V současné době, stejně jako v případě vymezení pojmu AR, neexistuje jednotná a ustálená klasifikace aplikací. Autoři konkrétních studií obvykle vytvářejí vlastní klasifikační systémy v souladu s vlastní výzkumnou otázkou, což vede k vzniku široké škály typologií. Ty se pohybují například od jednoduchého dichotomického rozlišení na `informativní` a `zábavní` aplikace, jak je tomu například v práci van Noorta a van Reijmersdala [48], až po podrobnější rozdělení na skupiny v konkrétním výzkumném sektoru, jako je tomu například v práci Röltgen et al. [36], kteří na základě zkoumání 75 AR aplikací vymezili 16 hlavních skupin spadajících do dvou nadřazených tříd – `instrukce` a `ilustrace`.

Obecná klasifikace AR aplikací

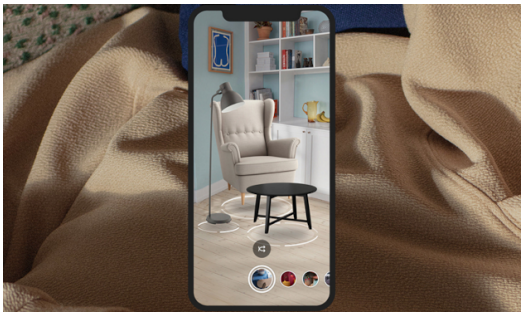
V rámci této práce navrhuji uplatnit obecnější klasifikační schéma, které vychází z analýzy AR aplikací pro mobilní zařízení v kontextu jejich funkčních charakteristik a obsahové struktury [42]. Autoři rozlišují 4 hlavní kategorie:

1. Nákupní aplikace
2. Zábavní aplikace
3. Informační aplikace
4. Aplikace sociálních médií

V poslední době, se zvýšením výkonu telefonů ve střední cenové třídě, lze pozorovat významný rozvoj všech uvedených kategorií. Jak již bylo dříve uvedeno, uživatelé často vnímají aplikace rozšířené reality (AR) jako intuitivnější a atraktivnější pro používání, a to i v případech, kdy konkrétní funkcionalita aplikace nepřinášejí významné zlepšení efektivity při plnění úkolů. U prvních dvou kategorií aplikací tyto systémy poskytují uživatelům nový typ uživatelského zážitku a nové funkční možnosti, které dříve neměli.

Klasifikace v rámci průmyslového využití

Následně se zaměřím na třetí kategorii, která je hlavním předmětem zájmu v rámci této práce. Tato kategorie bude podrobně analyzována na základě prostudované literatury [36, 38] a provedené analýzy trhu.



(a) IKEA App[6]



(b) Porsche AR Visualizer[7]

Obrázek 2.4: Příklady aplikací z kategorie – prezentace produktu.

- **Prezentace produktu** – aplikace primárně využívající trojrozměrné modely k vizualizaci informací. Do této kategorie spadají také v poslední době stále populárnější AR showroomy. Mohou být využívány jak k přilákání uživatelů, tak i k poskytování více imerzivní a interaktivní prezentace produktů.

  Příklady aplikací:

  - `IKEA App` (2.4a): umožňuje uživateli umístit vybrané produkty do svého bytu a zvolit tak nejvhodnější varianty jak z hlediska stylu, tak i barvy, aniž by bylo nutné zboží objednávat a následně vracet.
  - `Porsche AR Visualizer` (2.4b): umožňuje zákazníkům používat konfigurátor a sledovat změny v reálném čase, vybírat si nejvhodnější styl, aniž by se museli spoléhat pouze na obrázky na webu.

- **Školení (porozumění)** – Programy, které využívají ovládací prvky a interakční přístupy obdobné jako aplikace zařazené do první kategorie, se zpravidla rovněž neopírají o přímou manipulaci s reálnými objekty. Mají však jinou významovou náplň, a sice mohou být určeny k učení se reálným procesům nebo konstrukci objektů z reálného světa. Některé jejich varianty mohou dále uplatňovat způsoby prezentace blízký technickým manuálům, například prostřednictvím sekvenčních scénářů určených pro výuku krokových procesů.

  Příklady aplikací:

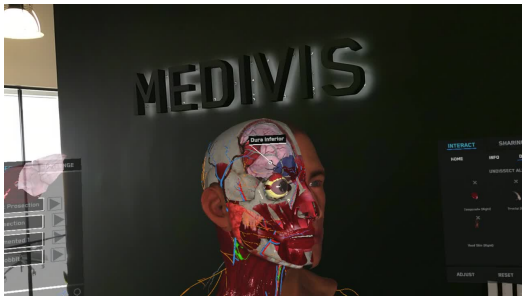
  - `AnatomyX`[8] (2.5a): umožňuje detailní studium anatomických struktur prostřednictvím vizualizace trojrozměrných modelů. Využití technologie rozšířené reality současně umožňuje jejich zobrazení v reálném měřítku 1:1.
  - `JigSpace`[9] (2.5b): nástroj umožňuje studovat složité technické struktury ve výrazně přehlednější a kognitivně přístupnější formě, a to bez použití obrázků nebo videí. Současně jej lze aplikovat k odborné prezentaci produktu se silnou orientací na technické parametry a detailní specifikace.

[6] Převzato z Beebom. Dostupné z https://beebom.com/ikea-ar-app-preview-multiple-furniture-items/

[7] Převzato z Porshe Newsroom. Dostupné z https://newsroom.porsche.com/en/2019/digital/porsche-augmented-reality-visualizer-app-car-configuration-17619.html

[8] AnatomyX: Full-body 3D anatomy education with augmented reality. Dostupné z https://www.anatomyx.com/

[9] JigSpace: 3D Communication for complex products. Dostupné z https://www.jig.com/

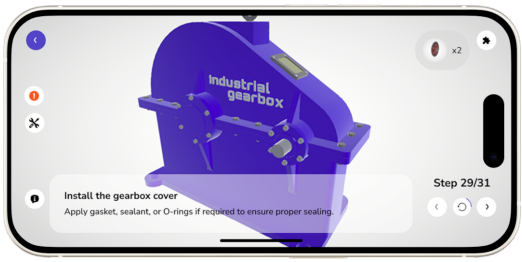


(a) AnatomyX[10]



(b) JigSpace[11]

Obrázek 2.5: Příklady aplikací z kategorie – školení (porozumění).



(a) Easemble[14]



(b) 3DQR[15]

Obrázek 2.6: Příklady aplikací z kategorie – školení (aktivity).

- **Školení (aktivity)** – Ve většině případů tato aplikace představuje soubor nástrojů podporujících osvojování technologických procesů, případně poskytuje zjednodušené varianty aplikací, které lze využít také při jejich integraci do výrobního procesu. Současně se vyznačuje relativně omezenou funkční výbavou, což však přispívá k její nízké hardwarové i softwarové náročnosti a nevyžadují specializované vybavení.

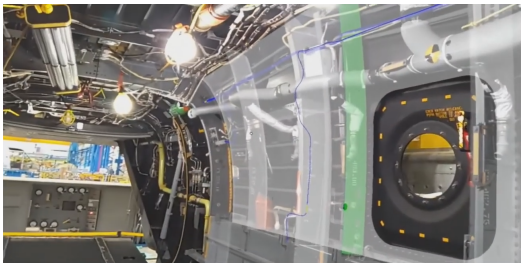
  Příklady aplikací:

  - `Easemble`[12] (2.6a): představuje soubor standardizovaných instrukcí, určených především jako přehledné montážní postupy pro různé typy modelů. Díky své vysoké míře flexibility a rozsáhlým možnostem parametrizace však může být využit také pro komplexnější aplikační scénáře, zahrnující například stavební realizace, údržbové procesy, školicí aktivity a další specializované činnosti.
  - `3DQR`[13] (2.6b): Sada nástrojů, která poskytuje vývojářům flexibilní a rozsáhlé spektrum funkcionalit. Díky tomu ji lze použít pro širokou škálu aplikačních scénářů. Díky podpoře Unity Assets je možné vytvářet velmi komplexní scény s následnou integrací jak IoT, tak i dalších navazujících služeb.

[10]Převzato z Medivis. Dostupné z https://www.medivis.com/anatomy-x
[11]Převzato z JigSpace. Dostupné z https://www.jig.com/case-studies/complex-3d-product-demos-with-triangle-waterquip
[12]Easemble: 3D Interactive Instruction Manuals. Dostupné z https://www.easemble.com/
[13]3DQR: The intuitive Augmented Reality app and software for Augmented Reality development and management. Dostupné z https://3dqr.de/en/software/
[14]Převzato z Easemble. Dostupné z https://www.easemble.com/how-it-works
[15]Převzato z 3DQR. Dostupné z https://3dqr.de/lernszenarien/



(a) Vuforia[17]



(b) Boeing AR[18]

Obrázek 2.7: Příklady aplikací z kategorie – podpora výroby.

- **Podpora výroby** – Z velké části lze programové prostředky zařazené do této sekce klasifikovat obdobně jako nástroje uvedené v předchozí kategorii. Na rozdíl od nich však tato skupina poskytuje podstatně komplexnější a hlubší podporu při jejich aplikaci v reálných výrobních procesech. Typicky využívají komplexnější metody, které zahrnují mimo jiné prvky, jako jsou: systematická kontrola kvality, sledování a řízení realizace postupných kroků, detailní analýza efektivity prováděných operací a další pokročilé funkce.

  Příklady aplikací:

  - `Vuforia` (2.7a): i když primárním produktem této společnosti je plugin určený pro integraci do AR aplikací, poskytují také širokou škálu nástrojů pro tvorbu a implementaci AR manuálů ve vlastním engine[16]. Vzhledem k vysoké funkční komplexitě a individuálnímu přístupu k vývoji nejsou výsledné aplikace veřejně dostupné. Jsou navrhovány a implementovány buď koncovými uživateli, nebo přímo vývojářským týmem společnosti. Řada z výše uvedených podniků využívá pro realizaci svých projektů plugin Vuforia.
  - `Boeing AR` (2.7b): uzavřená aplikace vyvinutá v rámci interního projektu integrace AR do výrobního procesu společnosti Boeing. Dále popsané principy se týkají i interních projektů jiných společností: aplikace je navrhována a implementována specificky podle požadavků dané organizace a pokrývá celý životní cyklus nasazení – od správy aktualizací a distribuce na cílová zařízení až po přímou integraci do konkrétních pracovišť. Disponuje rozsáhlým souborem funkcí, přičemž je současně maximálně optimalizována pro daný pracovní postup.

[16] Vuforia Engine 11. Dostupné z https://developer.vuforia.com/home

[17] Převzato z PTC. Dostupné z https://www.ptc.com/en/technologies/augmented-reality/solutions-for-manufacturing

[18] Převzato z YouTube: The Boeing Augmented Reality Kit (BARK), and its Use Throughout Airplane Manufacturing at Boeing. Dostupné z https://www.youtube.com/watch?v=1ctwGlWAkQw

Kapitola 3

Prostorový tracking v AR

Neoddělitelnou součástí tohoto projektu je řešení úloh detekce v systémech rozšířené reality. V této souvislosti se jeví jako účelné provést analýzu současných metod používaných v AR aplikacích a rovněž zhodnotit míru jejich využití a specifika jejich implementace v existujících softwarových řešeních.

3.1 Přístupy ke sledování polohy

Pro zajištění korektní interakce mezi virtuálním a reálným prostředím je nezbytné využívat metodou sledování trojrozměrných pozic prvků scény vzhledem k jejich fyzickému umístění. Konkrétně je vyžadován postup, který na základě posloupnosti obrazových snímků získávaných z kamery zařízení v reálném čase umožní odhadnout prostorovou polohu a orientaci kamery vůči scéně a současně provést jednoznačné prostorové mapování reálných bodů na body virtuální.

Zatímco v minulosti představoval tento problém jeden z klíčových limitujících faktorů rozvoje technologií rozšířené reality [3], postupný technologický pokrok vedl k formulaci a implementaci celé řady přístupů k jeho řešení. Dále budou vymezeny a podrobněji analyzovány dva hlavní přístupy [31]:

- Markerless – SLAM, GPS, senzory.
- Marker-based – referenční markery, 3D modely, obrázky.

Markerless

Přístup bez použití markerů vychází výhradně z analýzy vizuální informace obsažené ve scéně samotné, aniž by využíval jakékoli předem definované či fyzicky umístěné orientační body.

Nejčastější metodou v této kategorii je SLAM (Simultaneous Localization and Mapping) [45] – simultánní lokalizace a tvorba mapy. Systém paralelně vytváří mapu okolního prostoru a určuje v ní svou polohu, přičemž se spoléhá na charakteristické body obrazu – rohy, hrany, oblasti s vysokým jasovým gradientem. SLAM také využívá senzory zařízení ke snížení zátěže na zařízení, v případě, že se takové senzory nepoužívají, se algoritmus nazývá vSLAM (visual Simultaneous Localization and Mapping).

Algoritmy typu ORB nebo SIFT [21] extrahují deskriptory těchto bodů a porovnávají je mezi po sobě jdoucími snímky, čímž rekonstruují trajektorii pohybu kamery. Právě na

tomto principu jsou založeny ARCore a ARKit – detekce horizontálních a vertikálních rovin je výsledkem klasifikace již vytvořené mapy bodů scény.

Hlavním omezením tohoto přístupu je závislost na kvalitě textury povrchů – na homogenních nebo lesklých površích algoritmus ztrácí orientační body a stabilita sledování klesá. Kromě toho vytváření mapy v reálném čase klade vysoké nároky na výpočetní prostředky zařízení.

Marker-Based

Přístup založený na markerech se zásadně liší v tom, že systém má předem k dispozici vizuální referenci – objekt, který na obrazu hledá. V závislosti na typu reference se tento přístup dělí na několik variant:

- použití referenčních markerů (ArUco, AprilTag, QR kódy),
- rozpoznávání libovolných obrazů (image tracking),
- porovnání se předem známým 3D modelem objektu.

Ve všech případech je spojujícím principem to, že systém nezkoumá scénu obecně – hledá konkrétní známý vzor. Jakmile je vzor nalezen, jeho známá geometrie v kombinaci s parametry kamery umožňuje přesně vypočítat prostorovou polohu a orientaci.

Detekce je přitom výrazně rychlejší a stabilnější než u markerless přístupu a výpočetní zátěž je podstatně nižší. Hlavním omezením je nutnost předchozí přípravy – markery je potřeba vytisknout a rozmístit, 3D model je nutné předem vytvořit. Kromě toho částečné zakrytí markeru nebo jeho vypadnutí z zorného pole kamery přeruší sledování.

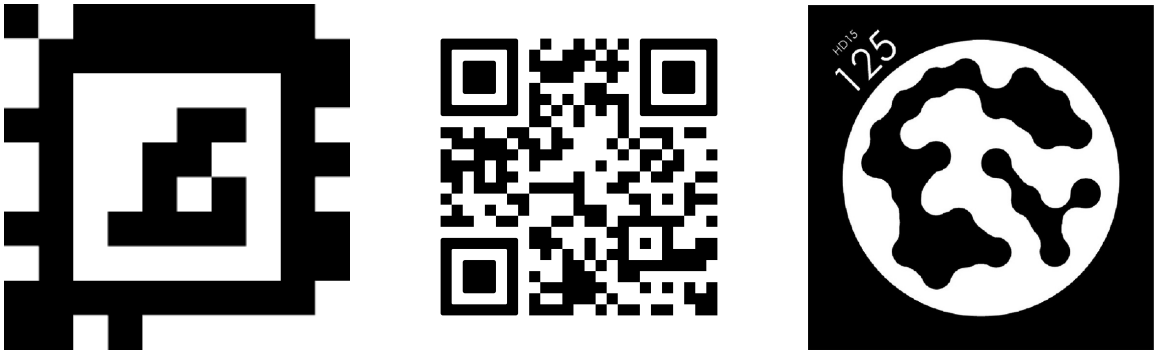
Kombinace přístupů

Oba přístupy se navzájem nevylučují – mnoho systémů je kombinuje. ARCore používá SLAM jako základní mechanismus pro určování polohy, ale zároveň podporuje image tracking pro konkrétní objekty.

V kontextu této práce, kde je vyžadováno přesné připojení interaktivních prvků k předem známým bodům fyzického objektu, je preferován přístup s využitím markerů – poskytuje jednoznačné prostorové ukotvení bez nutnosti předchozího skenování scény a je odolný vůči změnám osvětlení a textury okolního povrchu. Nicméně pro stabilizaci scény a snížení celkových výpočetních nákladů během provozu na pozadí lze také spustit SLAM.

3.2 Referenční markery (Fiducial markers)

Referenční markery představují uměle vytvořené vizuální vzory s přesně definovanou a známou geometrií 3.1, které slouží jako referenční body pro odhad prostorové polohy a orientace kamery k markeru. Na rozdíl od přirozených rysů scény využívaných v bezmarkerových přístupech jsou referenční markery konstruovány explicitně pro účely strojového zpracování obrazu. Jejich vnitřní struktura a kontrastní charakteristiky jsou optimalizovány tak, aby maximalizovaly spolehlivost detekce a jednoznačnost identifikace napříč širokým rozsahem světelných podmínek, pozorovacích úhlů a vzdáleností.



(a) AprilTag (b) QR kód (c) STag

Obrázek 3.1: Příklady referenčních markerů. QR kód se zakódovanou zprávou: ARManual-00000002. AprilTag 52h13 se zakódovaným ID 1. STag se zakódovaným ID 125 (Převzato z [5])

Struktura referenčních markerů

Typický referenční marker se skládá z několika funkčních vrstev:

- Vnější hranice, obvykle realizovaná jako černý rámeček na bílém pozadí, slouží k rychlé lokalizaci markeru v obraze prostřednictvím algoritmů pro detekci hran a extrakci kontur.
- Vnitřní binární matice kódu slouží k jednoznačné identifikaci markeru a současně obsahuje dostatečné množství redundantních informací umožňujících korekci chyb při detekci v podmínkách částečného zakrytí či geometrického a fotometrického zkreslení.

Samozřejmě existují také méně běžné typy markerů – jejich vnější kontura může být například kruhová, jako je tomu u `RUNE-tag`, případně jejich hranice netvoří jednoznačný čtvercový tvar, jak je tomu u `AprilTag 52h13`. Ne všechny markery navíc obsahují vnitřní binární matici s kódovaným identifikátorem. Tato odlišnost typicky vyplývá buď z konkrétního účelu, pro který byl marker navržen, nebo z estetických preferencí jejího autora.

Kalaitzakis M. et al. ve své studii [19] zaměřené na komparativní analýzu různých typů markerů nejen popisují jejich klíčové výhody, ale zároveň zmiňuje i zásadní charakteristiky jednotlivých markerů vyplývající z jejich vizuálních odlišností, a to konkrétně:

- Podle tvaru:
  - Čtverec
  - Kruh
- Podle barvy:
  - Monochromatický
  - Vícebarevný
  - Odstíny šedé
- Podle kódování:
  - Topologické
  - Frekvenční spektrum
  - Gradient

Rozvoj a srovnání referenčních markerů

Od okamžiku vzniku prvních podobných systémů byla vyvinuta již několikátá generace markerů, z nichž každá řešila nedostatky té předchozí.

ARTag [9] zlepšil odolnost vůči falešným detekcím ve srovnání s ARToolkit [22].

AprilTag [30] výrazně snížil procento chyb mezi markery a zlepšil detekci při malých velikostech markeru v záběru.

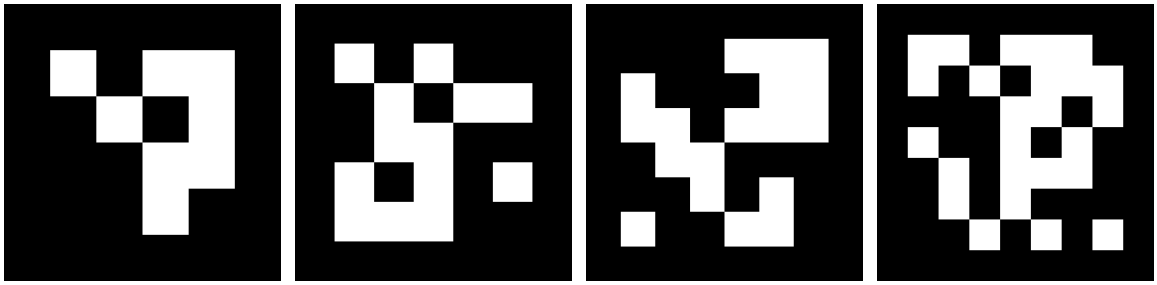
ArUco [12] nabídl flexibilní slovníkový přístup — velikost slovníku a počet bitů matice lze nastavit v závislosti na požadavcích úlohy, což umožňuje vyvažovat mezi počtem unikátních markerů a odolností vůči chybám.

Vývoj v této oblasti pokračuje až do současnosti a zahrnuje řadu dílčích směrů a variant. V dalším textu se zaměřím specificky na systém ArUco. Tato volba byla realizována na základě provedené analýzy [19], která komparativně hodnotila jednotlivé markery z hlediska jejich klíčových charakteristik, konkrétně:

- **Zvýšení vzdálenosti** – tato oblast představuje jednu z hlavních slabin systému ArUco. Se zvyšující se vzdáleností kamery od markeru, případně se zmenšující se fyzickou velikostí markeru, dochází k rychlejšímu poklesu detekční přesnosti ve srovnání s ostatními typy markerů.
- **Zvýšení úhlu náklonu** – v tomto testovacím scénáři dosahuje ArUco relativně dobrých výsledků. Vykazuje vysoké procento úspěšných detekcí, často srovnatelné nebo dokonce vyšší než konkurenční systémy.
- **Rozmazání při pohybu (rotace)** – v případě rotačního pohybu markeru vykazuje ArUco nejvyšší frekvenci úspěšných detekcí ze všech porovnávaných metod.
- **Rozmazání při pohybu (translace)** – při translačním pohybu markeru dosahuje ArUco výkonu srovnatelného se systémem STag a překonává ostatní hodnocené metody.
- **Výpočetní náročnost (paměť)** – jedná se o jednu z hlavních předností systému ArUco. Z hlediska paměťových nároků vykazuje ArUco relativně stabilní převahu nad ostatními metodami a jen výjimečně dosahuje horších výsledků než STag.
- **Výpočetní náročnost (CPU)** – také v oblasti procesorové výpočetní náročnosti představuje ArUco výraznou přednost. Dosahuje stabilních výsledků; v některých případech je překonáno systémem ARTag při detekci jednotlivého markeru, někdy se jeho výkon pohybuje na úrovni systému STag.

Každá charakteristika byla experimentálně ověřena za dvou světelných podmínek – při standardním osvětlení a při osvětlení se stínem. Současně byly použity dvě kamery s odlišnými technickými parametry. Měření probíhala v konfiguraci s jedním markerem i s dvěma až třemi markery. Tato kombinace experimentálních podmínek zvyšuje objektivitu výsledků.

Na základě provedených experimentů lze konstatovat, že ArUco patří mezi nejstabilnější markery i při zohlednění různých podmínek a současně vykazuje nízké nároky na výpočetní zdroje. Tato volba je však vykoupena sníženou robustností detekce ve větších pozorovacích vzdálenostech, což však v kontextu této práce nepředstavuje zásadní omezení.



(a) Šířka: 4 (b) Šířka: 5 (c) Šířka: 6 (d) Šířka: 7

Obrázek 3.2: Příklady různých ArUco slovníků, zahrnujících markery s různé velikosti vnitřní binární mřížky, ale všechny mají ID = 0.

3.3 Detekce markerů

Je vhodné podrobněji analyzovat proces detekce markeru, i když většina moderních knihoven skrývá svou vnitřní logiku za rozhraním typu „černá skříňka“. Podle konkrétní implementace mohou být některé kroky tohoto procesu volitelné, doplněné či zcela vynechané. Proto má smysl postup systematicky rozebrat, jelikož následující kapitoly se zaměří na návrh a implementaci vlastního detekčního algoritmu.

Samozřejmě existuje celá řada implementací algoritmů pro detekci markerů, z nichž některé jsou stále ve fázi aktivního vývoje a průběžně vznikají nové přístupy. V tomto textu bude nicméně popsán obecný metodický rámec, se zaměřením na postup využívaný při detekci markerů `ArUco` za použití knihovny `OpenCV` [40].

Počáteční nastavení

Před zahájením procesu rozpoznávání je nezbytné inicializovat vstupní parametry použitých algoritmů. Většina těchto parametrů bude detailně popsána v následujících částech, nicméně jedním z klíčových kroků je volba slovníku 3.2, který bude v průběhu zpracování používán.

Autoři [12] zmiňují, že zvětšení dimenze vnitřní binární mřížky markeru má přímý vliv na jeho spolehlivost při detekci a identifikaci. Větší počet bitů umožňuje vytvořit slovník s vyšší minimální vzájemnou Hammingovou vzdáleností $\hat{\tau}$, což vede k několika zásadním výhodám:

- zvýšení minimální vzájemné vzdálenosti mezi markery, což snižuje pravděpodobnost chyby detekce v rámci jednoho slovníku. Autoři tuto hodnotu definují jako

$$\hat{\tau}(\mathcal{D}) = \min\left\{\min_{m_i \in \mathcal{D}}\{\mathcal{S}(m_i)\},\ \min_{m_i \neq m_j \in \mathcal{D}}\{D(m_i, m_j)\}\right\} \tag{3.1}$$

  kde $\mathcal{S}(m_i)$ je vlastní vzdálenost markeru vůči jeho rotacím a $D(m_i, m_j)$ je rotačně invariantní Hammingova vzdálenost mezi dvěma markery.

- zvýšení počtu detekovatelných a opravitelných chyb, protože maximální počet bitů, které lze opravit, se vypočítají ze vzorce

$$\left\lfloor \frac{\hat{\tau} - 1}{2} \right\rfloor \tag{3.2}$$

takže větší markery umožňují robustnější identifikaci i při šumu, rozmazání nebo částečné okluzi. Bylo spočítáno, že pro slovník o rozměrech 6×6 a 30 markerů je $\hat{\tau} = 12$, z čehož vyplývá, že lze opravit až 5 chybných bitů.

- větší počet bitových přechodů ($0 \to 1, 1 \to 0$), snižuje riziko falešné detekce způsobené texturami v prostředí. Autoři explicitně uvádějí, že markery s nízkým počtem přechodů mohou být snadno zaměněny za homogenní oblasti obrazu.
- vyšší teoretické maximum vlastní vzdálenosti markeru, které roste přibližně podle vztahu

$$\mathcal{S}_n^* = 2 \left\lceil \frac{4C}{3} \right\rceil \tag{3.3}$$

kde n je počet bitů na jedné straně. To znamená, že například marker 7×7 má výrazně vyšší možnou odolnost vůči chybám než marker 4×4.

Zvětšení dimenze vnitřní binární mřížky markeru s sebou nevyhnutelně přináší i významné nevýhody – od relativně marginálních v kontextu současných výpočetních systémů, jako je zvýšená výpočetní náročnost, až po zásadní, například degradaci kvality a stability rozpoznávání markerů se zvyšující se snímací vzdáleností či při zmenšování jejich zobrazené velikosti. Tato degradace patří k hlavním nedostatkům ArUco. Uvedené jevy jsou determinovány jak kvalitou vstupního obrazu, tak i přítomnými optickými a geometrickými zkresleními. Řada studií [24, 20] proto tuto problematiku označuje za klíčové omezení dané třídy markerů a hlavním cílem návrhu nových typů markerů je právě eliminace těchto nedostatků.

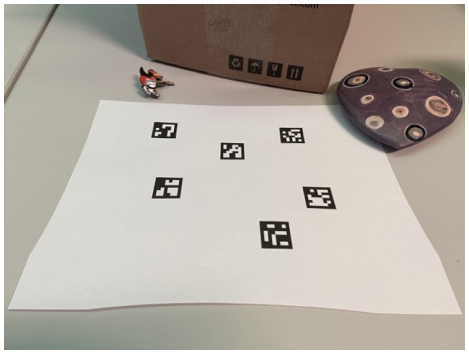
Předzpracování obrazu

Existuje celá řada metod a postupů, jimiž lze kvalitu obrazu zvýšit, například:

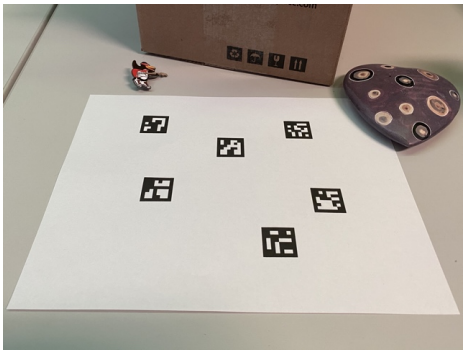
- **Převedení na monochromatický obraz** – většina markerů nekóduje informace pomocí barvy, takže se jí lze zbavit. Ve většině případů to neovlivní algoritmus díky následujícímu kroku, nicméně v případě častého přenosu dat lze dosáhnout snížení objemu zpracovávaných informací.
- **Odstranit zkreslení kamery** – každá kamera vnáší radiální a tangenciální zkreslení, kvůli nimž se čáry, které měly být původně rovné, deformují, a v důsledku toho algoritmus detekce kontur může marker ignorovat.
- **Potlačení šumu** – při nízké úrovni osvětlení nebo u nekvalitních kamer se v obraze typicky vyskytuje zvýšená úroveň šumu, která při následném zpracování vede ke vzniku arteficiálních hran nebo zhoršuje spolehlivost detekce skutečně existujících hran.
- **Vyhlazování obrazu** – je podobná situace jako potlačování šumu, také umožňuje při správném použití získat stabilnější výsledky, avšak při příliš velkém vyhlazení algoritmus může nerozpoznat hranice, nebo může považovat přímé čáry za zakřivené.

Samozřejmě nejde o povinný krok, nicméně je zásadní zohlednit specifické požadavky, s nimiž se uživatel může setkat. Nevhodná či naopak vhodná konfigurace totiž může jak významně zvýšit stabilitu detekce, tak ji i podstatně snížit, a tím současně ovlivnit i celkový výkon systému.

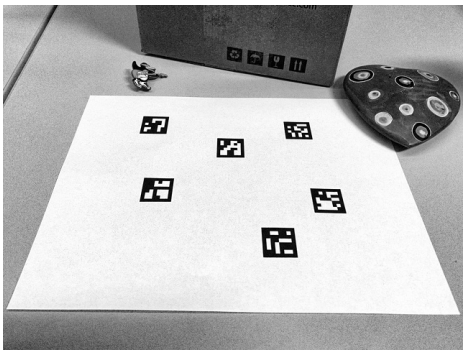
Na obrázku 3.3 je znázorněna postupná transformace obrazu od původní, zkreslené podoby až po výslednou verzi převedenou do odstínů šedi.



(a) Příklad obrázku se zkreslením



(b) Po odstranění zkreslení



(c) Po převodu do odstínů šedé

Obrázek 3.3: Postupný proces přípravy obrazu, který obsahuje dvě fáze: odstranění zkreslení kamery (z `a` do `b`), což lze poznat podle zkreslení okrajů listu, a převedení obrazu do odstínů šedi (z `b` do `c`). Převzato a zpracováno z [40].

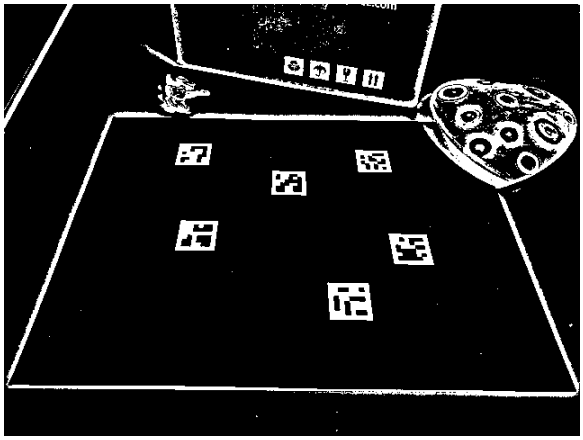
Adaptivní prahování

Inicializační fáze rozpoznávacího algoritmu přebírá předzpracovaný obraz z předchozí etapy. Následuje operace obdobná převodu do odstínů šedi, avšak s použitím adaptivního prahování, kdy je hodnota prahu lokálně určována podle osvětlení v jednotlivých oblastech obrazu. Tím se efektivně minimalizují negativní dopady nerovnoměrného osvětlení, stínů, odlesků, jasových gradientů a dalších fotometrických jevů na kvalitu segmentace.

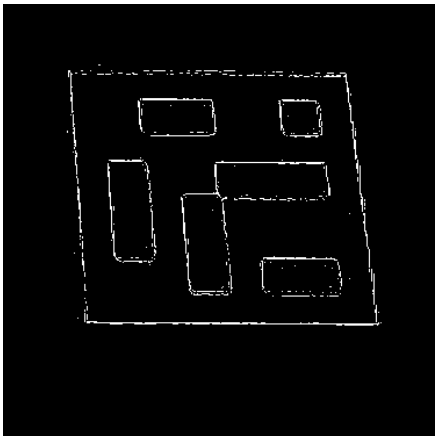
Hlavní myšlenka metody spočívá ve výpočtu kontrastu každého pixelu vzhledem k jeho sousedům, což umožňuje poměrně přesnou detekci a zvýraznění hran. V `OpenCV` se pro tuto úlohu obvykle používá jeden ze dvou algoritmů:

- `Adaptive Mean Thresholding` – prahová hodnota se bere jako průměrná hodnota okolních pixelů. Je rychlý na zpracování, ale nedává optimální výsledek.
- `Adaptive Gaussian Thresholding` – práh se bere jako vážená střední hodnota mezi sousedními pixely, která je křížovou korelací s Gaussovým oknem. Která zase potlačuje vysokofrekvenční obsah obrazu a ponechává pouze pomalé prostorové změny. Myšlenkou je, že pomalé změny jsou většinou způsobeny změnami osvětlení v obrazu, zatímco vysoké frekvence jsou způsobeny hranami.

Tato operace poskytuje kontrastní obraz se zřetelně definovanými hranicemi markerů (viz obr. 3.4a). Při nevhodné volbě parametrů algoritmu však mohou vznikat markery nevhodné k použití (viz obr. 3.4b). Při manuálním nastavování parametrů je nutné zohlednit provozní



(a) Příklad výsledku



(b) Příklad „rozbitého“ markeru

Obrázek 3.4: Výsledek práce adaptivního prahování (a). A také příklad chyby v práci algoritmu (b). Převzato z [40].

podmínky cílové aplikace – průměrnou úroveň osvětlení scény, očekávané rozměry markeru a prostorové rozlišení kamery.

Detekce kontur a filtrace čtyřúhelníků

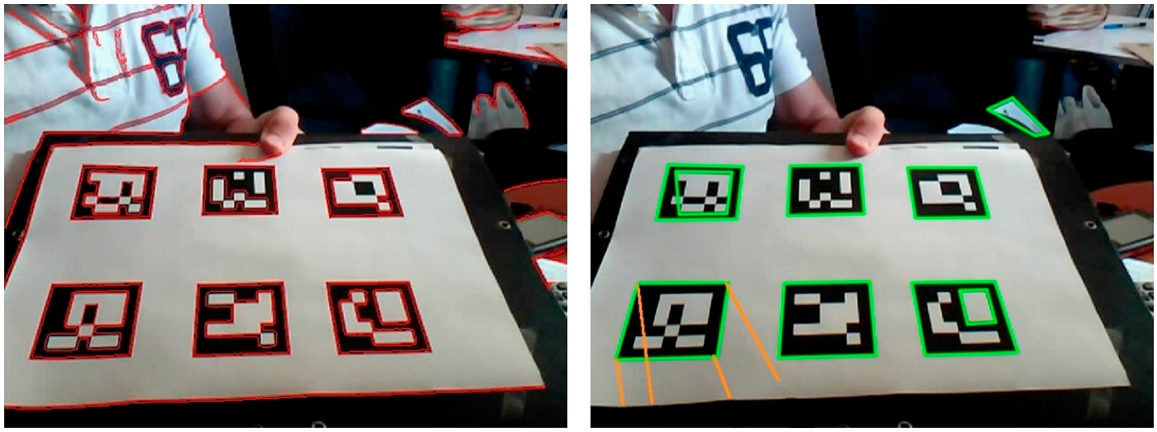
Po provedení binarizace obrazu dochází k výraznému zjednodušení procesu detekce kontur. V této fázi je realizováno primární oddělení kandidátních markerů od pozadí, textur a artefaktů, které mohou vznikat v důsledku šumu nebo následného zpracování obrazu.

K vyhledávání kontur se obvykle používají algoritmy typu Suzuki–Abe [1], které procházejí binární obraz a generují množinu uzavřených křivek. Tento přístup je vhodný i proto, že ArUco marker vždy obsahuje vnější černý čtverec na bílém pozadí.

Dalším krokem je zjednodušení těchto křivek s cílem odstranit nadbytečné informace a urychlit zpracování. K tomu lze využít algoritmy podobné aproximaci křivek pomocí algoritmu Douglas–Peucker [18], které redukují počet bodů při zachování celkového tvaru a geometrických vlastností kontury.

Na základě provedených operací jsou získány body vhodné pro následné zpracování, jak je ilustrováno na obrázku 3.5 (zvýrazněno červeně). S ohledem na geometrické charakteristiky markeru je však možné zredukovat počet kandidátních markerů, a proto jsou dále definována následující pravidla:

- Marker má čtvercový tvar, a proto musí mít jeho obrys přesně čtyři vrcholy.
- Vrcholy obrysu musí být uspořádány sekvenčně po směru nebo proti směru hodinových ručiček, aby byla zachována konzistentní orientace polygonu.
- Lze stanovit minimální přípustnou plochu obrysu, jelikož při přílišném zmenšení nedochází k dostatečnému rozlišení binární matice, tím není možné spolehlivě určit identifikátor ani určit rotaci markeru. Současně se tímto způsobem odfiltrují obrysy odpovídající šumu.
- Poměr stran by se měl blížit tvaru čtverce, s odchylkou vyplývající z perspektivní deformace.



Obrázek 3.5: Ukázka výsledků detekce kontur (zvýrazněno červeně) a následné aproximace polygonálních tvarů včetně eliminace nevhodných či nerelevantních kontur (zvýrazněno zeleně). Převzato z [12].

- Vnitřní úhly obrysu by měly být přibližně 90°, avšak jsou přípustné určité odchylky dané zkreslením a nepřesností detekce.

Po aplikaci výše uvedených filtračních pravidel je výsledná množina omezena pouze na nejpravděpodobnější kandidáty markerů, jak je ilustrováno na obrázku 3.5 (zvýrazněno zeleně).

Perspektivní korekce

V důsledku předchozího kroku byly získány souřadnice kandidátních markerů na obraze. Než však lze přistoupit k přečtení binární matice a k verifikaci, že daný objekt skutečně odpovídá hledanému markeru, je nezbytné kompenzovat zkreslení způsobené perspektivní projekcí.

Čtvercový marker umístěný v trojrozměrném prostoru a snímaný kamerou se na obrazové rovině jeví jako perspektivně zkreslený čtyřúhelník. Protilehlé strany už nejsou rovnoběžné a vnitřní úhly nejsou pravé, což je důsledek centrální projekce kamery. K rekonstrukci původního čtverce je proto nutné použít transformační zobrazení, které tento čtyřúhelník „rozvine“ zpět do čtverce. Toto zobrazení se nazývá homografie.

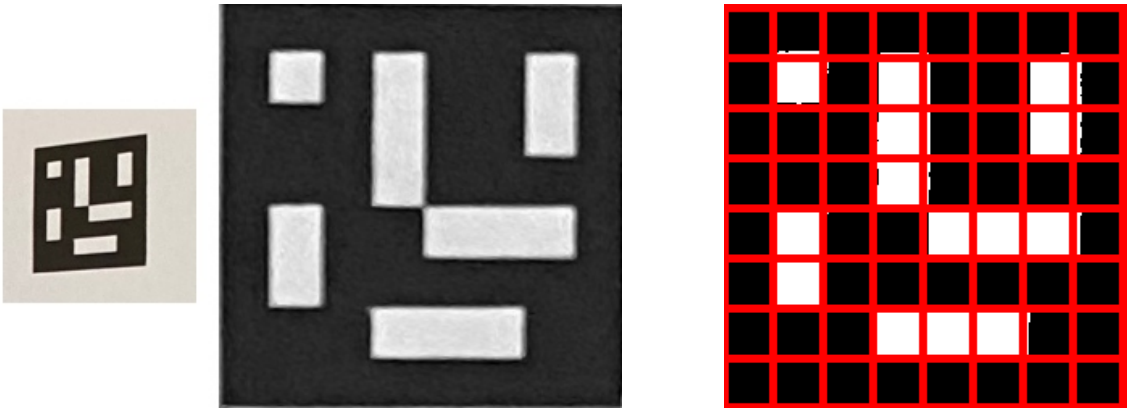
Homografie je reprezentována maticí rozměru 3×3, kterou označíme H, a která v našem případě jednoznačně určuje korespondující body mezi čtyřmi rohovými body obrysu a čtyřmi rohovými body ideálního čtvercového šablonového vzoru. Současně musí matice H splňovat následující vztah:

$$p' = Hp \tag{3.4}$$

kde:

- p – bod na obrázku (v perspektivně deformovaném markeru).
- p' – odpovídající bod v „rozvinutém“ čtverci.

V našem případě, vzhledem k použití čtvercových markerů, nepředstavuje toto zobrazení složitý úkol. Při použití jiných markerů nebo při aplikaci tohoto zobrazení na jiné objekty však vzniká řada komplikujících faktorů, kvůli nimž někteří autoři [8] zkoumají tuto problematiku mnohem podrobněji.



(a) Perspektivní korekce (b) Jednotlivé bity

Obrázek 3.6: Příklad markeru s perspektivním zkreslením, následnou projektivní transformací do tvaru ideálního čtverce (a) a s vyznačením jednotlivých bitů v odpovídající binární matici (b). Převzato z [40].

Po výpočtu homografie se provede promítnutí původního obrazového fragmentu do čtverce s pevným rozměrem, například 50×50 nebo 100×100 pixelů. Tím vznikne normalizovaný obraz markeru, vhodný pro binarizaci a analýzu. Jak to můžeme vidět na obrázku 3.6.

Dekódování binární matice

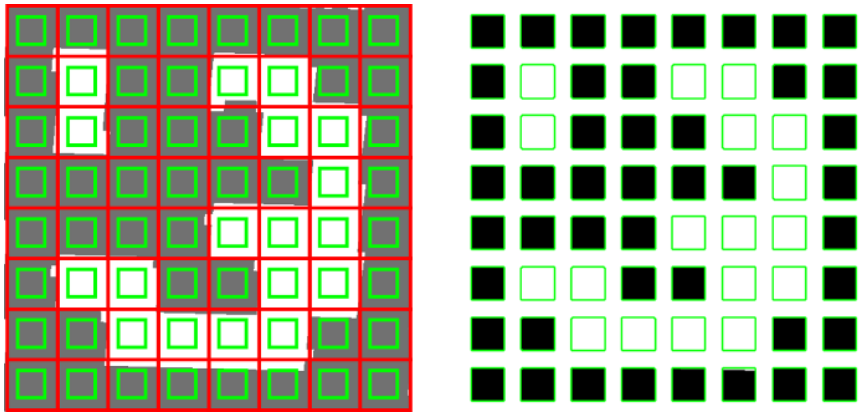
Nyní je potřeba dekódovat obraz získaný v předchozím kroku. Právě teď definitivně oddělujeme skutečné markery od kandidátů na markery. K tomu musíme pro každý z markerů provést posloupnost kroků:

1. Rozdělení na buňky – protože normalizace markeru vzhledem k perspektivní transformaci může vytvářet artefakty a nemusí být úplně přesná. Například část bílých pixelů bude „zasahovat“ do černých, jak je vidět na obrázku 3.6. Nejprve je třeba vzít pouze centrální oblast každého bitu v binární matici markeru, aby se zvýšila přesnost. Po tomto kroku získáme mnohem přesnější obraz, jak je patrné na obrázku 3.7.

2. Určení jednotlivých bitů – dalším krokem bude analýza každého zvoleného sektoru. Za účelem zvýšení přesnosti a minimalizace vlivu artefaktů vzniklých při transformaci se z dané oblasti vypočítá průměrná hodnota pixelů. Tímto postupem je následně odvozena binární matice obsahující hodnoty 1 a 0.

3. ArUco vždy obsahuje v binární matici vnější černý rámeček – lze jej ověřit kontrolou přítomnosti bílých pixelů (hodnot 1 v našem kontextu). Pokud jsou v této oblasti detekovány jakékoli bílé pixely, je daný marker bezprostředně vyřazen z dalšího zpracování. Právě v této fázi je vyřazena většina objektů, které byly rozpoznány, protože mají obdélníkový tvar.

4. Marker může být vůči kameře orientován rotacemi (0, 90, 180, 270 stupňů) – proto algoritmus generuje čtyři varianty binárního kódu, z nichž každá odpovídá jedné z možných prostorových orientací.



Obrázek 3.7: Příklad optimalizace procesu čtení buněk markeru. Převzato z [40].

5. Poté se provede porovnání se slovníkem, který byl nastaven dříve – jak již bylo zmíněno, kvůli specifikům implementace dokážeme rozpoznat marker obsahující až 5 chybných bitů. Proto se porovnání provádí pomocí výpočtu Hammingovy vzdálenosti $\hat{\tau}(\mathcal{D})$ z 3.1. V případě, že tato vzdálenost překročí přípustný práh, je marker zamítnut.

6. A nakonec můžeme odvodit jedinečný identifikátor – na základě nejbližší shody bitového vzoru ve slovníku. Následně určíme jeho orientaci vůči kameře, což nám umožní jednoznačně stanovit jeho polohu na obrazovce.

Determinace polohy na zobrazovací ploše

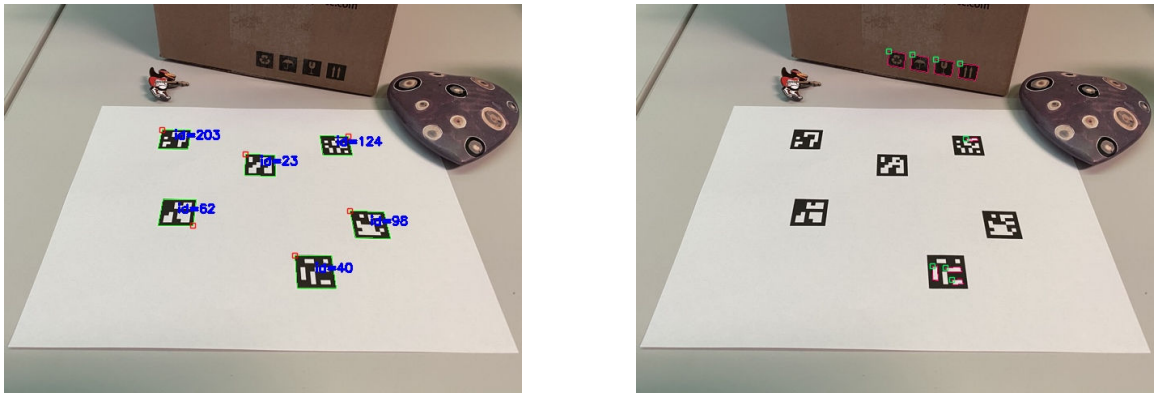
V této fázi jsme získali veškeré nezbytné informace o markeru. Následně se proces detekce přesouvá do etapy interpretace získaných dat. Na základě předchozích kroků budeme nadále pracovat se třemi klíčovými prvky:

- Obrysy markerů – s výjimkou těch, které byly odfiltrovány v průběhu dekódování. Každý marker je definován čtveřicí vrcholů, jež představují výstup algoritmického zpracování. Před samotnou analýzou je však nezbytné tyto obrysy jednoznačně přiřadit k dekódovaným informacím o jednotlivých markerech.

- Jednoznačný identifikátor markeru – který může být vyžadován při následném zpracování dat mimo rámec daného algoritmu.

- Orientace markeru – získaná na základě jeho jednoznačného identifikátoru, bude využita k přesné detekci a přiřazení jeho rohů. Tato informace je zásadní pro následné stanovení prostorových souřadnic markeru v trojrozměrném prostoru.

Jako výsledek této fáze získáme dvě množiny dat – rozpoznané markery a markery, které byly v průběhu detekce vyřazeny, jak je vidět na obrázku 3.8. Druhou množinu dat lze použít jako ladicí informaci během nastavování algoritmu, případně se lze pokusit provést ještě jednu iteraci s jinými parametry ve snaze je rozpoznat. Rozpoznané markery pak obsahují hlavní informace potřebné pro jejich další zpracování: **jednoznačný identifikátor** a **souřadnice každého ze čtyř rohů na zobrazovací ploše**.



(a) Rozpoznané markery (b) Vyloučené kandidátní markery

Obrázek 3.8: Příklad výsledku algoritmu rozpoznávání markerů zahrnuje rozpoznané markery (a) se zřetelně vyznačenými hranicemi, prvním rohem a identifikátorem, a také sadu kandidátních markerů (b), které byly v průběhu činnosti algoritmu vyřazeny. Převzato z [40].

Sub-pixelové zpřesnění rohů

Jednou z možností zlepšení konečného výsledku je využití sub-pixelového mapování, poprvé představené Atkinsonem [2]. Protože pracujeme s rastrovými obrazy konečné velikosti, jsou úhly z aproximace kontury určeny jen s přesností na jeden pixel. Při větší vzdálenosti markeru od kamery, a tedy jeho relativním zmenšení vzhledem k rozlišení obrazu, může tato omezená přesnost výrazně ovlivnit odhad jeho prostorové polohy, zejména určení vzdálenosti od kamery.

Existuje velké množství algoritmů, které tento princip implementují. V následující části se však zaměříme na jeden z nejjednodušších z nich, abychom názorně objasnili obecný princip jeho fungování.

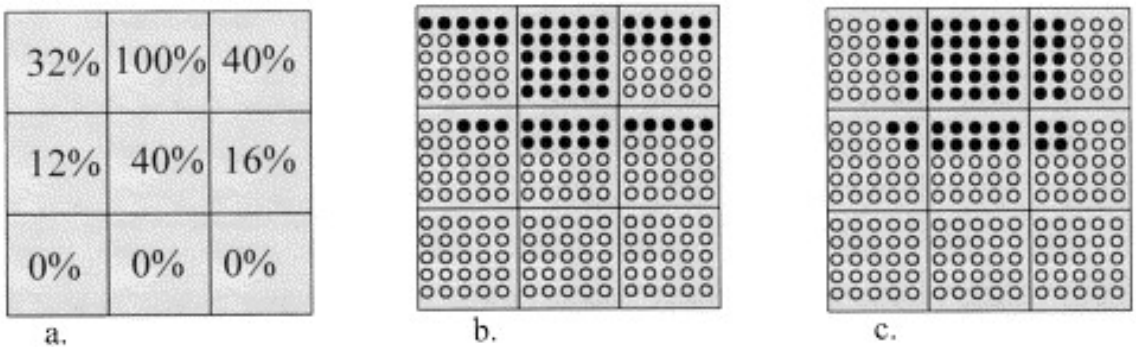
Na obrázku 3.9 je znázorněn průběh daného procesu. Uvažujme na vstupu obraz o rozlišení 3×3 pixelů v odstínech šedi, přičemž hodnota jednotlivého pixelu je určena mírou jeho blízkosti k zcela černé barvě.

1. Nejprve získáme hodnoty jednotlivých pixelů v matici.

2. Nastavíme velikost rozdělení, v našem případě každý pixel obsahuje 25 subpixelů. Také určíme počet aktivních pixelů v souladu s hodnotou, kterou jsme získali v předchozím kroku.

3. Následně, za účelem zachování prostorové koherence a zároveň pro vyhlazení a udržení kontinuity výsledné struktury, následuje fáze rozložení subpixelů, která sama o sobě představuje nejnáročnější krok celého postupu.

V důsledku tohoto postupu lze v konečné struktuře získat souřadnice vrcholů markeru ne jako celočíselné hodnoty, ale jako reálná čísla s plovoucí desetinnou čárkou. Tím se zvyšuje numerická přesnost následné determinace trojrozměrné polohy markeru vzhledem ke kameře.



Obrázek 3.9: Princip sub-pixelového mapování na příkladu jednotlivého obrázku o velikosti 3×3 pixelů s následným rozdělením každého z nich na 25 sub-pixelů. Převzato z [49].

3.4 Odhad polohy markeru

Po úspěšné detekci markeru ve scéně, následném dekódování jeho binární matice a přesném určení obrazových souřadnic jeho rohů disponuje systém veškerými nezbytnými informacemi pro přechod k výpočtu jeho prostorové polohy.

V kontextu rozšířené reality (AR) je pojem „poloha“ chápán jako šest stupňů volnosti (6 Degrees of Freedom, 6DoF), tj. trojrozměrná translační poloha markeru vůči kameře a jeho orientace v prostoru. Tento úkol je formulován jako klasický problém Perspective-n-Point (PnP), jejímž cílem je určit takovou prostorovou transformaci, která zobrazí známé trojrozměrné souřadnice rohů markeru na jejich odpovídající dvourozměrné souřadnice pozorované v obrazové rovině [15].

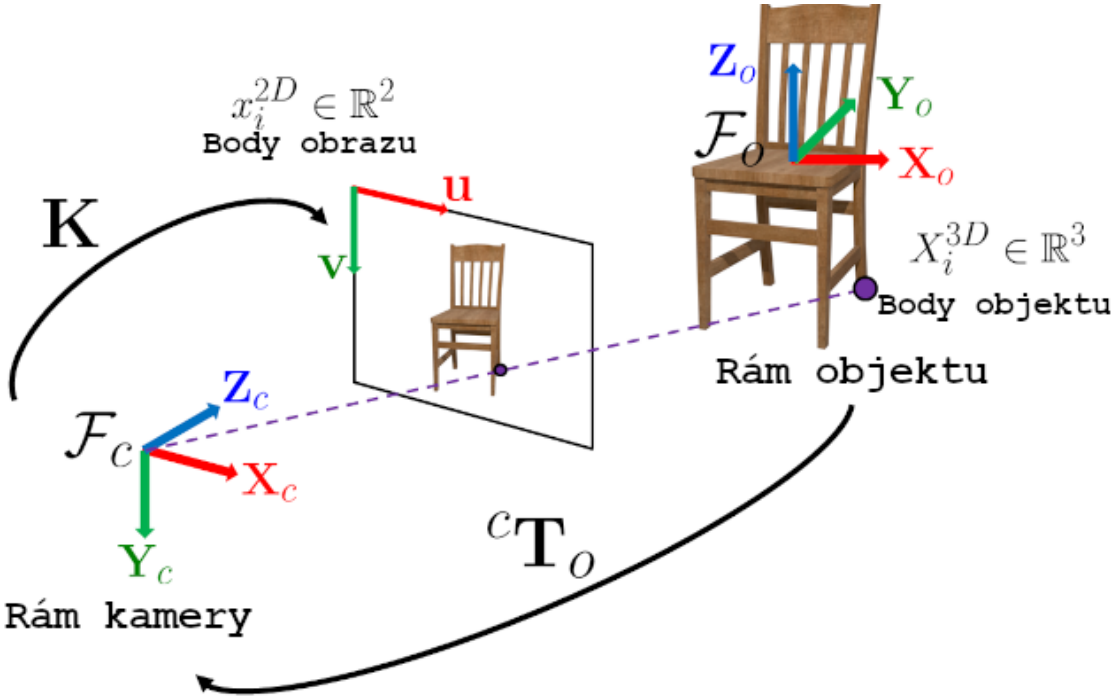
Geometrický aspekt úlohy

Analogicky jako u ostatních dříve popsaných algoritmů existuje pro daný problém celá řada přístupů, které jsou navrženy a optimalizovány s ohledem na specifické typy úloh. Ve většině prací se autoři snaží řešit problém PnP společně s úlohou detekce příslušných obrazových bodů. Tyto metody mohou mít formu jak jednokrokového postupu [34], nebo jako dvoukrokové přístupy, které nejprve samostatně detekují charakteristické body v obraze a následně na základě těchto detekovaných bodů odhadují jejich prostorovou polohu [46].

V našem případě již byly vrcholy hledaných markerů identifikovány, a proto se nyní můžeme zaměřit na obecný přístup k řešení daného problému. Jak je vidět na obrázku 3.10, každý bod objektu vyjádřený ve svém lokálním rámci je nejprve transformován do kamerového souřadného systému a následně promítnut do obrazové roviny. Tento vztah lze zapsat jako

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \Pi {}^{c}T_{W} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \tag{3.5}$$

kde K představuje matici vnitřních parametrů kamery, Π je perspektivní projekce a ${}^{c}T_{W}$ je rigidní transformace popisující polohu a orientaci markeru vzhledem ke kameře. Tato



Obrázek 3.10: Schéma ukazuje, jak jsou body objektu definované v jeho lokálním rámci převáděny do obrazové roviny kamery pomocí prostorové transformace. Převzato a přeloženo z OpenCV Documantation[1].

transformace má tvar

$$ {}^cT_W = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{3.6} $$

přičemž R je rotační matice a t je translační vektor. V prostředí OpenCV jsou tyto dvě složky reprezentovány jako `rvec` a `tvec`, kde `rvec` je rotační vektor ve formátu Rodriguesovy parametrizace a `tvec` je trojrozměrný translační vektor. Oba dohromady jednoznačně určují prostorovou pozici markeru vůči kameře. Model předpokládá, že zkreslení objektivu bylo předem odstraněno, což je nezbytné pro správnou funkci PnP algoritmů.

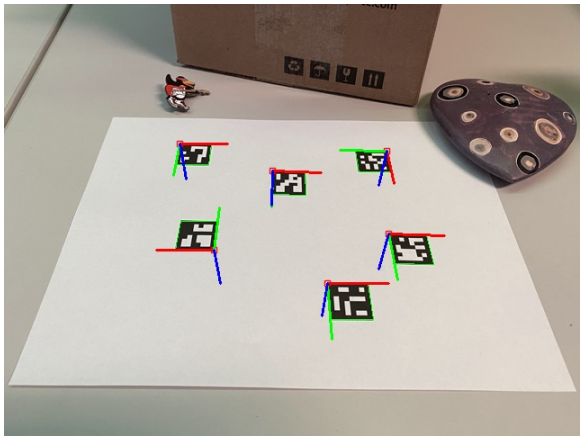
Model markeru v trojrozměrném prostoru

Jak již bylo uvedeno, naším cílem je přiřadit vrcholy markeru detekované ve scéně k odpovídajícím vrcholům jeho trojrozměrného modelu. Nabízí se tedy otázka, co přesně je míněno 3D modelem markeru.

Vzhledem k tomu, že marker má tvar plochého čtverce, je jeho lokální 3D geometrie triviální. Předpokládáme, že leží v rovině $Z = 0$ a že jeho vrcholy mají pevně dané souřadnice, odvozené z fyzických rozměrů markeru. Uvažujme pro ilustraci marker se stranou délky s, potom lze souřadnice jeho vrcholů zapsat následovně:

- $(-s/2, -s/2, 0)$

[1] OpenCV: Camera Calibration and 3D Reconstruction. Dostupné z https://docs.opencv.org/4.12.0/d9/d0c/group__calib3d.html/



Obrázek 3.11: Příklad vizualizace virtuálních objektů v obraze v souladu se 6DoF rozpoznaného markeru. Převzato z [40].

- $(s/2, -s/2, 0)$
- $(s/2, s/2, 0)$
- $(-s/2, s/2, 0)$

Získané body lze přiřadit k dříve určeným úhlům na obrázku. Nesprávně zvolená fyzická velikost markeru sice nebrání jeho detekci, ale může zhoršit přesnost navazujících výpočtů. Geometrický model lze sestavit samostatně pro každý marker, takže není nutné zachovávat jednotnou velikost markerů v celé sadě.

Řešení úlohy PnP

Nyní můžeme přistoupit k určení matice R a vektoru t. K tomuto účelu lze využít některý z existujících algoritmů. V následující části se podíváme na některé z nich:

- **EPnP** (Efficient Perspective-n-Point Camera Pose Estimation) – rychlá metoda, dobře vhodná pro zpracování v reálném čase a odolná vůči šumu [25].
- **Iterativní PnP** – je přesnější, používá optimalizaci k minimalizaci reprojekční chyby, což představuje součet druhých mocnin vzdáleností mezi pozorovanými projekcemi „obrazkových bodů“ a odpovídajícími projektovanými „objektovými body“. Na základě metody Levenberg-Marquardtovy optimalizace [26].
- **IPPE** (Infinitesimal Plane-Based Pose Estimation) – metoda navržená speciálně pro planární objekty, poskytující velmi stabilní výsledky zejména při malých úhlech náklonu markeru; její varianta **IPPE_SQUARE** dále optimalizuje výpočet pro čtvercové markery a dosahuje vysoké přesnosti při použití čtyř koplanárních bodů [7].
- **AP3P** (Algebraic Perspective-Three-Point) – moderní algebraická metoda řešící P3P úlohu efektivněji a stabilněji než klasické přístupy; poskytuje rychlé a přesné řešení při minimálním počtu bodů [23]. Její implementace v OpenCV však vyžaduje na vstupu 4 body.

Výsledná poloha a její využití

Výsledkem procesu je jednoznačně určená prostorová poloha markeru vůči kameře, daná rotační maticí R a translačním vektorem t. Ty tvoří homogenní transformační matici ${}^{c}T_{W}$, která převádí body z lokálního rámce markeru do kamerového souřadného systému a poskytuje úplnou informaci o jeho orientaci a vzdálenosti, klíčovou pro většinu AR úloh.

Takto určená pozice markeru umožňuje vykreslení virtuálních objektů do scény (viz obrázek 3.11), jejich stabilní ukotvení k fyzickým povrchům i interakce mezi reálnými a digitálními prvky.

Transformace ${}^{c}T_{W}$ se dále využívá pro predikci pohybu, filtraci šumu, kompenzaci nestability kamery a synchronizaci více senzorů v komplexních AR systémech. Odhad pozice markeru tak propojuje detekci vizuálních prvků s prostorovou interpretací scény a umožňuje vytvářet konzistentní a přesvědčivé AR zážitky.

Kapitola 4

Návrh systému pro práci s AR manuály

Moderní přístupy k práci s technickou dokumentací se stále častěji setkávají s omezeními tradičních forem – statičností, obtížnou orientací, nedostatkem kontextu a vysokou kognitivní zátěží pro uživatele. Jak bylo ukázáno v teoretické části práce, technologie rozšířené reality umožňují tato omezení odstranit díky prostorové registraci, vizuálnímu propojení instrukce se skutečným objektem a interaktivnímu zpřístupnění informací přímo v místě jejich použití.

Následně bude s ohledem na aktuální stav trhu s aplikacemi rozšířené reality představen návrh koncipované aplikace.

4.1 Cíle systému a kontext použití

Pro vypracování plánu vývoje aplikace je v počáteční fázi nutné stanovit její cílové záměry, cílovou skupinu, pro kterou je určena, a také typické scénáře a podmínky provozu, v jejichž rámci bude aplikace používána.

Především je vhodné provést analýzu stávajících aplikací uvedených v části 2.4 a také zvážit koncepce představené v řadě dříve zmíněných studií a závěry, k nimž autoři dospěli.

Problémy existujících řešení

Může vyvstat otázka, z jakého důvodu existuje tak rozsáhlé spektrum nástrojů a proč dochází k jejich neustálému rozšiřování o nové varianty, jestliže by teoreticky bylo možné využít některá z výše uvedených univerzálních řešení?

V současné době se největší rozdíly mezi existujícími nástroji projevují v následujících oblastech:

- **Způsob detekce** – v současné době většina aplikací používá detekci roviny pro následné umisťování objektů na ni. Tento přístup však neumožňuje správně kombinovat fyzický a virtuální svět v předem určených místech. Další nejrozšířenější metodou je detekce markeru, která už umožňuje propojit polohu v reálném světě s AR prostorem. Způsob realizace tohoto typu detekce se však výrazně liší aplikaci od aplikace, což bude podrobněji rozebráno v následující kapitole.
- **Složitost vývoje** – čím větší funkcionalitu má vývojový nástroj a čím je flexibilnější, tím složitější se stává proces vývoje nových aplikací. Z tohoto důvodu některé spo-

lečnosti nabízejí vlastní vývojáře pro rozvoj projektů, jak to dělají například Vuforia nebo 3DQR. V důsledku toho se podpora a rozvoj podobných projektů stávají velmi nákladným procesem.

- **Složitost přidávání nových manuálů** – většina aplikací používá složité způsoby přidávání nových manuálů, kvůli čemuž není možné využít člověka, který je zodpovědný za tvorbu tištěných manuálů, také pro jejich integraci do rozšířené reality. To vede ke zvýšení nákladů na podporu těchto funkcí a ke zvýšené poptávce po málo rozšířených specialistech v této oblasti. Mnoho aplikací navíc používá 3D modely jako základ manuálu, což také vede k zvýšení nákladů na tvorbu manuálů.

- **Sada funkcí** – zatímco jednoduché aplikace nevyžadují hluboké studium pro implementaci nových návodů, některým společnostem může chybět potřebná funkcionalita. U rozsáhlejších aplikací naopak pozorujeme opačnou situaci – počet funkcí je tak velký, že složitost vývoje roste neúměrně ve srovnání s přínosem širokého spektra funkcí, případně aplikace ztrácí na optimalizaci. V důsledku toho se velké společnosti snaží rozvíjet vlastní produkt.

V důsledku toho lze identifikovat poměrně jednoznačnou zákonitost: jednoduché aplikace určené pro využití v malých a středních podnicích vykazují vysoce specifickou funkčnost, často se od sebe neliší z hlediska způsobu použití a jeví se jako vhodnější nástroj pro školicí a instruktážní účely než jako samostatné metodické návody.

Rozsáhlé aplikace, současně se svou vysokou mírou flexibility, obvykle přinášejí také významné navýšení nákladů na vývoj. V porovnání s výsledným zvýšením efektivity tyto náklady často postrádají ekonomickou opodstatněnost, pokud je integrace manuálů uvažována jako jediný způsob využití systému. Ve většině případů se přitom nejedná o jednorázovou investici, ale o tým specialistů, kteří budou po dlouhou dobu zajišťovat nejen běžný provoz aplikace, ale také průběžné rozšiřování její funkcionality a doplňování nových manuálů.

Ve výsledku se tím na trhu vytváří situace, v níž si integraci AR do výrobních procesů nebo nahrazení tradiční dokumentace pro koncového uživatele mohou finančně a organizačně zajistit převážně jen velké podniky. Na trhu chybí jednoduché, univerzálně použitelné řešení, explicitně zaměřené na transformaci návodů do prostředí rozšířené reality, které by nabízelo pouze ty funkce, jež jsou nezbytné pro tento specifický aplikační obor.

Kontext použití

Vzhledem k tomu, že je na trhu k dispozici velké množství univerzálních aplikací vybavených rozsáhlými sadami funkcí, které, jak bylo výše uvedeno, významně komplikují výrobní a implementační procesy, navrhuji vyvinout univerzálně zaměřený systém pro jeden konkrétní úkol. Takto zvolený přístup podstatně zjednoduší proces vývoje a údržby aplikace a současně poskytne jedno jednoznačně definované uživatelské rozhraní určené přímo pro práci s manuály.

Pod univerzálně orientovanou aplikací se rozumí softwarový produkt, který lze efektivně využívat zástupci různých skupin uživatelů v širokém spektru kontextů použití.

Místo použití:

- Použití v domácích podmínkách, při kterém má uživatel k dispozici návody pro opravy a diagnostiku domácích spotřebičů, montáž nábytku, obsluhu domácích technických zařízení a realizaci dalších obdobných činností.

- Použití ve výrobě, kde může uživatel při realizaci komplexních výrobních procesů, prostřednictvím systému poskytovat doporučení pro následné pracovní kroky, návody pro rychlou identifikaci a odstranění poruch výrobních zařízení a rovněž zprostředkovat během školení na nová zařízení více imerzivní, strukturovanou a srozumitelnou zkušenost a obdobné formy asistence.

Je třeba poznamenat, že první bod platí také v pracovním kontextu, pokud jde o podporu uživatele při používání specifického a jemu neznámého fyzického zařízení.

Zkušenost uživatele:

- Uživatel bez technických zkušeností má zkušenost s běžnými mobilními aplikacemi, ale ne s aplikacemi využívajícími rozšířenou realitu. Pro tuto skupinu je nutné navrhnout rozhraní co nejpodobnější konvenčním mobilním aplikacím, s minimem složitých ovládacích prvků a případnou kontextovou nápovědou k jejich použití.
- Uživatel s minimálními technickými zkušenostmi je obeznámen s daným pracovním postupem a má základní kompetence pro práci s technickou dokumentací a manuály. Znalost principů rozšířené reality není nutná, proto je třeba zachovat standardizované zobrazovací postupy obdobné těm z tištěných příruček.
- Pokročilý uživatel s technickými zkušenostmi a hlubokým porozuměním dané problematice, obvykle technik ve výrobě. Předpokládá se, že lze cíleně proškolit v používání aplikací rozšířené reality, pokud s nimi dosud nemá praktické zkušenosti. Je nutné navrhnout a implementovat systém, který při používání nezpůsobuje vnímané nepohodlí ani nezvyšuje kognitivní či časovou zátěž, aby uživatel neměl dojem, že aplikace zpomaluje nebo jinak zhoršuje jeho pracovní výkon.

Ačkoli samotná kvalita manuálů v AR představuje nemalý faktor, z těchto kategorií lze současně odvodit i dílčí aspekty relevantní pro jejich praktickou implementaci.

Účel použití

- Montáž/instalace.
- Diagnostika poruch.
- Pravidelná údržba.
- Školení a instruktáž.
- Běžné používání.

Z uvedených faktorů se skládá funkční část aplikace, která bude podrobněji popsána dále.

Cíle systému

Na základě identifikovaných nedostatků stávajících aplikací, specifik kontextu jejich použití a také výhod, které AR přináší a které byly popsány v kapitole 2.3, jsou pro navrhovaný systém formulovány následující cíle:

- Cílem je vytvořit **jednoduchý a univerzálně použitelný** nástroj pro zobrazování manuálů v AR, který nebude uživatele zatěžovat nadbytečnými funkcemi. Z tohoto důvodu by aplikace měla implementovat výhradně funkce nezbytné pro dosažení stanoveného účelu.

- Je nezbytné rozšířit systém o možnost **podpory velkého množství markerů**. S ohledem na specifika současných komerčně dostupných řešení, která typicky umožňují práci pouze s jedním markerem (např. `3DQR`) až po pět markerů (např. `Vuforia SDK`[1]), může takové omezení představovat zásadní problém při aplikaci na konstrukčně složitých či rozměrných zařízeních, jako jsou například celé výrobní linky.

- **Minimalizace kognitivní zátěže** uživatele. Velké množství podobných aplikací je založeno na využití 3D modelů, čemuž se v rámci tohoto řešení záměrně vyhýbáme. Ačkoliv může tento přístup při školení nových zaměstnanců přinášet určité výhody, v jiných kontextech se jako vhodnější jeví přímé provázání interaktivních prvků s konkrétními body ve fyzickém prostředí. Tímto způsobem dochází k redukci kognitivní mezery mezi uživatelem, dostupnými informacemi a příslušným objektem či procesem použití.

- Zajistit **multiplatformní použitelnost**, primárně pro mobilní zařízení, s možností rozšíření na HMD. Univerzální použitelnost vyžaduje, aby uživatelé mohli aplikaci provozovat na pro ně optimální hardwarové platformě, což je zásadní zejména v domácím prostředí, kde si obvykle nemohou dovolit specializované zařízení pro jediný účel. HMD představují perspektivní směr zejména pro výrobní procesy díky vlastnostem popsaným v kapitole 2.2, avšak v současné fázi vývoje nejsou hlavní prioritou.

- **Zjednodušit a formalizovat proces tvorby** nových manuálů tak, aby jej mohl provádět specialista, který běžně připravuje dokumentaci. Jak již bylo dříve uvedeno, jedním z klíčových problémů integrace AR manuálů je složitost jejich vývoje a s tím spojené vysoké náklady. Tyto problémy lze částečně eliminovat zjednodušením procesu tvorby manuálů. Takový přístup je však nutně provázen kompromisy, zejména redukcí dostupné funkčnosti a omezením možností personalizace pro specifické úlohy. Tyto limity mohou být kritické pro některé typy použití, nicméně představují nezbytný krok směrem k překonání omezení stávajících řešení.

4.2 Uživatelský průzkum a požadavky

Aby návrh systému odpovídal reálným potřebám budoucích uživatelů, byl proveden orientační uživatelský průzkum zaměřený na identifikaci očekávání, problémů a preferovaných způsobů interakce při práci s technickou dokumentací. Průzkum proběhl formou neformálních rozhovorů s osmi potenciálními uživateli, kteří reprezentovali různé typy použití i různé úrovně zkušeností s technologiemi rozšířené reality.

Do průzkumu byli zahrnuti:

- Pět uživatelů bez předchozí zkušenosti s AR, typicky reprezentující domácí použití a běžné spotřebitele.

[1] Doporučené maximum Vuforia SDK pro stabilní sledování https://developer.vuforia.com/library/vuforia-engine/images-and-objects/image-targets/detect-and-track-multiple-targets-simultaneously/

- Tři uživatelé se základní nebo střední technickou zkušeností, kteří občas pracují s manuály v rámci běžné údržby nebo montáže (převážně v domácích podmínkách).
- Dva zaměstnanci výrobních podniků, kteří mají zkušenosti s údržbou nebo provozem výrobního zařízení.
- Tři uživatelé, kteří již dříve pracovali s AR aplikacemi, ať už v rámci školení nebo testování technologií.

Cílem bylo zachytit rozdílné potřeby dvou hlavních kontextů použití – domácího a profesního – a zároveň porovnat očekávání uživatelů, kteří AR znají, s těmi, kteří se s ní setkají poprvé.

Získané poznatky lze shrnout následovně:

- Uživatelé bez zkušeností s AR preferují **co nejjednodušší rozhraní**, které se chová podobně jako běžné mobilní aplikace. Podle jejich názoru působila většina aplikací pracujících s AR, které jim byly předvedeny, z jejich pohledu příliš složitě a nepřehledně.
- Profesionální uživatelé pochybují, že jim systém umožní rychle se zorientovat při používání aplikace, protože se budou muset znovu **učit nové technologie**. Připouštějí však, že při práci s novým vybavením by jim podobný systém pomohl zorientovat se rychleji a přesněji.
- Profesionální uživatelé také, na rozdíl od běžných, uvádějí, že by se při používání chtěli **vyhnout velkému množství informací** na obrazovce. Při pohybu kolem stroje jim budou stačit krátké popisky pro rychlou orientaci.
- Všichni uživatelé zmínili, že při práci by pro ně nebylo vždy pohodlné **držet zařízení** v rukou, což je zejména v profesionálním prostředí považováno za nepraktické.
- Při práci s většími zařízeními je důležité, aby se informace **automaticky přizpůsobovaly** tomu, co má uživatel právě před sebou.
- Někteří uživatelé také uvedli, že by pro ně bylo praktičtější, kdyby tato aplikace zároveň obsahovala **standardní elektronickou verzi** manuálu, ve které by mohli rychle vyhledat potřebné informace.
- V rámci rozhovorů byl rovněž kladen požadavek na **integraci aplikace s IoT**, která by umožnila přímo v uživatelském rozhraní poskytovat relevantní informace o připojených zařízeních.

Tyto poznatky tvoří základ návrhu uživatelského rozhraní, struktury manuálů i celkové architektury systému a byly zohledněny v následujících částech návrhu.

4.3 Architektura systému

Na základě požadavků, uživatelských scénářů a omezení formulovaných v předchozí části lze navrhnout architekturu systému, která zajišťuje stabilní provoz v prostředí rozšířené reality, zachovává dostatečnou míru uživatelské přívětivosti pro běžné uživatele a současně poskytuje potenciál pro další škálování a rozšiřování funkcionality. V následující části budou představeny hlavní komponenty nezbytné pro zajištění funkčnosti aplikace a bude popsán princip jejich vzájemné interakce a integrace.

Přehled architektury

Navrhovaná architektura bude koncipována na základě tří klíčových komponent, a to:

- Mobilní aplikace (AR klient) – hlavní uživatelské rozhraní, prostřednictvím něhož bude realizována interakce s manuály v prostředí rozšířené reality, musí disponovat následujícími funkcemi:
  - Rozpoznávání markerů.
  - Zobrazení interakčních bodů. Pod interakčním bodem se rozumí vizuální zobrazení oblasti na obrazovce, na kterou může uživatel klepnout a která je spojena s konkrétním místem zájmu ve fyzickém prostředí.
  - Sledování rozpoznaných interakčních bodů.
  - Rozhraní pro poskytování nápovědy uživateli po klepnutí na interakční bod.
  - Výběr scénářů interakce. Pod scénářem se rozumí konkrétní instrukce, která závisí na aktuálním úkolu. Například oprava prvku `X`, spuštění režimu `Y`, údržba zařízení po ukončení práce a podobně.
  - Práce v režimu krokových scénářů.
  - Režim určený pro vývojáře, který umožňuje přímé umístění interakčních bodů přímo v rozšířené realitě v rámci reálného prostoru.
  - Způsob lokálního ukládání manuálů, aby byl zajištěn princip „**Decentralizace**“ popsaný v kapitole 2.3.
  - Podpora volby jazyka, která zajistí princip „**Interoperability**“ popsaný v kapitole 2.3.
- Backend – Serverová komponenta zajišťuje centralizované ukládání informací a jejich dostupnost z libovolného místa sítě. Tato architektura současně umožňuje využití aplikace i malým podnikům, které nedisponují vlastní serverovou infrastrukturou pro správu a uchovávání dat. Zároveň zajistí princip „**Funkce v reálném čase**“ popsaný v kapitole 2.3. Musí disponovat následujícími funkcemi:
  - Ukládání struktury manuálů.
  - Poskytování dat prostřednictvím API dotazů.
  - Validace a aktualizace manuálů.
  - Jednoduchý systém správy verzí pro bezpečnou úpravu stávajících manuálů.
- Desktopová aplikace poskytuje uživatelské rozhraní pro tvorbu a správu technických manuálů. Jejím cílem je nabídnout ergonomicky navržené a sémanticky jednoznačné ovládací prvky, optimalizované pro uživatele se zkušenostmi s nástroji typu Microsoft Word, Dr.Explain[2] a dalšími obdobnými systémy pro vytváření dokumentaci. Musí disponovat následujícími funkcemi:
  - Vytváření manuálů a scénářů k nim.
  - Vytváření interakčních bodů a jejich přiřazování scénářům.
  - Vytváření krokových scénářů.
  - Tisk značek v jim přiřazených velikostech pro rychlé testování.

[2]Sada nástrojů pro vytváření uživatelské dokumentace. Dostupné z https://www.drexplain.com/.

Komponenty systému

Za účelem zachování principu „**Modularity**“ uvedeného v kapitole 2.3 je nezbytné navrhovat aplikaci modulárním způsobem. V případě mobilní aplikace lze přitom identifikovat několik hlavních komponent:

- AR modul je zodpovědný za práci kamery, určení polohy uživatele v prostoru a sledování pozic již nalezených markerů.
- Modul QR je zodpovědný za rozpoznávání QR kódů, díky čemuž program získává informace o manuálu, se kterým aplikace v daném okamžiku pracuje. Tuto funkcionalitu lze rovněž integrovat do detekčního modulu rozšířením jeho schopností, neboť se v obou případech realizuje totožný typ úlohy.
- Modul detekce je zodpovědný za rozpoznávání markerů na obrazovce a za výpočet jejich prostorové pozice ve scéně vzhledem k uživateli, jak bylo podrobně popsáno v kapitolách 3.3 a 3.4.
- Modul markerů je zodpovědný za správu detekovaných markerů a využívá datový modul k získávání informací o aktuálně používaném manuálu. Současně zprostředkovává vizualizaci aktuálních interakčních bodů.
- Modul scénářů zajišťuje správu dostupných scénářů a uchovávání informací o příslušných bodech interakce.
- Datový modul zajišťuje nahrávání manuálů a informací o konkrétních markerech. Současně poskytuje funkcionalitu pro jejich lokální ukládání a následnou dostupnost v rámci systému.
- UI modul je zodpovědný za prezentaci uživatelského rozhraní na cílovém zařízení a za zprostředkování interakce uživatele s jednotlivými ovládacími prvky rozhraní.
- Modul dat poskytuje sjednocené rozhraní pro načítání informací o manuálu a jednotlivých krocích z libovolného zdroje. Zajišťuje předávání strukturovaných dat ostatním částem systému a abstrahuje způsob jejich získávání.

Tato struktura díky principu zapouzdření, byla navržena tak, aby umožňovala efektivní a transparentní výměnu modulů jak v průběhu vývojového cyklu, tak i při následné údržbě a rozšiřování aplikace.

Tok dat

Celkový tok dat v rámci této architektury lze charakterizovat následovně:

1. Detekce QR kódu
   AR modul předává snímek z kamery do QR modulu, který provádí dekódování QR kódu. Následně jsou z něj extrahované informace předány k dalšímu zpracování.
2. Načtení manuálu
   AR klient iniciuje požadavek na backendovou službu obsahující identifikátor (ID) příslušného QR kódu, na jehož základě backend vrátí odpovídající datovou strukturu manuálu.

3. Detekce markerů
   Následně se uživatel pohybuje v rámci scény a lokalizuje markery, které jsou identifikovány modulem detekci. Tomuto modulu jsou předávány obrazové informace získané kamerou z AR modulu.

4. Určení polohy markerů
   V závislosti na podmínkách program rozhoduje, zda potřebuje upřesnit polohu markeru, což je podrobněji popsáno v kapitole 4.5.

5. Inicializace interakčních bodů
   V místech, kde byl rozpoznán marker a určena jeho poloha, dochází k inicializaci interaktivních bodů v závislosti na jejich pozici uvedené ve struktuře dat.

6. Zobrazení interakčních bodů
   Po kliknutí na interakční body uživatel obdrží informace obsažené v odpovídající datové struktuře. V případě velkého objemu dat nebo při přítomnosti obrázků či videí je třeba o jejich získání požádat datový modul.

7. Výběr scénáře
   Modul scénářů oznamuje UI modulu, jaký typ rozhraní má být v danou chvíli zobrazen v závislosti na jeho typu. Také informuje ostatní moduly o změně kontextu scény a filtruje interakční body podle aktuálního kroku.

8. Režim vývojáře
   Informuje ostatní moduly o změně obsahu scény za účelem zobrazení aktuálního prvku, nad nímž dochází ke změně. Současně odesílá požadavky do datového modulu za účelem uložení stavu aktuálního bodu interakce.

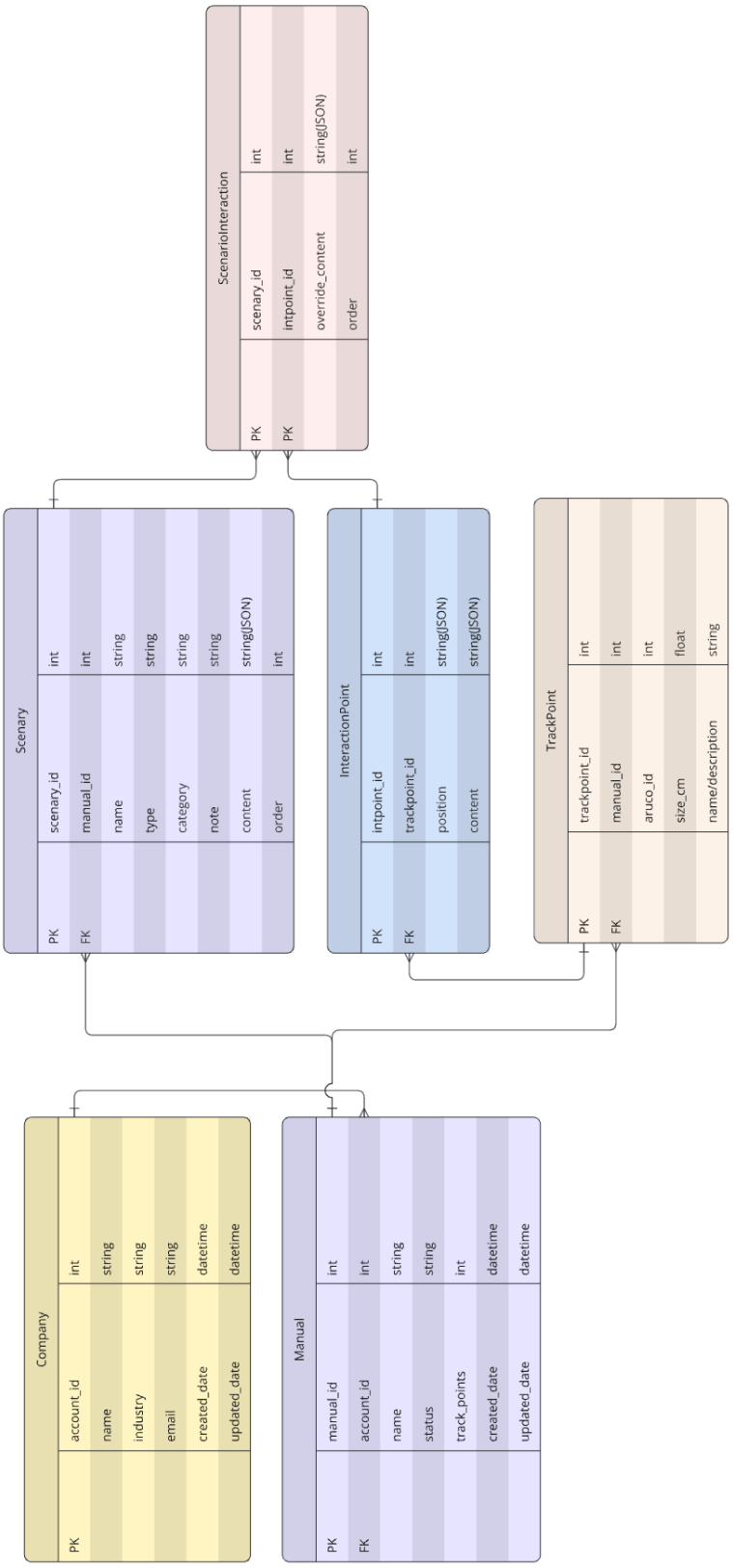
4.4 Datový model

Datový model představuje základní strukturu, která určuje způsob ukládání, organizace a následného zpracování informací o manuálech, scénářích, markerů a interakčních bodech. Návrh modelu vychází z požadavků popsaných v předchozí kapitole a reflektuje specifika práce s AR manuály, zejména podporu více markerů a jednoduchou rozšiřitelnost.

ERD model

Datový model se skládá ze šesti hlavních entit, což lze vidět na obrázku 4.1:

- Společnost (Company) – představuje základní informace o společnosti, která používá tuto aplikaci. Mohou být využívány při podpoře, přidělování přístupových práv a podobně.
- Manuál (Manual) – každá z firem může mít několik manuálů. Také uchovává informace o počtu markerů, které jsou v daném manuálu použity.
- Scénář (Scenary) – každý jednotlivý manuál je koncipován jako soubor scénářů. Tato struktura uchovává informace o typu scénáře, jeho kategorii, doplňujících poznámkách (určených pro vývojáře) a dalších relevantních metadatech.



Obrázek 4.1: Datový model navrženého systému

- Marker (TrackPoint) – pro optimalizaci procesu detekce je nezbytné disponovat předem stanovenými informacemi o počtu markerů použitých v rámci daného manuálu (z Manual entity), stejně jako o jejich fyzických rozměrech udávaných v centimetrech.
- Interakční bod (InteractionPoint) – každý bod je definován ve vztahu k prostorové poloze ve fyzickém prostředí vzhledem ke svému referenčnímu markeru. Data jsou ukládána odděleně, aby byla usnadněna tvorba technických manuálů a následná aktualizace jejich obsahu.
- Vztah mezi scénáři a interakčními body (ScenarioInteraction) spočívá v tom, že jednotlivé body interakce jsou vázány na konkrétní místa zájmu ve fyzickém prostoru, a proto není nutné je pro každý scénář znovu samostatně vytvářet. Kontext jejich použití se však může v jednotlivých scénářích lišit, právě k odlišení těchto kontextů a k jejich formálnímu vyjádření slouží uvedená entita.

4.5 Detekce a sledování markerů

Na trhu je v současnosti k dispozici celá řada již existujících řešení dané problematiky. Při práci s velkým počtem markerů však všechna tato řešení narážejí na zásadní omezení – neschopnost současně sledovat velké množství markerů.

Hlavním důvodem existence tohoto omezení je princip, na němž jsou založeny standardní implementace: sledování markerů typicky využívá tzv. image(object)-based tracking [35]. Tento přístup má nesporné výhody, zejména možnost volby prakticky libovolného obrázku (s určitými omezeními) jako referenčního vzoru, například firemního loga. Zároveň však tento přístup přináší řadu omezení a s narůstajícím počtem podobných „markerů“, které se snažíme využít, je nutné, aby každý jednotlivý obraz tím přísněji splňoval principy[3], na jejichž základě lze posoudit, do jaké míry je vhodný k použití.

Jedním z **klíčových rozdílů** navrhovaného řešení je požadavek na současné zpracování většího množství markerů, než umožňují běžně dostupné knihovny. Z tohoto důvodu bylo nutné navrhnout vlastní metodiku detekce a sledování, která je optimalizována pro potřeby systému a není omezena principy image-based trackingu.

Architektonické rozdělení procesu detekce

Proces je rozdělen do dvou samostatných částí:

- Detekce markerů – rychlá identifikace všech markerů přítomných ve scéně.
- Výpočet 6DoF – přesné určení prostorové pozice pouze u těch markerů, které jsou relevantní.

Toto rozdělení výrazně snižuje výpočetní zátěž při práci z větším množstvím markerů, aniž by docházelo ke zpomalení aplikace. Detekce markerů je také lépe optimalizovaná operace ve srovnání s detekcí obrazů.

Tento přístup také umožňuje současné sledování velkého množství **pohybujících** se markerů. Jedná se o obtížný problém, pro který v současné době na trhu neexistuje žádné stabilní, široce dostupné a otevřené řešení.

[3] Vuforia: Best Practices for Designing and Developing Image-Based Targets. Dostupné z https://developer.vuforia.com/library/vuforia-engine/images-and-objects/image-targets/best-practices/best-practices-designing-and-developing-image-based-targets/

Identifikace a validace markerů

Proces detekce markerů probíhá kontinuálně v rámci AR modulu. Kamera generuje sekvenci jednotlivých snímků, které jsou následně předávány detekčnímu modulu, jenž zajišťuje následující kroky zpracování:

- Identifikace přítomných markerů.
- Určení polohy markeru na obrazovce (upřesnění polohy rohů).
- Filtrování markerů podle filtru aktuálního scénáře.

Tímto způsobem program získá informace o přítomných markerech, ale výpočet jejich přesné pozice provádí pouze tehdy, pokud je to skutečně potřeba. Využití filtru zároveň zajišťuje, že program neprovádí nadbytečné výpočty a nealokuje výpočetní prostředky na rozpoznávání markerů, které nejsou v kontextu daného scénáře relevantní.

Rozhodnutí o zpracovaní markerů

V každém časovém okamžiku může být viditelných několik markerů, případně nemusí být detekován žádný. Systém proto udržuje jejich seznam a na jeho základě rozhoduje o dalším zpracování následovně:

- Zda je třeba skrýt markery, které nebyly zaznamenány při posledním rozpoznání. Díky tomuto přístupu při chybě v rozpoznávání body interakce nebudou neustále mizet a znovu se objevovat.
- V závislosti na vzdálenosti od markeru skrývá body interakce, aby se nepřetěžovalo uživatelské rozhraní.
- Při nalezení nového markeru by měl program požádat detekční modul o výpočet jeho 6DoF.
- Při detekci ze vzdáleného místa může být poloha markeru vypočtena chybně, proto by měl program při přiblížení provést kalibraci.

Právě v tomto okamžiku program rozhoduje o výpočtu jeho 6DoF, což umožňuje aplikaci optimalizovat. Při sledování pohybujících se markerů však musí být kalibrační operace prováděna mnohem častěji.

Lokální souřadnicový systém markeru

Z důvodu usnadnění integrace manualů a s ohledem na optimalizační požadavky není vhodné umisťovat samostatný marker ke každému jednotlivému interaktivnímu prvku. Namísto toho se navrhuje umístit jeden marker v bezprostřední blízkosti každého shluku interakčních míst. Následně je možné, s využitím informací o tomto markeru, jeho fyzické velikosti a pozici ve scéně, umístit v lokálním souřadnicovém systému body interakce. To zajistí několik výhod:

- Přesné umístění informací bez nutnosti použití 3D modelů.
- Opakované použití interakčních bodů v průběhu tvorby manuálu.

- Poměrně snadné vytváření nových interakčních bodů, tvůrce musí pouze zadat jejich polohu vzhledem k markeru v centimetrech pomocí souřadnic (x, y, z).
- Interakční body se zobrazují pouze v případě přítomnosti markeru.

Zobrazení interakčních bodů

Po provedení předchozích kroků má systém k dispozici údaje o prostorovém umístění markerů ve scéně, a je tak možné do prostoru umístit body interakce. Postupuje se přitom v následujícím pořadí:

1. Po dokončení práce s novým markerem se provede požadavek na získání souvisejících bodů interakce v rámci daného scénáře.
2. Pomocí lokálního souřadnicového systému markeru a relativní polohy interakčního bodu ve scéně je umístěn objekt, se kterým může uživatel interagovat.
3. Při kliknutí na objekt se uživateli zobrazí odpovídající informace.
4. V závislosti na implementaci lze zvážit také variantu s dotazem na data pro každý bod interakce zvlášť, zejména při velkém objemu dat, jako jsou například obrázky nebo videa.

Díky výše popsanému mechanismu se daný program zároveň eliminuje problém okluze [41] při zobrazování interaktivních bodů, který představuje významnou obtíž jak pro koncové uživatele, tak pro vývojáře při práci s AR aplikacemi. V kontextu práce s větším množstvím markerů jde o aspekt zásadního významu.

Umístění interakčních bodů

Pro zjednodušení procesu tvorby je vhodné autory manuálu zbavit nutnosti manuálního určování polohy bodů interakce pomocí pravítka. Tato činnost je obzvláště nepraktická v situacích, kdy se referenční marker a bod interakce nacházejí v odlišných rovinách. Z uvedených důvodů je účelné zavést samostatný režim určený k automatizovanému výpočtu prostorové polohy umísťovaných bodů.

Kapitola 5

Uživatelské rozhraní a UX návrh

Zatímco předchozí kapitoly se zaměřovaly na technické aspekty detekce markerů, datový model a architekturu systému, tato část práce se věnuje návrhu rozhraní z hlediska použitelnosti, ergonomie a celkové uživatelské zkušenosti.

Návrh uživatelského rozhraní vychází z poznatků uživatelského průzkumu uvedeného v kapitole 4.2, který ukázal rozdíly mezi potřebami domácích uživatelů a profesionálních pracovníků. Zároveň potvrdil, že technologie rozšířené reality může uživatelům významně pomoci, avšak pouze za předpokladu, že je rozhraní navrženo s důrazem na jednoduchost, nízkou kognitivní zátěž a přirozené chování v AR prostředí. Rovněž je využita příslušná literatura, jejíž autoři se již setkali s realizací podobných aplikací a identifikovali klíčové faktory, které ovlivňují efektivitu jejich používání uživateli.

Cílem této kapitoly je proto představit návrhové principy, strukturu uživatelského rozhraní a specifické interakční mechanismy, které umožňují efektivní práci s manuály v prostředí rozšířené reality. Součástí kapitoly je také návrh rozhraní pro autory manuálů.

5.1 Cíle návrhu uživatelského rozhraní

Cíle návrhu uživatelského rozhraní vycházejí z požadavků uživatelů a z praktických omezení práce v prostředí rozšířené reality. Návrh se proto zaměřuje na následující oblasti:

- Srozumitelnost a jednoduchost – rozhraní musí být použitelné i pro uživatele bez předchozí zkušenosti s AR a nesmí vyžadovat složité ovládání.
- Nízká kognitivní zátěž – uživatel má v AR omezenou pozornost. Systém proto zobrazuje pouze prvky, které jsou relevantní k aktuální situaci.
- Stabilita a ergonomie – uživatel často drží zařízení v ruce a současně manipuluje s fyzickým objektem. Rozhraní musí být vizuálně stabilní a snadno ovladatelné jednou rukou.
- Přizpůsobení kontextu – rozhraní reaguje na změny v prostoru (např. přiblížení/oddálení od markeru) a přizpůsobuje množství zobrazených informací.
- Konzistence napříč scénáři – uživatel musí být schopen rychle pochopit strukturu manuálů a orientovat se v jednotlivých krocích bez ohledu na typ zařízení.
- Podpora tvorby obsahu – systém musí umožnit autorům manuálů vytvářet a upravovat obsah bez nutnosti specializovaných technických znalostí.

5.2 Návrhové principy

Principy navrhování konkretizují cíle uvedené v části 5.1 a převádějí je do praktických pravidel pro zobrazování a interakci v AR. Níže jsou uvedeny klíčové principy spolu s uvedením jejich přímého vlivu na design.

- Minimalizace informací – zobrazuje se pouze to, co je nezbytné pro provedení aktuálního kroku. Vedlejší prvky jsou skryty nebo sloučeny. Tím se snižuje vizuální šum a urychluje se rozhodování uživatele.
- Kontextová relevance – obsah a úroveň podrobnosti se přizpůsobují kontextu: poloze uživatele, fázi scénáře, stavu zařízení. Pravidlo se realizuje filtrováním podle značek scény a pravidel aktivace prvků.
- Vizuální stabilita – anotace a značky si musí zachovávat stabilní polohu vůči reálnému objektu při malých výkyvech kamery. Při krátkodobé ztrátě sledování se používá predikce pozice. Tím se snižuje pocit „poskakující“ grafiky a zvyšuje se důvěra uživatele.
- Princip „méně je více“ – grafika a typografie jsou orientovány na čitelnost: omezená paleta barev, jednoduché ikony, minimální animace. Prioritou jsou kontrast a jednoznačnost, nikoli estetické ozdoby.
- Adaptace podle vzdálenosti – množství a forma informací se mění s vzdáleností k objektu: při větším odstupu zobrazit orientační nápovědu, při přiblížení aktivovat detailní kroky.
- Bezpečnost fyzické manipulace – rozhraní nezasahuje do zorného pole kritických částí objektu. Implementován pohodlný způsob ovládání a současné práce s fyzickým zařízením.
- Okamžitá zpětná vazba – každá akce uživatele (výběr kroku, potvrzení, chyba sledování) musí být okamžitě vizuálně nebo hapticky indikována. Zpětná vazba obsahuje stavovou informaci a návrh dalšího kroku, čímž snižuje nejistotu při práci.

Koncept adaptace podle vzdálenosti byl převzat z práce [33], v níž autoři představují funkci „úrovní podrobnosti“. Ačkoli tato koncepce není v původní podobě v kontextu této studie přímo aplikovatelná, představuje dobrý základ. Ze stejné práce lze rovněž převzít myšlenku dočasného pozastavení vizuálního výstupu na obrazovce za účelem zajištění bezpečnosti fyzické manipulace, o které se zmiňují i další práce, a to buď jako o součásti implementace, nebo jako účinný způsob řešení problémů spojených s používáním aplikace [28].

5.3 Uživatelské rozhraní aplikace

Následující část prezentuje podrobný popis konkrétní struktury obrazovek a klíčových interakčních prvků implementovaných v uživatelské části aplikace. Tento popis vychází z dříve definovaných základních konceptů. Návrh uživatelského rozhraní byl vytvořen s využitím online nástroje Figma.



Obrázek 5.1: Obrazovka skenování QR kódu

Úvodní obrazovka: skenování QR kódu

Při spuštění aplikace je uživateli hned nabídnuto naskenování QR kódu na zařízení, jak je uvedeno na obrázku 5.1. Toto řešení je dáno požadavkem zobrazovat manuál pouze při fyzické přítomnosti u zařízení: předchozí navigace v katalozích před skenováním zvyšuje riziko zobrazení nerelevantních informací a také počet zbytečných kroků, které musí uživatel provést.

Obrazovka: přehled zařízení

Po naskenování QR kódu je uživatel přesměrován do prvního scénáře, jímž je přehled zařízení. Na návrzích jsou prezentovány tři režimy zobrazení 5.2. Prostřednictvím tlačítka pro volbu „režimu“, umístěného v pravém dolním rohu obrazovky, může uživatel volně přepínat mezi jednotlivými režimy zobrazení. Tato funkcionalita představuje kompromisní řešení problému prezentace informací, který byl diskutován v kapitole 4.2.

Vedle tlačítka pro volbu režimů je také umístěno tlačítko pozastavení obrazu, které uživateli umožní snadno pracovat s manuálem, aniž by musel zařízení neustále držet naproti požadovanému bodu.

V levé dolní části rozhraní se nachází tlačítko umožňující výběr scénáře, jemuž se podrobněji věnuje následující část.

Obrazovka: výběr scénáře

Obrazovka pro výběr scénáře poskytuje minimalistické uživatelské rozhraní, znázorněné na obrázku 5.3, které umožňuje rychle přepínat mezi různými kontexty. Jednotlivé scénáře jsou systematicky tříděny podle své kategorie a zároveň jsou uspořádány podle předem definovaného pořadí v rámci každé kategorie. Takto navržená struktura umožňuje uživateli rychle



(a) „Point-only“.



(b) „Headers-only“.

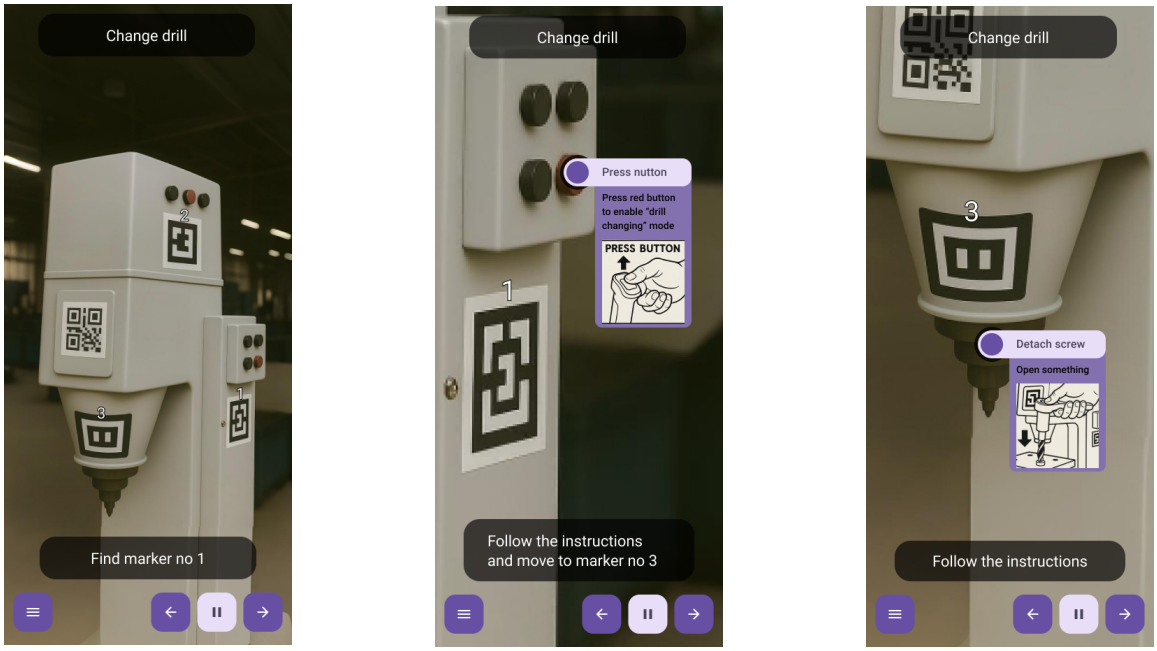


(c) „Full-view“.

Obrázek 5.2: Přehledová obrazovka: tři varianty zobrazení informací o zařízení. Režim „Point-only“ – skryje detailní informace o bodech interakce a zobrazí pouze jejich existenci. Režim „Headers-only“ – zobrazuje pouze názvy interakčních bodů. Režim „Full-view“ – zobrazuje kompletní informace o interakčních bodech.



Obrázek 5.3: Obrazovka výběru scénáře



(a) Úvodní obrazovka (b) Zobrazení kroku (c) Následující krok

Obrázek 5.4: Příklad zobrazení krokového scénáře.

měnit potřebné zobrazované informace bez významného narušení probíhajícího pracovního procesu.

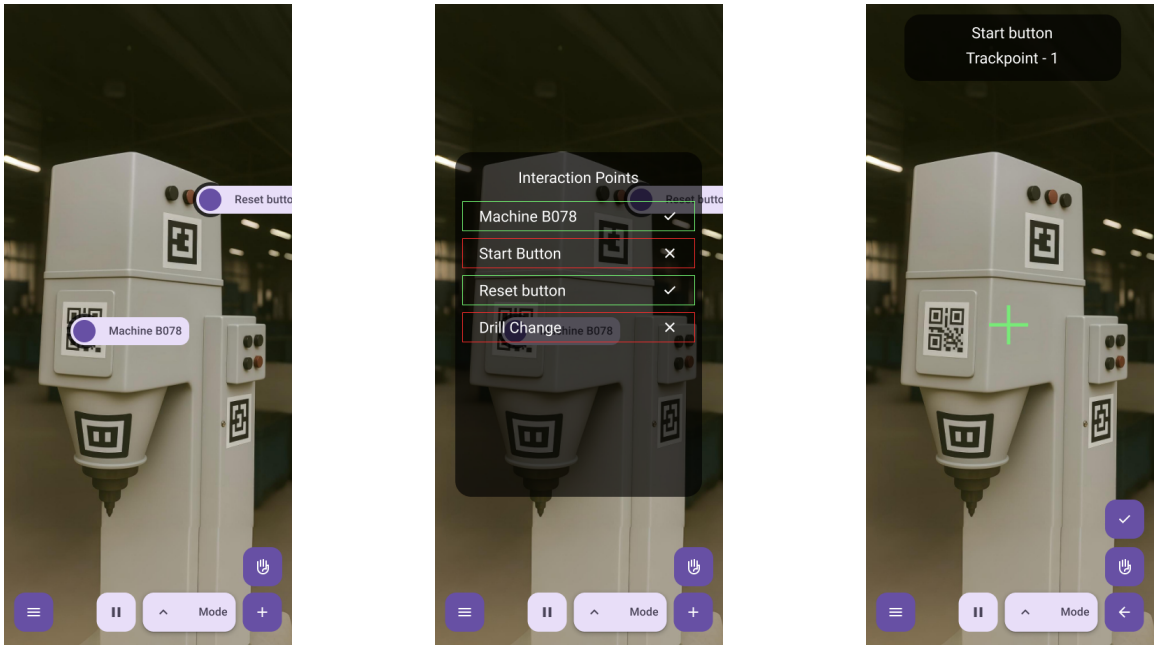
Obrazovka: krokové scénáře

Při aktivaci krokových scénářů se v rozhraní přidají tlačítka určená k přepínání jednotlivých kroků, jak je ilustrováno na obrázcích 5.4. Tato tlačítka musí svým vizuálním provedením jednoznačně indikovat nemožnost přepnutí v případech, kdy takové omezení existuje. V horní části rozhraní se současně zobrazuje název aktuálně zvoleného scénáře, aby se minimalizovalo riziko chyby při jeho výběru a eliminovala se možnost realizace nesprávných kroků.

Každý krok je navržen jako atomární jednotka. Instrukce v dolní části obrazovky jednoznačně indikují aktuální marker, k němuž se má uživatel přesunout, a současný bod interakce. Ostatní body by neměly být zobrazeny, aby se minimalizovalo kognitivní zatížení a nedocházelo k rozptylování uživatele. Současně je zobrazován i pokyn, který explicitně specifikuje požadovanou akci pro daný krok.

5.4 Rozhraní pro autory manuálů

Tato sekce popisuje uživatelské rozhraní určené pro tvorbu, modifikaci a validaci obsahu technických manuálů. Rozhraní je rozděleno na dvě vzájemně se doplňující části: lehký mobilní dev-mode pro rychlé připojení a kontrolu interakčních bodů na místě a desktopový editor pro systematickou přípravu scénářů.



(a) Aktuální stav (b) Seznam bodů (c) Umístění bodů

Obrázek 5.5: Rozhraní režimu vývojáře zabudovaného v mobilní aplikaci se sadou funkcí, které umožňují upravovat polohu interaktivních bodů.

Cíle autorského rozhraní

Rozhraní musí zajišťovat: jednoduchost a jednoznačnost sémantiky interaktivních bodů, minimální křivku učení a možnost ověřit správnost přiřazení přímo na samotném zařízení.

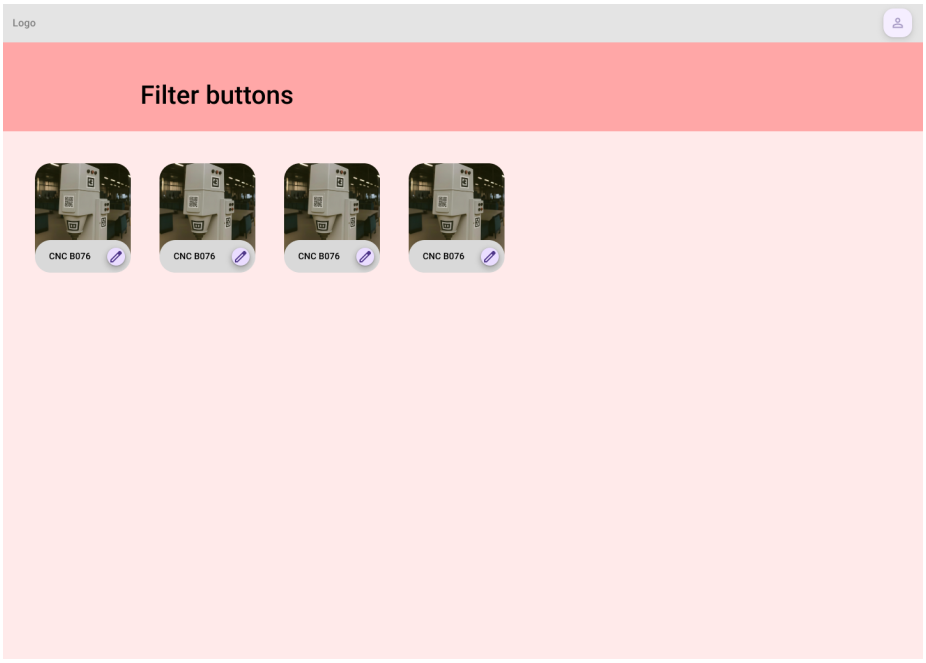
Jak bylo uvedeno v předchozích kapitolách, jednou z charakteristických vlastností této implementace je snadné vytváření nových manuálů a podpora stávajících. Z tohoto důvodu bylo rozhodnuto nepoužívat 3D modely a co nejvíce zjednodušit rozhraní pro jejich vytváření. Přestože desktopová část není v rámci této práce prioritní, zůstává důležitou součástí návrhu.

Dev-mode v mobilní aplikaci

Režim pro vývojáře poskytuje základní sadu operací potřebných pro rychlou přípravu a kontrolu obsahu přímo na zařízení:

- Kontrola stávajících bodů interakce a míst jejich umístění, jak je vidět na obrázku 5.5a.
- Výběr bodů pro další úpravu s uvedením již umístěných a těch, které je třeba ještě umístit, jak je vidět na obrázku 5.5b.
- Rozhraní pro umístění interakčních bodů na zařízení nebo pro změnu polohy již existujících bodů, jak je vidět na obrázku 5.5c.

V tomto režimu není k dispozici možnost měnit obsah interakčních bodů, protože tato funkčnost by pouze zkomplikovala rozhraní, aniž by přinesla významné výhody oproti provedení této operace v rámci desktopové aplikace.



Obrázek 5.6: Koncept obrazovky pro filtrování a výběr manuálů.

Desktopová aplikace

Desktopový editor poskytuje funkcionalitu pro vytváření a editaci obsahu manuálů. Jak již bylo uvedeno v předchozích částech, tato část není v rámci této práce prioritní, nicméně její návrh a plánování však bylo nezbytné. Způsob interakce uživatele při tvorbě manuálů a scénářů totiž přímo determinoval požadované funkční vlastnosti a obsahové možnosti mobilní aplikace.

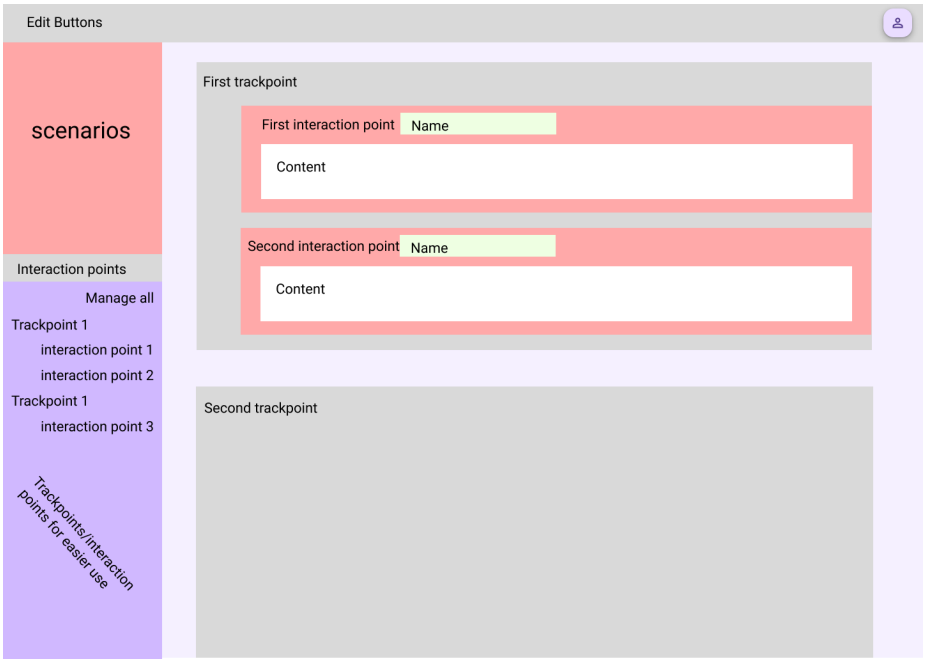
Jako rozhraní pro tvorbu manuálů byla, z několika důvodů a v porovnání s jinými dostupnými možnostmi, zvolena desktopová aplikace:

- Většina stávajících aplikací určených pro tvorbu manuálů v průmyslovém prostředí je realizována jako desktopová řešení. Volbou této varianty proto uživatelům poskytujeme jim dobře známé softwarové prostředí a infrastrukturu, s nimiž již mají předchozí pracovní zkušenosti.

- Na mobilním zařízení by byl proces vytváření výrazně ztížen v důsledku omezení souvisejících s dostupnými způsoby zadávání vstupu. Integrace této funkcionality by navíc vedla ke zvýšení komplexity logiky mobilní aplikace.

- Také bylo možné zvážit realizaci jako online nástroje. Jedinou výhodou v rámci této aplikace by však byla okamžitá aktualizace aplikace. Zaměstnanci zodpovědní za tvorbu manuálů totiž obvykle pracují ze svého pracoviště, a proto nepotřebují mít k aplikaci neustálý přístup odkudkoli. Tato část navíc není určena pro široké publikum.

Při návrhu desktopové aplikace byla učiněna řada koncepčních a technických rozhodnutí, která zásadním způsobem ovlivnila jak funkcionalitu a obsah mobilní aplikace, tak i podobu použitého datového modelu, jenž byl popsán v kapitole 4.4. V následující části budou popsána některá z těchto rozhodnutí.



Obrázek 5.7: Koncept obrazovky pro úpravu manuálu. Levý sloupec představuje řídicí část, ve které může uživatel vytvářet nebo vybírat scénáře k úpravě (vyznačeno červeně) a také spravovat existující interakční body a jejich přiřazení k markerům (vyznačeno fialovou). Střední část je pak určena k úpravě obsahu scénáře.

V rámci jedné společnosti může existovat velké množství manuálů, a proto je zapotřebí pohodlný způsob třídění a výběru scénáře, jak je vidět na obrázku 5.6. Lze také zavést systém správy verzí, který by byl užitečný při úpravách obsahu scénářů bez rizika poškození stávající pracovní verze.

Při návrhu rozhraní pro editaci obsahu byl kladen důraz na podobnost s nástroji určenými pro tvorbu standardních uživatelských manuálů. Na tomto základě byl definován soubor základních prvků rozhraní, mezi nimiž se má autor manuálu nejčastěji pohybovat:

- Seznam scénářů – předpokládá se, že při tvorbě se autor bude mezi nimi často přepínat.
- Seznam markerů a interakčních bodů – jelikož zobrazení informací je přímo navázáno na body zájmu v zařízení, musí mít autor nepřetržitý přístup k jejich zobrazení a přiřazování k příslušným scénářům.
- Úprava současného scénáře – který je v dané realizaci hlavním prvkem v rámci manuálu.
- Rychlá navigace k ostatním, méně používaným prvkům – jako jsou název manuálu, jeho dostupnost a verze.

Implementaci těchto prvků v rámci koncepce lze vidět na obrázku 5.7. Je také vidět, že běžné (ne-krokové) scénáře se skládají ze seznamu markerů (Trackpointů na obrázku), v jejichž rámci jsou definovány používané body interakce. Každý takový bod obsahuje data, a to buď v textové podobě, nebo ve formátu XML pro pohodlnější práci se zobrazením prvků různého charakteru.

Formát ukládání informací o interaktivních bodech je rovněž důležitou součástí plánování. V současné chvíli se lze zastavit u textového ukládání informací ve formě krátkých instrukcí. Při dalším rozšiřování je však vhodné uvažovat o standardizaci formátu instrukcí pro snadný export či import. Autoři některých prací se pokusili standardizovat AR instrukce; například v již dříve zmíněné práci [14] jsou uvedena základní doporučení pro použití formátů obrázků a způsob psaní textových instrukcí.

5.5 Průzkum uživatelů na základě představeného konceptu

V rámci návrhu uživatelského rozhraní byl proveden doplňující uživatelský průzkum zaměřený na ověření předpokladů formulovaných v kapitole 4.2. Cílem tohoto opakovaného šetření bylo zjistit, zda navržené interakční mechanismy odpovídají reálným potřebám uživatelů a zda některé z původně plánovaných funkcí nejsou redundantní. Průzkum probíhal na základě dříve představeného prototypu ve Figmě, který byl sestaven v prezentačním režimu.

Průzkum byl realizován formou krátkého dotazníku, který byl distribuován mezi stejnou skupinu respondentů jako v předchozí fázi. Dotazník obsahoval několik otázek:

1. Zhodnotit srozumitelnost prvků uživatelského rozhraní a zároveň specifikovat, jakou funkci by podle názoru respondenta měl každý jednotlivý prvek plnit.
2. Je princip fungování interakčních bodů srozumitelný?
   (a) Jaký je očekávaný efekt interakce s bodem?
   (b) Která metoda vizualizace je nejužitečnější?
3. Je sled kroků při práci s aplikací srozumitelný?
4. Princip práce s krokovými pokyny.
   (a) Je nutné vizuální označení požadovaného markeru?
   (b) V jaké formě jsou nápovědy nejpohodlnější?
   (c) Jaké funkce by pomohly při práci s krokovým scénářem?
5. Je proces výběru scénářů dostatečně srozumitelný a uživatelsky komfortní a jaká je očekávaná odezva aplikace na zvolený scénář?

Režim zobrazení

Výsledky ukázaly, že většina respondentů preferuje jednoduché a konzistentní rozhraní bez nutnosti přepínat mezi více režimy. Funkcionalita volby režimu zobrazení, která byla původně navržena jako klíčový prvek přehledové obrazovky, byla uživateli hodnocena jako málo využitelná a potenciálně zatěžující. Na základě těchto zjištění byla tato funkcionalita přesunuta mezi volitelné prvky a v základní verzi aplikace je aktivní pouze jeden výchozí režim – režim zobrazení bodu spolu s jeho názvem (viz obrázek 5.2b).

Režim úplného zobrazení obsahu interakčních bodů se některým uživatelům líbil (viz obrázek 5.2c), neboť jej subjektivně vnímali jako „zajímavější“ a „estetičtější“ formu prezentace informací. Z hlediska jeho potenciální efektivity při praktickém používání však byl celkový uživatelský odhad označen jako nízký.

Respondenti uvedli, že z jejich hlediska by nejvhodnějším způsobem prezentace informací bylo zobrazení úplného textu po kliknutí na interakční bod. V režimu „krok za krokem“ však většina z nich požaduje zobrazit pouze název kroku (který by měl stručně popsat prováděnou akci). Důvodem je skutečnost, že při opakovaném využívání návodu ve většině případů postačuje připomenutí správného pořadí kroků, aniž by bylo nutné znovu číst podrobná vysvětlení.

Možnost pozastavení obrazu

U některých respondentů vyvolávalo tlačítko pauzy v počáteční fázi nejistotu ohledně svého významu. Po následném vysvětlení jeho funkce v kontextu reálné aplikace však uživatelé porozuměli jeho účelu a způsobu použití. Zároveň byl vznesen návrh nahradit ikonu pauzy ikonou fotoaparátu za účelem zvýšení srozumitelnosti a intuitivnosti ovládacího prvku.

Krokové scénáře

Ve vztahu k režimu postupných scénářů, bylo uvedeno, že mohou nastat problémy s rychlým prováděním po sobě jdoucích akcí. Při tvorbě scénářů je rovněž vhodné explicitně uvádět očekávaný následek dané akce, aby uživatel mohl rychleji posoudit, zda postupuje správným způsobem.

Z užitečných funkcí bylo navrženo přidání dodatečné kontroly přítomnosti odpovídajícího markeru, která by zabránila uživateli přejít k dalšímu kroku v případě, že některá akce nebyla vykonána. Tato funkcionalita by měla minimalizovat riziko neúmyslného vynechání kroků v rámci scénáře.

Je však třeba zdůraznit, že implementace uvedeného mechanismu může mít negativní dopad v situacích, kdy uživatel omylem uzavře scénář nebo aplikaci. Z tohoto důvodu se doporučuje, aby implementované řešení neblokovalo možnost přejít k dalšímu kroku, ale aby uživatele pouze varovalo a ponechalo mu finální rozhodnutí o pokračování v postupu.

Výběr scénáře

Někteří uživatelé na počátku předpokládali, že po kliknutí konkrétního scénáře dojde k zobrazení odpovídající část z elektronické verze uživatelského manuálu. Reálné fungování scénářů však bylo účastníky vnímáno jako efektivnější a pohodlnější způsob interakce se systémem.

Kladně bylo hodnoceno rovněž členění scénářů do „kategorií“. Současně se však objevila obava, že při větším množství dostupných scénářů může být identifikace a nalezení relevantního scénáře časově náročná a kognitivně zatěžující.

Někteří respondenti dále uvedli, že by preferovali integraci elektronické verze manuálu přímo do aplikace, a to včetně funkcionalit pro textové vyhledávání a pro zobrazení scénářů asociovaných s jednotlivými kapitolami manuálu.

Kapitola 6

Implementace

Tato kapitola popisuje praktickou realizaci navrženého systému. Zatímco předchozí části práce se soustředily na analýzu, návrh uživatelského rozhraní a vymezení způsobů realizace, zde je pozornost zaměřena na konkrétní technické řešení jednotlivých komponent aplikace. Cílem kapitoly je představit způsob, jakým byly implementovány klíčové funkční celky – od struktury projektu v prostředí Unity, přes zpracování AR vstupů a práci s interakčními body, až po komunikaci s backendem.

Implementace byla vedena snahou o jednoduchost, modularitu a snadnou rozšiřitelnost. Vzhledem k tomu, že aplikace pracuje v prostředí rozšířené reality a je určena pro mobilní zařízení, byl kladen důraz na stabilitu, výkon a minimalizaci zbytečné zátěže. Kapitola proto popisuje nejen výsledné řešení, ale také některé technické volby, které byly učiněny s ohledem na omezení platformy a praktické potřeby uživatelů.

6.1 Použité technologie a nástroje

Implementace aplikace byla realizována s využitím několika technologií a knihoven, které společně umožňují zpracování obrazu, detekci markerů, práci s rozšířenou realitou a správu dat. Tato část stručně popisuje jednotlivé nástroje a jejich roli v systému.

Unity a AR Foundation

Základ aplikace je implementován v herním enginu Unity 6.2[1], který byl zvolen díky své multiplatformnosti, dostupné dokumentaci a především díky oficiální podpoře rozšířené reality prostřednictvím balíčku AR Foundation[2]. Ten poskytuje sjednocenou vrstvu nad ARCore[3] (Android), ARKit[4] (iOS) a ostatními technologiemi pro jiné zařízení, což umožňuje implementovat AR logiku nezávisle na konkrétní platformě.

Důležitou vlastností Unity je rovněž podpora nativních pluginů, která je v kontextu této implementace klíčová. Proces kompilace těchto pluginů je realizován v závislosti na cílové platformě, čímž je zachována multiplatformní kompatibilita a současně je umožněno nasazení individualizovaných řešení odpovídajících specifickým požadavkům systému.

[1] Unity 6.2 User Manual. Dostupné z https://docs.unity3d.com/6000.2/Documentation/Manual/UnityManual.html

[2] AR Foundation: Documentation. Dostupné z https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@6.2/manual/index.html

[3] ARCore: Google's augmented reality SDK. Dostupné z https://developers.google.com/ar?hl=en

[4] ARKit: Apple's augmented reality SDK. Dostupné z https://developer.apple.com/augmented-reality/arkit/

Jazyky programování

Aplikace využívá několik programovacích jazyků, z nichž každý plní specifickou úlohu:

- **C#** – je v současné době hlavním a jediným podporovaným jazykem pro psaní logiky aplikací v Unity.
- **C++** – jazyk použitý při psaní vlastního nativního pluginu, který umožnil začlenit logiku rozpoznávání markerů do aplikace napsané v C#.
- **Python** – byl použit při psaní backendu, byl vybrán vzhledem k snadnosti realizace potřebné funkcionality v této části aplikace.

Knihovny pro zpracování QR a detekci markerů

Ačkoli bylo technicky možné implementovat vlastní řešení těchto problémů, tato funkcionalita nebyla hlavním cílem této práce a její realizace by vývojový proces výrazně zkomplikovala a prodloužila. Z tohoto důvodu byly využity následující knihovny:

- **ZXing**[5] (Zebra Crossing) – open-source knihovna pro detekci QR kódů. Používá se při inicializaci aplikace pro identifikaci zařízení a načtení příslušného manuálu.
- **OpenCV** [37, 13] – knihovna pro zpracování obrazu, využitá zejména pro detekci a výpočet pozic markerů.

Backendové technologie

Backendová část systému byla implementována v jazyce `Python` s využitím technologií, jako jsou:

- `FastAPI` – framework který poskytuje jednoduchý a výkonný způsob tvorby REST API.
- `SQLAlchemy` – slouží jako objektově-relační mapovací (Object Relational Mapping, ORM) vrstva.
- `Pydantic` – pro validaci a serializaci datových modelů.

Pro účely vývoje a testování byla použita databáze `SQLite`, která nevyžaduje samostatnou instalaci a umožňuje rychlou iteraci nad datovým modelem. Samotná API služba je spouštěna pomocí `Uvicorn`, lehkého ASGI (Asynchronous Server Gateway Interface) serveru optimalizovaného pro asynchronní aplikace.

6.2 Struktura projektu v Unity

Projekt je organizován podle principů modularity, jasného rozdělení odpovědností a snadné orientace pro vývojáře. Aplikace používá jednu scénu: všechny režimy (skenování QR, prohlížení zařízení, krokové scénáře, režim pro vývojáře) jsou realizovány jako stavy a sada UI panelů uvnitř této scény. To umožňuje vyhnout se restartu AR modulu při přepínání režimů a minimalizovat dobu načítání.

[5] Zebra Crossing. Dostupné z https://github.com/zxing/zxing

Klíčové skripty a jejich role

Pro lepší orientaci a pochopení principů fungování v rámci aplikace je také vhodné se seznámit se klíčovými skripty:

- `Core/`
  - `AppCoordinator.cs` – slouží jako centrální orchestrátor systému. Zajišťuje aktivaci a deaktivaci jednotlivých subsystémů v závislosti na aktuálním stavu aplikace a dohlíží na správnou posloupnost kroků v rámci těchto stavů.
  - `CameraController.cs` – poskytovatel CPU snímků (které jsou následně použity v `QrScanner.cs` a `ArUcoDetector.cs`) a dat o kameře, zároveň koordinuje frekvenci dotazování kamery, což má následně vliv na stabilitu detekce.
- `Detection/`
  - `ArUcoDetector.cs` – napojení nativního pluginu (C++): odesílání snímků, přijímání detekcí (id, rohy, pose).
  - `MarkerDetectionResult.cs` – DTO (Data Transfer Object) výsledku detekce markeru pro další zpracování v pohodlné formě.
  - `QrScanner.cs` – dekódování QR pomocí knihovny ZXing, předání výsledku do `AppCoordinator.cs`.
- `ManualSession/`
  - `ScenarioManager.cs` – správce scénářů v rámci manuálu: seskupování, plnění seznamu v UI, výběr scénáře.
  - `StepScenarioManager.cs` – kompletní správa krokových scénářů: implementace systému přepínání kroků, hlášení o aktivním bodu interakce.
- `Network/`
  - `APILoader.cs` – představuje rozhraní pro komunikaci se zdrojem dat.
  - `IManualDataSource.cs` – abstrakce nad zdrojem dat, třídy používající toto rozhraní mohou být použity v `APILoader.cs` pro dotazy na data.

Je třeba poznamenat, že interakce těchto skriptů s uživatelským rozhraním je minimalizována, veškeré operace jsou realizovány prostřednictvím dodatečné abstrakční vrstvy ve formě skriptů umístěných ve složce `UI/`. Toto uspořádání je zvoleno s cílem maximalizovat oddělení aplikační logiky od uživatelského rozhraní. Uvedený přístup umožňuje flexibilně modifikovat funkcionalitu nebo vizuální podobu systému bez nutnosti zasahovat do druhé z těchto vrstev.

Využitím orchestrátoru `AppCoordinator.cs` je umožněna rychlá záměna jednotlivých subsystémů, například nahrazení aktuální implementace detektoru její novou verzí. Použití rozhraní bylo nicméně omezeno z důvodu absence podpory těchto konstrukcí v editoru Unity, což mělo negativní dopad na modularitu systému, avšak nikoli v kritickém rozsahu.

Komunikace mezi moduly je realizována pomocí událostmi řízené architektury. Tento přístup usnadňuje jak jejich testování, tak i následnou údržbu a aktualizaci. Jednotlivý modul, stejně jako jeho vývojář, by měl mít přehled pouze o své vlastní funkcionalitě a o informacích, které může získat od ostatních modulů prostřednictvím odběru jejich událostí, s několika výjimkami. Současně poskytuje výsledky svých výpočtů ostatním modulům opět ve formě událostí.

6.3 Implementace AR části v Unity

V této části budou podrobně popsány specifické aspekty a zdůvodnění jednotlivých rozhodnutí přijatých v průběhu implementace výše uvedeného návrhu v kapitole 4.

V rámci implementace bylo zvoleno souběžné využití dvou přístupů popsaných v kapitole 3.1, konkrétně markerless a marker-based sledování. Tato kombinace umožnila snížit výpočetní zátěž na cílovém zařízení, neboť po inicializaci a nalezení markerů je jejich poloha znovu přepočítávána pouze v případě rekalibrace, jak je podrobně popsáno v kapitole 4.5.

Během experimentálního ověřování v reálných podmínkách se nicméně ukázalo, že stabilita rozpoznávání není dostatečná k zajištění přijatelné úrovně uživatelského komfortu. Stávající řešení zároveň vykazuje zásadní limitaci spočívající v neschopnosti spolehlivě detekovat pohybující se markery, tedy markery, které si neudržují svou pozici vůči fyzickému prostředí. Vzhledem k modulární architektuře systému a zvolenému vývojovému přístupu však byla zachována možnost tento algoritmus dále rozšiřovat a optimalizovat.

Markerless přístup je umožněn využitím komponenty `AR Plane Manager`. Přítomnost této komponenty ve scéně iniciuje sběr vizuálních příznaků a na jejich základě generuje ve scéně rovinné plochy, které následně slouží ke stabilizaci pozice kamery v daném prostředí. Tímto způsobem objekty umístěné v prostoru scény dlouhodobě zachovávají svou prostorovou polohu a v případě chyb či výpadků ve sledování polohy uživatele jsou schopny tuto pozici rychle a konzistentně rekonstruovat.

Získávání snímků z kamery

Pro zefektivnění procesu získávání aktuálního snímku z kamery uživatele byl vytvořen jeden modul – `CameraController.cs`. Tento modul získává snímky z CPU pomocí vestavěné funkce `ARCameraManager.TryAcquireLatestCpuImage(out XRCpuImage image)`, která po zpracování šíří data prostřednictvím události `Action<byte[], int, int>`, díky čemuž může každý odběratel samostatně převést obraz do jím požadované podoby.

Mezi hlavní odběratele této události lze zařadit následující:

- `ArUcoDetector.cs` – přijímá snímky pro jejich další použití při rozpoznávání markerů. Ukládá pole bajtů v paměti a chrání je před uvolňováním pomocí `GCHandle.Alloc()`, díky čemuž je později možné získat odkaz na jeho adresu.
- `QrScanner.cs` – používá obrázek k rozpoznání QR kódu. Převádí pole bajtů do podoby textury pro správné zpracování knihovnou ZXing.

Je také vhodné zmínit `FreezeManager.cs`, který byl v předchozích verzích rovněž odběratelem této události. Od tohoto přístupu však bylo nutné upustit, neboť snímky zpracovávané na úrovni CPU se liší od snímků, které jsou ve skutečnosti prezentovány uživateli na výstupním zobrazovacím zařízení. Během vývoje tato skutečnost vedla k celé řadě komplikací, především při implementaci modulu detekce. Kromě toho novější verze Unity již neposkytují transformační matici umožňující převod snímku z CPU do podoby odpovídající vykreslenému obrazu.

Tento modul rovněž zodpovídá za poskytování parametrů kamery, které se používají ke zvýšení přesnosti rozpoznávání. V dané implementaci však nedochází k odstranění zkreslení kamery, které bylo popsáno v kapitole 3.3, protože Unity tuto operaci provádí automaticky.

Rozhraní pro ArUco nativní plugin

Samotná implementace tohoto pluginu bude podrobněji popsána v kapitole 6.4, zatímco `ArUcoDetector.cs` poskytuje abstraktní rozhraní pro práci s ním. K volání funkcí dochází prostřednictvím konstrukce `[DllImport("plugin_name")] static extern` vestavěné v Unity.

Pro správné fungování externích subsystémů postačuje vědět, že informace o detekovaných markerech jsou předávány prostřednictvím DTO: `MarkerDetectionResult` a události `Action<List<MarkerDetectionResult>> OnMarkersDetected`. Tato událost nese souřadnice rohů markerů v obrazové rovině (důležité: poloha je určena vzhledem k CPU obrazu, jak již bylo uvedeno). V případě potřeby specifikace 6DoF vzhledem k uživateli lze přímo využít funkci `EstimatePose(...)`.

Další zpracování markerů

Po získání informace o poloze markeru na snímku je v souladu s postupem popsaným v kapitole 4.5 nutné rozhodnout, zda bude dále provedeno zpřesnění jeho 6DoF a zda s tímto markerem bude nadále operováno v navazujících částech systému. Za tuto funkcionalitu v současné době zcela zodpovídá subsystém definovaný v `MarkerManager.cs`. Takové řešení však není nejlepší praxe a s ohledem na budoucí rozšiřování systému bude nezbytné uvedenou logiku dekomponovat do většího počtu modulů.

Současná funkcionalita tohoto subsystému zahrnuje:

- Operace konverze realizovaná funkcí `OpenCvToUnityWorldPose(...)`, která zajišťuje transformaci pozice ze souřadnicového systému používaného knihovnou `OpenCV` (reprezentovaného dvojicí trojrozměrných vektorů popisujících translační posun vůči kameře uživatele v metrech a rotační složku ve tvaru Rodriguesovy reprezentace) do formátu kompatibilního s Unity, konkrétně do instance třídy `Pose`.
- Vytváření nových a ukládání stávajících markerů umístěných ve scéně.
- Funkcionalita pro zobrazení konkrétních bodů interakce díky funkci `ShowOnly(...)`.

Při obdržení informace o markerech, které dosud nebyly detekovány, je inicializován proces jejich vytvoření, přičemž každý marker je prostřednictvím třídy `ARAnchor` umístěn přímo do globálního prostoru scény. Tento proces zahrnuje požadavek na výpočet 6DoF; v případě úspěšného výpočtu je následně proveden pokus o vytvoření kotevního bodu (anchor). Vzhledem k tomu, že tento postup probíhá asynchronně vůči hlavnímu běhu programu, jsou informace o probíhajících operacích ukládány do fronty, která zajišťuje, že pro jeden marker není vytvořeno více kotev. Při úspěšné tvorbě kotevního bodu je vytvořen podřízený objekt této kotvy, jenž využívá skript `MarkerController.cs`.

`ARAnchor` byl zvolen jako robustnější a stabilnější alternativa k jednoduchému umístění bodů ve scéně, neboť disponuje schopností automatické rekalibrace bez nutnosti manuální intervence, a to při minimálních nárocích na výpočetní výkon.

V případě, že byl daný marker již dříve identifikován, je skriptu `MarkerController.cs` předána informace o jeho aktuální velikosti vzhledem k zobrazené scéně. Na základě této informace skript rozhoduje, zda je nutné provést dodatečnou rekalibraci, která je následně vyžádána od detekčního modulu (popsaného v předchozí části).

Oddělení logiky jednoho markeru

Zatímco `MarkerManager.cs` zajišťuje správu všech existujících markerů ve scéně, odpovědnost za operace na úrovni jednotlivých bodů je delegována na `MarkerController.cs`.

Díky tomu se zjednodušuje logika v `MarkerManager.cs` a logicky se oddělují oblasti odpovědnosti. K základním funkcím této třídy patří:

- Inicializace interakčních bodů.
- Rozhodnutí o nutnosti rekalibrace.
- Rozhodnutí o nutnosti skrýt nebo zobrazit jak celý marker, tak i jednotlivé body interakce.
- Část logiky funkce zmrazení, která zmrazuje pozici ukazatele vzhledem ke kameře uživatele.
- Logika adaptace podle vzdálenosti, která byla popsána v kapitole 5.2.

Skrytí markeru nastává v případě, že marker není po určitou dobu viditelný. Tímto způsobem se eliminuje zbytečné využívání výpočetních a systémových prostředků na markery, které se aktuálně nenacházejí v oblasti viditelnosti, a současně se částečně zmírňuje problém okluze v AR prostoru, je třeba poznamenat, že tato metoda problém okluze zcela neodstraňuje, nicméně jej redukuje v takové míře, aby byla interakce s aplikací pro uživatele dostatečně komfortní.

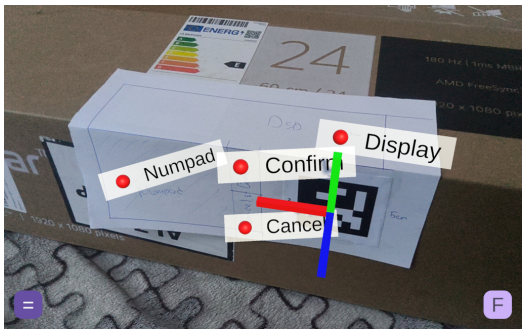
Rekalibrace se provádí při přiblížení k markeru. Jelikož existence této funkcionality byla určena k optimalizaci výpočtu 6DoF, bylo nutné vzdálenost určovat jiným způsobem. Jako nejjednodušší a nejúčinnější se ukázal přístup, při němž si program zapamatuje velikost markeru vzhledem ke vstupnímu CPU obrazu. Pro výpočet této charakteristiky se používá vzorec $(width+height)/2$. Přes svoji jednoduchost tento vzorec dobře reaguje na velké úhly, v důsledku čehož rekalibrace při naklonění vyžaduje větší přiblížení uživatele k markeru. To zvyšuje stabilitu a přesnost.

Z hlediska adaptace podle vzdálenosti je skrytí či zobrazení všech interakčních bodů (viz obr. 6.1b) řízeno aktuální relativní velikostí markeru. Po překročení prahové hodnoty (`CollapseThreshold`) může uživatel zobrazit všechny interakční body, zatímco při poklesu relativní velikosti markeru pod minimální práh (`CollapseThresholdMin`) jsou tyto body skryty a zůstane pouze bod zájmu. Tento přístup řeší dvě klíčové oblasti: redukci vizuálního šumu a zachování dostatečné přesnosti detekce. Jedním z hlavních omezení knihovny ArUco je snížená přesnost detekce na větší vzdálenost (viz kapitola 3.2). Ve větší pozorovací vzdálenosti však není vysoká přesnost nutná, uživatel si chyb obvykle nevšimne a z hlediska funkčnosti systému nejsou kritické.

Implementace interaktivních bodů

Při inicializaci interakčních bodů je vytvořen objekt, který je potomkem markeru, konkrétně je využit prefab `Prefabs/InteractionPoint`, který obsahuje strukturu jednotlivého bodu a je k němu připojen skript `InteractionPointController.cs`. Toto řešení umožňuje oddělit logiku inicializace. Konkrétně třída `InteractionPointController` odpovídá za následující činnosti:

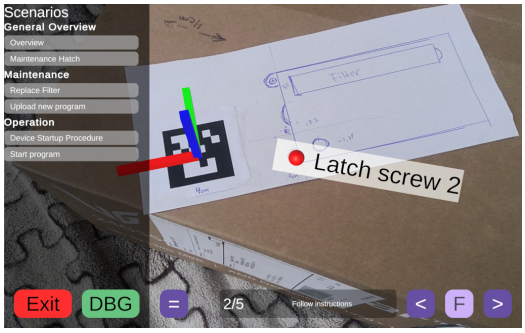
- Správné nastavení velikosti interakčního bodu.



(a) Příklad markeru



(b) Skrytý marker



(c) Scénáře a krokové rozhraní

Obrázek 6.1: Příklad uživatelského rozhraní aktuální verze aplikace. Pro zvýšení přehlednosti je poloha markeru vyznačena pomocí souřadných os. Na obrázku `a` je zobrazen marker s několika definovanými body interakce. Na obrázku `b` je zobrazen tentýž marker, avšak skrytý z důvodu velké vzdálenosti uživatele od jeho polohy. Obrázek `c` ilustruje rozhraní scénářů a současně znázorňuje proces vykonávání krokového scénáře.

- Zobrazení ve formě billboardu (text je vždy natočen směrem ke kameře).

Původní návrh předpokládal zobrazování interakčních bodů přímo v uživatelském rozhraní (UI), jak bylo uvedeno v kapitole 5.3. V průběhu vývoje se však ukázalo, že umístění 3D objektů přímo do scény (viz obr. 6.1a a 6.1c) představuje vizuálně vhodnější řešení.

Logika zobrazení ve formě billboardu navíc nebyla realizována na úrovni shaderu, nýbrž byla implementována jako rotace 3D objektu v místě interakce směrem ke kameře uživatele. Dopad tohoto přístupu na výkonnostní optimalizaci je zanedbatelný, nicméně výsledný vizuální dojem je výrazně estetičtější.

Konečná struktura markeru

Jako výsledek předchozích kroků získáme následující strukturu markeru:

- `ARAnchor`
  - `Prefabs/MarkerRoot` – je řízen třídou `MarkerController`.
    - `Prefabs/AnchorPref`
      - `MeshRenderer`
    - `Prefabs/MarkerPlaceholder`
      - `MeshRenderer`

* `iPointHolder_`*`Marker_ID`*
  - `Prefabs/InteractionPoint`.
  - `...`

Tato struktura může na první pohled působit nadbytečně komplexně, nicméně umožňuje jednoznačné vymezení oblastí odpovědnosti a současně definici interakcí mezi komponentami takovým způsobem, že vyšší úrovně mají znalost pouze o existenci svých podřízených a o jejich poskytované funkcionalitě.

Kotva a kořenový objekt byly odděleny z toho důvodu, že v případě, kdy se marker nepodaří detekovat a je skryt, dojde k deaktivaci objektu, avšak instance `ARAnchor` při deaktivaci ztrácí svou pozici ve světě. Proces rekalibrace současně ovlivňuje pouze lokální pozici `MarkerRoot`, neboť pro změnu polohy kotvy je nezbytné její odstranění a následné znovuvytvoření, což představuje z hlediska výpočetní náročnosti netriviální operaci.

`Prefabs/MarkerPlaceholder` reprezentuje objekt, který je zobrazen v situaci, kdy dojde ke „svinutí“ markeru na velkou vzdálenost, jak bylo diskutováno v předchozí části textu. Jeho vydělení jako samostatného objektu umožnilo dosáhnout vyšší úrovně flexibility během vývoje systému a zároveň usnadnilo implementaci mechanismů pro přepínání jednotlivých stavů zobrazení.

Vyčlenění interakčních bodů do struktury `iPointHolder_`*`Marker_ID`* umožnilo implementaci funkce zmrazení snímku, která bude podrobněji popsána v následující části.

Funkce zmrazení snímku

Implementace této funkčnosti přinesla do tohoto projektu několik nestandardních řešení. Jak již bylo uvedeno, CPU snímek neodpovídal vizuálnímu výstupu, který uživatel v průběhu používání aplikace pozoruje na své obrazovce. Z tohoto důvodu byl pro její realizaci navržen a implementován následující algoritmus:

1. Při spuštění se zavolá `StartCoroutine(FreezeCoroutine())`, což následně umožní funkci běžet po dobu delší než jeden snímek.
2. Poté program deaktivuje plátno (prvek, na kterém Unity vykresluje uživatelské rozhraní).
3. Dále díky ‘WaitForEndOfFrame()‘ program vyčká na konec snímek, během něhož Unity spustí renderování GUI. Vzhledem k tomu, že jsme ale dříve vypnuli plátno, nebude se vykreslovat.
4. Pomocí metody `Texture2D.ReadPixels()` program načte do textury obraz, který byl vykreslen na obrazovku uživatele.
5. Dříve zobrazený obrázek se uživateli zobrazuje pomocí instance `RawImage`.
6. Plátno se znovu zapne, aby uživatel mohl používat uživatelské rozhraní.
7. Díky události jsou komponenty informovány o tom, že program přešel do režimu pauzy. Jedním z odběratelů je třída `MarkerController`, která následně odpojí interakční body od kotvy, což jim umožňuje zachovat stabilní polohu vzhledem k poloze uživatelovy kamery.

Proces ukončení pauzy probíhá v zásadě podle totožného principu, avšak bez jakékoli dodatečné manipulace s texturou.

V průběhu implementace bylo ověřeno několik alternativních přístupů, ale jako nejstabilnější byla zvolena aktuálně popisovaná varianta. Zastavení komponenty `AROrigin`, která představuje ústřední prvek architektury AR Foundation, totiž vede jak k narušení sledování prostorové pozice uživatelské kamery, tak k nefunkčnosti již existujících kotev. Pokusy o pozastavení kamerového výstupu vedly k narušení funkce všech dílčích AR subsystémů.

Interakční body jsou proto odpojovány od příslušné kotvy, aby s nimi uživatel mohl nadále pracovat bez nutnosti jejich znovuvytváření či implementace dodatečných algoritmů.

6.4 Nativní plugin ArUco využívající OpenCV

Rozhodnutí o vytvoření vlastního řešení vyplynulo z toho, že v současné době neexistují otevřené a dostupné varianty s funkcionalitou potřebnou pro tento projekt. V Unity Asset Store[6] jsou sice k dispozici některé knihovny umožňující práci s markery, avšak většina z nich je buď placená, nebo nepodporuje nejnovější verze Unity, na nichž samotný vývoj probíhal.

Integrace vlastního řešení také umožňuje optimalizovat projekt využitím pouze nezbytné funkcionality, přidat flexibilitu do implementace a rovněž přesunout složité operace mimo Unity a C#.

Výběr knihovny pro detekci

Knihovna OpenCV byla zvolena z důvodu své otevřenosti, rozsáhlé sady implementovaných funkcí a zároveň kvalitní a přehledné dokumentace. Jak již bylo zmíněno, existovala možnost navrhnout a implementovat vlastní algoritmus rozpoznávání bez využití externích knihoven, avšak hlavním cílem této práce je realizace univerzálního, opakovaně použitelného řešení určeného pro AR manuály.

V rámci této práce používáme následující komponenty:

- `opencv2/core.hpp` – Základní funkce.
- `opencv2/imgproc.hpp` – Zpracování obrazu.
- `opencv2/objdetect/aruco_detector.hpp` – Základní funkce pro detekci ArUco markerů.
- `opencv2/calib3d.hpp` – Kalibrace kamery a 3D rekonstrukce (SolvePnP).

Inicializace pluginu

Pro zahájení práce s detekcí je nejprve nutné provést úvodní nastavení, které bylo popsáno v kapitole 3.3. Pro zjednodušení je třeba pouze zvolit aktuální slovník, jenž závisí na celkovém počtu markerů, které se vyskytují v rámci tohoto manuálu. Nastavení ostatních parametrů se provádí ručně přímo v kódu.

Dále lze pro upřesnění markerů, které je potřeba najít, použít funkce filtrování, která je implementována pomocí `ArucoSetWhitelist(int* markerIds, int count)`, která vylučuje nepotřebné markery z dalšího zpracování. Tuto funkci je přitom možné volat i v

[6] Obchod s assety pro Unity, který zahrnuje jak vizuální prvky, tak hotová řešení. Dostupné z https://assetstore.unity.com/

průběhu běhu programu. Pokud je však předán parametr `count` roven nule, systém začne zpracovávat všechny markery.

Pro korektní funkci systému je nezbytné rovněž definovat parametry kamery, tato operace se realizuje prostřednictvím volání funkce `ArucoSetCameraIntrinsics(...)`. Uvedenou funkci lze vyvolávat i za běhu programu, neboť při otočení kamery dochází ke změně jejích vnitřních parametrů. V aktuální verzi je však kamera v prostředí Unity fixována, takže k volání této funkce dochází pouze v průběhu inicializace.

Detekce markerů

Proces detekce byl z hlediska teorie popsán v kapitole 3.3. V programu můžeme každý z uvedených kroků samostatně modifikovat prostřednictvím konfiguračních parametrů, přičemž tyto zásahy mohou mít jak pozitivní, tak i negativní vliv na výslednou kvalitu detekce.

Mezi specifické rysy implementace lze zařadit proces předzpracování obrazu na úrovni CPU. Vzhledem k tomu, že prostředí Unity předává snímek ve formátu RGBA (což je požadováno pro korektní fungování knihovny ZXing), je obraz v rámci aplikace převáděn do odstínů šedi a současně je rotačně transformován o 180° kolem své osy. Tato úprava je nezbytná pro následný správný výpočet 6DoF.

Po detekčním procesu lze k získání počtu v předchozí operaci identifikovaných markerů využít funkci `ArucoGetCount()`. ID rozpoznaných markerů jsou dostupné prostřednictvím funkce `ArucoGetId(...)`, přičemž markery jsou interně ukládány v poli v pořadí, v jakém byly detekovány. Souřadnice rohů rozpoznaných markerů lze následně zpřístupnit pomocí funkce `ArucoGetCornersByID(...)`. Uvedená modularizace funkcionality může na první pohled působit neintuitivně, avšak umožňuje efektivnější práci v asynchronním prostředí a zároveň se ukázala jako výhodná během vývoje a ladicích fází.

Výpočet 6DoF markeru

Tento proces byl teoreticky popsán v kapitole 3.4. Probíhá pomocí `ArucoEstimatePose()`. V průběhu jejího vývoje bylo vyzkoušeno několik přístupů, avšak jako nejvhodnější se ukázal standardní postup, protože při úpravách nebyla zjištěna žádná výrazná zlepšení.

Jak již bylo opakovaně uvedeno, obraz CPU neodpovídá obrazu prezentovanému uživateli. Tento nesoulad je závislý na konkrétním zařízení a v průběhu testování se nepodařilo nalézt jednotnou transformační matici pro převod souřadnic rohů, která by umožnila korektní vykreslení markeru přímo v uživatelském rozhraní.

Tato odchylka však neovlivňuje samotný proces výpočtu vektorů translačního a rotačního posunu. Jak bylo dříve popsáno, za převod těchto vektorů do souřadnicového systému kompatibilního s Unity odpovídá třída `MarkerManager`. Uvedená funkcionalita byla oddělena z toho důvodu, že výsledná transformační matice závisí na vstupním obrazu, nikoli na samotném procesu zpracování, a současně nepředstavuje výraznou zátěž z hlediska výpočetní náročnosti.

Proces převodu translačního vektoru vypadá následovně:

$$t_u = St_o = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} = \begin{bmatrix} -t_x \\ t_y \\ t_z \end{bmatrix} \tag{6.1}$$

kde t_u je posun v Unity, S je matice transformace posunu a t_o je posun v OpenCV. Hodnoty posunů jsou přitom v obou soustavách udávány v metrech. Po provedení transfor-

mace se také převede lokální posun vzhledem ke kameře do světového souřadného systému pomocí `Camera.TransformPoint(`t_u`)`.

Pro převod rotačního vektoru bude potřeba o něco více operací. Nechť $r = (r_x, r_y, r_z)$ je Rodriguesův vektor z OpenCV, $\theta = \parallel r \parallel$. Kvatérnion odpovídající tomuto rotačnímu vektoru je:

$$q_o = (x, y, z, w) = (u_x sin\frac{\theta}{2}, u_y sin\frac{\theta}{2}, u_z sin\frac{\theta}{2}, cos\frac{\theta}{2}) \tag{6.2}$$

kde $u = r/\theta$ (pro $\theta \approx 0$ bereme $q_o = (0, 0, 0, 1)$). Souřadnicové systémy OpenCV a Unity se liší, proto je třeba provést jejich převod:

$$q_u = Rq_o = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} q_x \\ q_y \\ q_z \\ q_w \end{bmatrix} = \begin{bmatrix} q_x \\ -q_y \\ -q_z \\ q_w \end{bmatrix} \tag{6.3}$$

Poté je třeba, obdobně jako u posunutí, převést lokální rotaci do světových souřadnic pomocí `Camera.Rotation`$\otimes q_u$, kde $\otimes$ je násobení kvaternionů.

Kompilace pluginu

Tento plugin umožňuje kompilaci pro libovolnou platformu podporovanou OpenCV, v případě potřeby je však u některých implementací možné využít i alternativní knihovny. Pokud nedojde ke změně názvů veřejných funkcí ani jejich chování, lze nadále používat třídu `ArUcoDetector` beze změn.

Je však nutné zdůraznit, že při modifikaci funkcionality s cílem zajistit multiplatformní podporu je nezbytné tento plugin zkompilovat samostatně pro každou cílovou platformu a její architektury. Tento proces lze zefektivnit pomocí nástroje `Makefile`.

Proces kompilace dále dynamicky zjišťuje a propojuje potřebné balíčky prostřednictvím nástroje `CMake`, takže je není nutné do prostředí Unity explicitně přidávat či konfigurovat ručně.

6.5 Backend

Backend představuje serverovou část systému, která zajišťuje poskytování všech dat nezbytných pro běh aplikace. Je nezbytné zdůraznit, že v daném kontextu se jedná pouze o prototyp, který pokrývá požadovanou funkcionalitu klientské (mobilní) aplikace.

Backend je koncipován jako samostatná a modulární komponenta, která zajišťuje oddělení logiky pro zpracování a správu dat od klientské části postavené v Unity. Takto navržená architektura usnadňuje údržbu systému, podporuje jeho škálovatelnost a budoucí rozšiřování a současně poskytuje jednoznačně definované rozhraní pro komunikaci a výměnu dat mezi oběma vrstvami aplikace.

Návrh API a hlavní endpointy

API backend-části byla navržena s ohledem na omezení mobilního klienta a specifika prostředí Unity. Hlavním cílem bylo zajistit minimální objem přenášených dat, předvídatelnou strukturu odpovědí a možnost efektivního načítání souvisejících entit. Architektura API je postavena kolem několika klíčových endpointů, které pokrývají celý životní cyklus práce s manuly.

Celkově je zapotřebí definovat pouze čtyři endpointy, níže podrobněji o každém z nich:

- `GET /manuals/id` – načítání manuálu.

  Vrací základní informace o manuálu podle zadaného ID. Do této struktury jsou také zahrnuty scénáře bez interakcí.

- `GET /scenarios/id/interactions` – načítání interakcí v rámci jednoho scénáře.

  Vrátí seznam interakcí pro konkrétní scénář.

- `POST /interactionpoints/batch` – načítání skupiny interakčních bodů.

  Přijímá seznam ID kontaktních bodů a vrací informace o těchto bodech rovněž ve formě seznamu.

- `POST /trackpoints/batch` – načítání skupiny markerů.

  Přijímá seznam ID markerů a vrací informace o těchto markerech také ve formě seznamu.

Představené datové struktury odpovídají těm, které jsou specifikovány v kapitole 4.4, a jsou reprezentovány ve formátu `JSON`. Tato volba formátu umožňuje efektivní a pohodlné zpracování dat v prostředí Unity. Navržené rozdělení struktur vychází primárně ze snahy o optimalizaci paměťových nároků. Vzhledem k předpokládanému použití velkého množství markerů není z hlediska efektivního využití účelné načítat celý obsah manuálu okamžitě.

Serializace dat a příprava pro Unity

Jedním z klíčových úkolů backendové části je příprava dat v takovém formátu, který může být správně zpracován klientskou částí postavenou v Unity. Vzhledem k tomu, že tato klientská část využívá vestavěný mechanismus `JsonUtility`, jenž uplatňuje specifická omezení na strukturu JSONu (například neexistenci přímé podpory slovníků, vnořených anonymních struktur a nestrukturovaných seznamů bez pojmenovaného záhlaví), je nezbytné, aby backend poskytoval data ve striktně definovaném a předvídatelném formátu.

Za tímto účelem je zavedena vrstva schémat `schemas/`, která specifikuje přesnou strukturu výstupních dat a zajišťuje jejich transformaci z ORM modelů do serializovatelných datových struktur. Tento přístup umožňuje jednoznačné oddělení vnitřní doménové reprezentace entit od jejich síťové podoby a garantuje, že klientská aplikace vždy obdrží data kompatibilní s jejím interním datovým modelem.

Struktura síťové části v Unity

Síťový subsystém v Unity byl navržen jako autonomní modul, striktně oddělený od veškeré ostatní aplikační logiky. Primárním cílem tohoto architektonického přístupu je umožnit transparentní záměnu zdroje dat bez nutnosti modifikace ostatních částí projektu.

Pro interakci s tímto podsystémem je zavedena fasáda `Network/APILoader.cs`, která v současné podobě zpřístupňuje pouze nezbytnou množinu operací odpovídajících dříve popsaným endpointům. Jako datový zdroj jsou využívány třídy implementující rozhraní `Network/IManualDataSource.cs`. V současnosti jsou k dispozici dvě implementace uvedeného rozhraní:

- `Network/FakeManualDataSource.cs` – implementace využívající data získaná ze souboru `Tests/DummyData.cs`. Existuje především pro testovací účely.

- `Network/APIManualDataSource.cs` – implementace používající připojení ke skutečnému API prostřednictvím `UnityWebRequest`.

Všechny otázky týkající se serializace dat jsou také řešeny konkrétní implementací s ohledem na specifika získaných dat. Do této struktury je možné v budoucnu také přidat možnost využití paměti zařízení pro lokální přístup k manuálům, což bylo popsáno v kapitole 2.3 jako princip decentralizace.

Wrappery a konverze dat

Vzhledem k omezením třídy `JsonUtility` v Unity není možné přímo deserializovat některé (většinou interakční body) JSON struktury poskytované backendem. Z tohoto důvodu byla vytvořena sada wrapperů, tedy pomocných tříd, jejichž jediným účelem je přesně odrážet formát přijatých dat a umožnit jejich bezpečnou konverzi do interních modelů aplikace.

Wrappery jsou umístěny ve složce `Network/APIWrappers/` a jsou navrženy tak, aby:

- odpovídaly struktuře JSONu vráceného API,
- používaly pouze datové typy a strukturu podporované `JsonUtility`,
- neobsahovaly žádnou aplikační logiku,
- sloužily jako mezivrstva mezi síťovou komunikací a interními modely.

V některých případech je možná deserializace přímo do modelu používaného v aplikaci, v jiných je druhým krokem konverze dat převod wrapperu na interní model.

Kapitola 7

Testování

Cílem této kapitoly je ověřit funkčnost, stabilitu a praktickou použitelnost navrženého systému v reálných podmínkách. Testování se zaměřuje jak na technické aspekty, tak na uživatelskou zkušenost při práci s interaktivním manuálem.

Součástí testování je rovněž porovnání s tradiční papírovou dokumentací, které umožňuje posoudit přínosy i limity navrženého řešení v kontextu běžných pracovních úloh. Výsledky jsou rozděleny na statistické metriky a empirické hodnocení respondentů, což poskytuje komplexní pohled na chování systému.

7.1 Metodika testování

Manuály a scénáře použité při testování nebyly vytvořeny nahodile. Jejich struktura a obsah byly odvozeny z reálně se vyskytujících situací, v nichž se tento typ instrukcí uplatňuje, a také z pracovních postupů pozorovaných ve výrobním prostředí. Zároveň byly zohledněny podněty několika uživatelů, kteří mají zkušenosti s obsluhou a údržbou zařízení ve výrobě.

Testování se zúčastnilo osm respondentů ve věku od 19 do 31 let. Účastníci se lišili úrovní zkušeností: od uživatelů, kteří se s technickými návody setkávají pouze občas, až po osoby s praxí v průmyslové výrobě či v technických profesích. Takto zvolená skladba vzorku umožnila posoudit hodnocený systém jak z perspektivy začínajících uživatelů, tak z hlediska osob zvyklých pracovat s dokumentací v náročnějších provozních podmínkách.

Testovací prostředí

Prostředí bylo zvoleno tak, aby odpovídalo běžným podmínkám použití:

- Osvětlení (přibližná hodnota v luxech byla určena převodem ISO→EV100→lux):
  - Intenzivní přirozené osvětlení (sluneční světlo, ≈ 257 lux).
  - Uměle zhoršené podmínky (nízké osvětlení, stíny, ≈ 100 lux).
  - Stabilní umělé osvětlení (LED/žárovky ≈ 498 lux).
- Mobilní zařízení:
  - Tablet o hmotnosti 469 gramů.
  - Úhlopříčka displeje je 11".
  - Zpracovávaný obraz CPU o rozlišení 640 × 480 pixelů.

- Zařízení / imitace zařízení:
  - Malé objekty o rozměrech $30 \times 20 \times 10$ cm (domácí spotřebiče).
  - Velké objekty až do $340 \times 100 \times 250$ cm (imitace výrobního zařízení)
- Markery:
  - Velikost: od 3 do 6 centimetrů.
  - Vytištěné na běžné tiskárně.

Testovaná verze aplikace odpovídá implementaci popsané v kapitole 6, včetně všech subsystémů detekce, sledování a komunikace s backendem.

Srovnání s papírovým manuálem

Součástí metodiky bylo také porovnání navrženého AR manuálu s tradiční papírovou dokumentací. Uživatelé byli požádáni, aby provedli totožné úkoly ve dvou režimech: s využitím papírového manuálu a pomocí AR aplikace. Pořadí režimů bylo mezi účastníky obměňováno, aby se minimalizoval vliv předchozí znalosti zařízení nebo samotného postupu.

Tento postup umožnil identifikovat oblasti, ve kterých AR přináší měřitelný přínos, a zároveň odhalit situace, kde tradiční forma dokumentace stále nabízí výhody.

7.2 Statistické výsledky

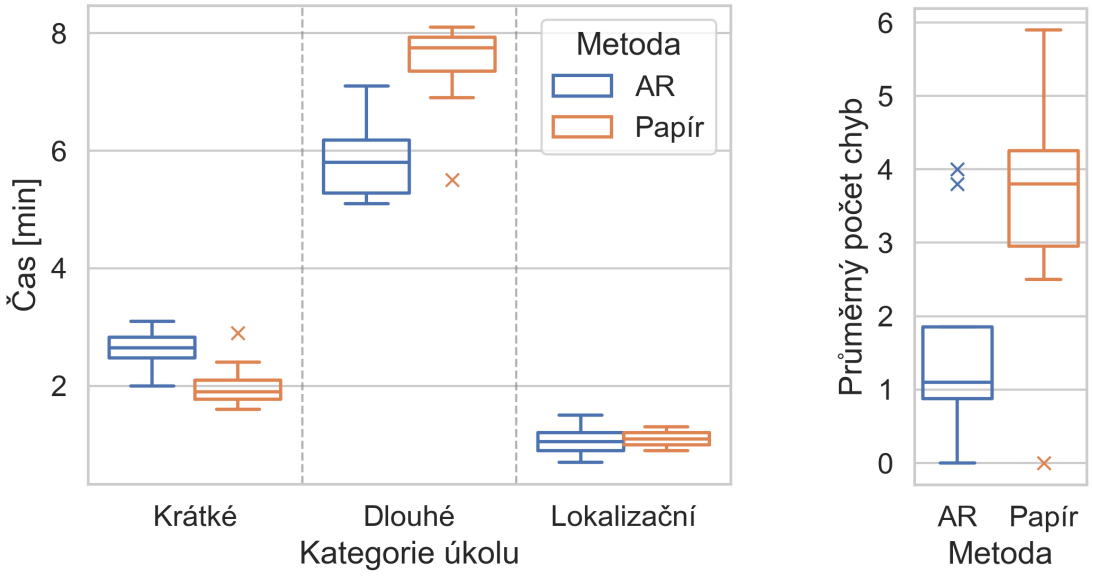
Statistické výsledky poskytují kvantitativní pohled na chování uživatelů při práci s navrženým AR manuálem. Zahrnují jak objektivní měření výkonu při plnění jednotlivých úkolů, tak subjektivní hodnocení uživatelské zkušenosti.

Metriky efektivity

Prvním analyzovaným parametrem byl čas provedení úloh různých typů. Jak je vidět na obrázku 7.1a, rozdíly mezi metodami závisí na charakteru úlohy.

- **Krátké úlohy**. Rozdíly mezi metodami v této kategorii byly minimální: uživatelé plnili úkoly srovnatelnou rychlostí a tištěná dokumentace vykázala pouze nepatrnou výhodu. To lze vysvětlit tím, že při jednoduchých a krátkých činnostech vyhledávání potřebných informací prakticky nevytváří kognitivní zátěž.
- **Dlouhé sekvenční úlohy**. Zde se výhoda AR manuálu projevila nejzřetelněji. Uživatelé netrávili čas hledáním dalšího kroku nebo návratem k předchozím částem návodu – systém je automaticky vedl scénářem. To snižovalo kognitivní zátěž, omezovalo pravděpodobnost chyb a zajišťovalo stabilnější provedení.
- **Lokalizační úlohy**. Rozdíly mezi metodami se ukázaly jako minimální, což souvisí s povahou samotných úloh: nalezení fyzického prvku závisí především na uspořádání objektu a podmínkách okolního prostředí, a nikoli na formě podání instrukce. Proto obě metodiky vykazovaly podobné výsledky.

Druhým ukazatelem byl průměrný počet chyb. Jak ukazuje obrázek 7.1b, při práci s papírovou dokumentací uživatelé chybovali výrazně častěji. Nejčastějšími chybami bylo vynechání kroku, nesprávné pořadí úkonů nebo nutnost vracet se zpět. V AR manuálu se



(a) Průměrný čas dokončení úkolů podle kategorií.

(b) Množství chyb.

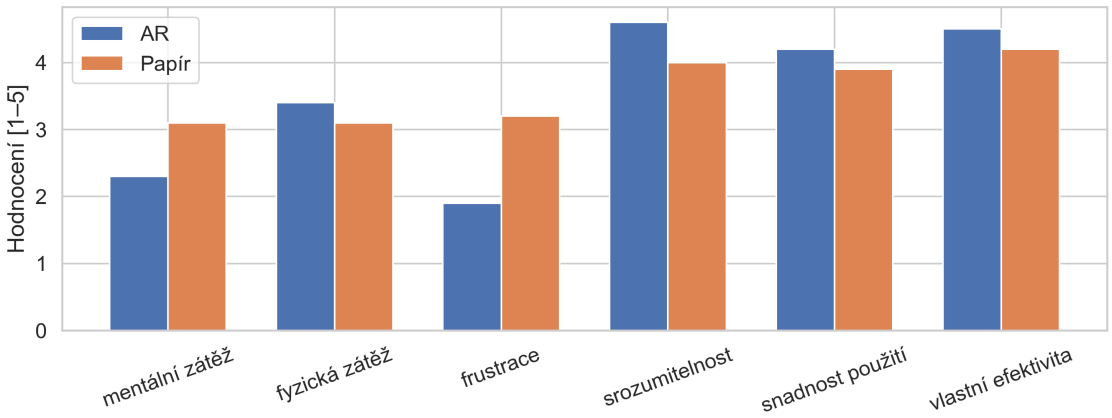
Obrázek 7.1: Srovnání výkonových metrik mezi AR a papírovou dokumentací

chyby vyskytovaly méně často a souvisely především se zvykáním si na rozhraní. Celkově přístup využívající AR vykazuje nižší chybovost a stabilnější provádění úkolů.

Uživatelská hodnocení

Subjektivní hodnocení byla získána na pětibodové škále, kde hodnota **1** označuje minimální míru sledovaného ukazatele (například nízkou zátěž nebo nízkou frustraci) a hodnota **5** — maximální. Konečné výsledky lze shrnout následovně:

- **Mentální zátěž.** Uživatelé ohodnotili AR manuál jako méně zatěžující ve srovnání s papírovou dokumentací. Nižší hodnoty ukazují, že postupné vedení snižuje potřebu udržovat strukturu úkolu v paměti.
- **Fyzická zátěž.** Rozdíly mezi metodami byly mírné. AR vyžadovala držení zařízení, což mírně zvyšovalo fyzickou zátěž, zatímco papírová verze vyžadovala neustálé vracení se k návodu.
- **Frustrace.** AR manuál získal výrazně nižší hodnocení frustrace. Uživatelé se méně často setkávali se situacemi, kdy se museli vracet zpět nebo hledat chybějící informace, což snižovalo jejich emoční napětí. Uživatelé také ocenili nový přístup k používání manuálu.
- **Srozumitelnost rozhraní a jednotlivých kroků.** AR byl vnímán jako srozumitelnější a intuitivnější. Vysoké hodnoty odrážejí to, že vizuální nápovědy a strukturované podání kroků usnadňovaly plnění úkolů.
- **Snadnost použití.** AR manuál získal vyšší hodnocení z hlediska uživatelské přívětivosti. Uživatelé poznamenali, že systém je zbavuje nutnosti procházet dokumentem a hledat potřebnou sekci, což je obzvlášť důležité u složitých úkolů.



Obrázek 7.2: Uživatelské hodnocení systému na pětibodové škále.

- **Vnímaná vlastní efektivita.** Účastníci se cítili sebejistěji a efektivněji při používání AR manuálu (což ne vždy odpovídalo metrikám uvedeným na obrázku 7.1a). Papírová dokumentace vykazovala podobné, avšak mírně nižší hodnocení, což souvisí s vyšší pravděpodobností chyb a ztrátou orientace v textu.

7.3 Empirické výsledky (uživatelské hodnocení)

Účastníci testování hodnotili aplikaci z hlediska použitelnosti, srozumitelnosti rozhraní a toho, jak snadno se jim dařilo plnit zadané úkoly. Níže jsou shrnuty nejčastěji zmiňované aspekty.

Pozitivní aspekty

Většina respondentů uvedla, že práce s AR manuálem byla vnímána jako jednodušší a „zajímavější“ než s papírovou dokumentací, především díky tomu, že informace byly navázány přímo na reálný objekt.

Kladně byla hodnocena i celková orientace v rozhraní: uživatelé rychle pochopili, jak se přepínat mezi kroky, kde hledat nápovědu a jakým způsobem se zobrazují body interakce. Dokonce i ti účastníci, kteří jen zřídka pracují s technickými návody, poznamenali, že se nemuseli „probírat“ strukturou manuálu – aplikace je sama dovedla na správné místo.

Přestože v současné době obsah manuálu neobsahuje obrázky ani videa, uživatelé nehlásili žádné výrazně negativní zkušenosti. Při opakovaném provádění úkolů začala většina respondentů aplikaci využívat jako rychlou příručku.

Negativní aspekty

Nejčastěji zmiňovaným problémem byla nutnost držet tablet v rukou, což při práci s velkými objekty, dlouhými scénáři a také v podmínkách snížené mobility vedlo k únavě a zhoršení stability zobrazení. V podmínkách nízkého osvětlení bylo navíc nutné držet v rukou zdroj světla pro rozpoznávání markerů, což bránilo práci.

U scénářů s postupnými kroky uživatelům, na rozdíl od papírové verze, vadilo, že si nemohli předem prohlédnout budoucí ani již splněné kroky. To je někdy zpomalovalo a také

v nich vyvolávalo pocit „nejistoty“. Také při malé znalosti zařízení měli uživatelé potíže najít požadovaný marker, pokud nebyl v přímém zorném poli.

Nápady na zlepšení

Někteří uživatelé zmínili funkce, které by podle jejich názoru bylo vhodné do aplikace doplnit, případně funkce, jejichž přítomnost při používání této aplikace očekávali. Mezi těmito podněty lze vyzdvihnout následující:

- Vizuální označení předpokládaného umístění požadovaného markeru, pokud jej uživatel dosud nenašel.
- Možnost vzdálené asistence.
- Přístup k elektronické verzi manuálů.
- Možnost zapínat a vypínat svítilnu na zařízení.

7.4 Stabilita fungování aplikace

Tato sekce popisuje stabilitu z technického hlediska, která byla hodnocena na základě chování systému v různých podmínkách. Cílem bylo zjistit limity, při kterých systém začíná ztrácet přesnost nebo zcela ztrácí sledování.

Některé technické aspekty již byly rozebrány dříve, například částečné zakrytí markeru, které bylo v kapitole 3.3 diskutováno v souvislosti se zvýšením počtu opravovaných chyb. Jak lze očekávat, robustnost detekce v takových situacích závisí na charakteru zakrytí: přirozené opotřebení markeru má na proces rozpoznávání zanedbatelný vliv, zatímco fyzické překrytí markeru jiným objektem vede k výraznému snížení stability detekce a k častějším výpadkům trackingu.

Úhel naklonění zařízení

Během testování bylo zjištěno, že program je schopen rozpoznávat markery při extrémních náklonech až do 84.6°, vzhledem k normále markeru. Stabilita detekce i přesnost výpočtu 6DoF však výrazně klesá. Optimální variantou je úhel do 64°, při této hodnotě je zachována jak přesnost výpočtů, tak i stabilita.

Tyto výpočty byly prováděny při přirozeném osvětlení ve vzdálenosti 30 centimetrů. Vzdalování od markeru zvyšuje minimální prahové hodnoty, avšak ne kriticky.

Vzdálenost od markeru

Níže jsou uvedeny výsledky měření stability rozpoznávání markerů různých velikostí při přirozeném osvětlení a matném povrchu objektu. Pro každou velikost je uvedena vzdálenost spolehlivého rozpoznávání, oblast snížené stability a maximální vzdálenost, za kterou detekce přestává fungovat.

Neměřitelné aspekty

Některé vlastnosti systému není možné v rámci provedeného testování kvantitativně změřit, avšak je lze subjektivně posoudit na základě pozorování.

Tabulka 7.1: Stabilita rozpoznávání v závislosti na velikosti markeru a vzdálenosti

Velikost markeru	Stabilní	Snížená	Maximální vzdálenost
6 cm	do 340 cm	340–430 cm	430 cm
5 cm	do 300 cm	300–360 cm	360 cm
4 cm	do 230 cm	230–290 cm	290 cm
3 cm	do 140 cm	140–210 cm	210 cm

Vliv komunikace s backendem při práci v lokální síti. Zpoždění při požadavcích bylo minimální a nemělo znatelný dopad na interakci uživatele s aplikací. Odpovědi přicházely dostatečně rychle na to, aby nenarušovaly plynulost práce rozhraní a nevytvářely pocit čekání.

Při rychlých nebo prudkých pohybech kamery byl pozorován pro AR systémy typický efekt: částečná ztráta prostorové registrace a krátkodobý drift virtuálních objektů. Ve většině případů systém samostatně obnovil stabilní polohu poté, co se pohyb zařízení ustálil. Při opakovaných prudkých posunech nebo nedostatku vizuálních orientačních bodů však obnovení trvalo déle. Pro zvýšení odolnosti v podobných situacích může být zapotřebí dodatečná logika pro opakovanou kalibraci prostoru.

7.5 Omezení testování

Je rovněž nezbytné explicitně vymezit omezení realizovaného testování, která souvisejí jak s charakterem použitého prototypu, tak s praktickými podmínkami, za nichž byl výzkum prováděn. Níže jsou specifikovány klíčové faktory, jež mohou podstatným způsobem ovlivnit interpretaci dosažených výsledků.

- **Malý rozsah vzorku**
  Testování se zúčastnilo pouze osm lidí. To stačí k odhalení hlavních problémů s použitelností, ale nestačí pro statisticky významné závěry. Výsledky proto odrážejí spíše kvalitativní tendence než kvantitativní zákonitosti.

- **Použití jednoho zařízení**
  Všechny testy byly provedeny na jednom tabletu. Jednotlivé modely zařízení se liší kvalitou fotoaparátu, výkonem, hmotností a ergonomií, což může výrazně ovlivnit pohodlí při práci a stabilitu AR systému.

- **Omezená sada scénářů**
  Ačkoli byly scénáře založeny na reálných každodenních a pracovních úkolech, jejich počet a rozmanitost jsou omezené. Při skutečném provozu mohou nastat složitější nebo nestandardní situace, které nebyly v rámci testování modelovány.

- **Prototypový charakter aplikace**
  Netestovala se finální verze systému, ale prototyp. V něm chybějí některé funkce, které uživatelé očekávají (například multimediální obsah, volba jazyka a podobné). Část negativních ohlasů může souviset právě s absencí těchto možností.

- **Krátkodobé používání**
  Uživatelé pracovali s aplikací jen po omezenou dobu. To neumožňuje posoudit dlouhodobé efekty: zvykání si na rozhraní, snižování nebo zvyšování efektivity v čase a vliv únavy při dlouhodobém používání.

Kapitola 8

Závěr

Cílem této práce bylo navrhnout a zhodnotit systém interaktivního manuálu v rozšířené realitě, určený k zobrazování technických instrukcí přímo v prostoru reálného objektu. Byl vytvořen funkční prototyp, který zahrnuje AR část s detekcí markerů, uživatelské rozhraní a serverovou infrastrukturu pro správu instrukcí.

Při návrhu architektury systému byla věnována zvláštní pozornost zajištění snadné udržovatelnosti a rozšiřitelnosti: modulární struktura, jednoznačně vymezená rozhraní mezi komponentami a abstrakce zdrojů dat umožňují přizpůsobit řešení různým scénářům použití a usnadňují další integraci nových funkcí. Návrh uživatelského rozhraní a způsobu interakce byl vytvářen s ohledem na výsledky dotazníkových šetření cílové skupiny a na analýzu stávajících prací v oblasti AR.

Jedním z klíčových technických výsledků práce je realizace podpory současné práce s velkým množstvím markerů. To umožňuje přiřazovat instrukce k velkému počtu prvků objektu a činí systém použitelným v podstatně složitějších scénářích než řešení omezená jedním nebo několika markery.

Pro zhodnocení praktické užitečnosti a spolehlivosti řešení bylo provedeno komplexní testování. Byly analyzovány metriky efektivity uživatelů – rychlost plnění úkolů, počet chyb a subjektivní hodnocení pohodlí použití. Experimentální data ukazují, že při provádění složitých sekvenčních operací poskytuje využití AR oproti tradiční papírové dokumentaci významnou časovou výhodu a zároveň dochází ke snížení počtu chyb, kterých se uživatelé dopouštějí. V rámci dotazníkového šetření účastníci uvedli snížení mentální zátěže a míry frustrace při práci s AR-instrukcemi, stejně jako lepší pochopení posloupnosti kroků a aktuálního stavu úkolu.

Kromě uživatelského testování bylo provedeno také testování stability běhu aplikace a spolehlivosti detekce markerů v různých provozních podmínkách. Získané výsledky ukazují vhodnost prototypu pro reálné použití – spolehlivé rozpoznávání bylo zachováno při náklonu až do 64° (vzhledem k normále markeru) a vzdálenost stabilního rozpoznávání markeru o velikosti 5 cm dosahovala 300 cm. Testování bylo prováděno v různých světelných podmínkách, což umožnilo vyhodnotit vliv osvětlení na spolehlivost detekce.

Další rozvoj systému může zahrnovat zlepšení stability trackingu, optimalizaci vizuálních podnětů a realizaci dalších funkcí uvedených v kapitolách 4.2 a 7.3. Z dlouhodobého hlediska je možné zavést tracking pohybujících se markerů, adaptovat řešení pro jiné AR platformy a využívat data IoT zařízení k automatické aktualizaci manuálů.

Literatura

[1] Abe, E. a Suzuki, K. On normal subgroups of Chevalley groups over commutative rings. *Tohoku Mathematical Journal, Second Series*. Mathematical Institute, Tohoku University, 1976, sv. 28, č. 2, s. 185–198.

[2] Atkinson, P. *GIS and GeoComputation: Innovations in GIS 7*. CRC Press, 2000.

[3] Azuma, R. T. A survey of augmented reality. *Presence: teleoperators & virtual environments*. MIT press One Rogers Street, Cambridge, MA 02142-1209, USA journals-info . . . , 1997, sv. 6, č. 4, s. 355–385.

[4] Barsom, E. Z.; Graafland, M. a Schijven, M. P. Systematic review on the effectiveness of augmented reality applications in medical training. *Surgical endoscopy*. Springer, 2016, sv. 30, č. 10, s. 4174–4183.

[5] Benligiray, B.; Topal, C. a Akinlar, C. STag: A stable fiducial marker system. *Image and Vision Computing*. Elsevier, 2019, sv. 89, s. 158–169.

[6] Buchner, J.; Buntins, K. a Kerres, M. The impact of augmented reality on cognitive load and performance: A systematic review. *Journal of Computer Assisted Learning*. Wiley Online Library, 2022, sv. 38, č. 1, s. 285–303.

[7] Collins, T. a Bartoli, A. Infinitesimal plane-based pose estimation. *International journal of computer vision*. Springer, 2014, sv. 109, č. 3, s. 252–286.

[8] Dubrofsky, E. Homography estimation. *Diplomová práce. Vancouver: Univerzita Britské Kolumbie*, 2009, sv. 5.

[9] Fiala, M. Designing highly reliable fiducial markers. *IEEE Transactions on Pattern analysis and machine intelligence*. IEEE, 2009, sv. 32, č. 7, s. 1317–1324.

[10] Fiorentino, M.; Uva, A. E.; Gattullo, M.; Debernardis, S. a Monno, G. Augmented reality on large screen for interactive maintenance instructions. *Computers in Industry*. Elsevier, 2014, sv. 65, č. 2, s. 270–278.

[11] Garcia, C.; Ortega, M.; Ivorra, E.; Contero, M.; Mora, P. et al. Holorailway: an augmented reality system to support assembly operations in the railway industry. *Advances in Manufacturing*. Springer, 2024, sv. 12, č. 4, s. 764–783.

[12] Garrido Jurado, S.; Muñoz Salinas, R.; Madrid Cuevas, F. J. a Marín Jiménez, M. J. Automatic generation and detection of highly reliable fiducial markers under occlusion. *Pattern Recognition*. Elsevier, 2014, sv. 47, č. 6, s. 2280–2292.

[13] Garrido Jurado, S.; Munoz Salinas, R.; Madrid Cuevas, F. J. a Medina Carnicer, R. Generation of fiducial marker dictionaries using mixed integer linear programming. *Pattern recognition*. Elsevier, 2016, sv. 51, s. 481–491.

[14] Gattullo, M.; Scurati, G. W.; Fiorentino, M.; Uva, A. E.; Ferrise, F. et al. Towards augmented reality manuals for industry 4.0: A methodology. *Robotics and Computer-Integrated Manufacturing*. Elsevier, 2019, sv. 56, s. 276–286.

[15] Guan, J.; Hao, Y.; Wu, Q.; Li, S. a Fang, Y. A survey of 6dof object pose estimation methods for different application scenarios. *Sensors*. MDPI, 2024, sv. 24, č. 4, s. 1076.

[16] Henderson, S. a Feiner, S. Exploring the benefits of augmented reality documentation for maintenance and repair. *IEEE transactions on visualization and computer graphics*. Ieee, 2010, sv. 17, č. 10, s. 1355–1368.

[17] Henderson, S. J. a Feiner, S. Evaluating the benefits of augmented reality for task localization in maintenance of an armored personnel carrier turret. In: IEEE. *2009 8th IEEE international symposium on mixed and augmented reality*. 2009, s. 135–144.

[18] Hershberger, J. E. a Snoeyink, J. Speeding up the Douglas-Peucker line-simplification algorithm. University of British Columbia, Department of Computer Science Vancouver, BC, 1992.

[19] Kalaitzakis, M.; Cain, B.; Carroll, S.; Ambrosi, A.; Whitehead, C. et al. Fiducial markers for pose estimation: Overview, applications and experimental comparison of the artag, apriltag, aruco and stag markers. *Journal of Intelligent & Robotic Systems*. Springer, 2021, sv. 101, č. 4, s. 71.

[20] Kam, H. C.; Yu, Y. K. a Wong, K. H. An improvement on aruco marker for pose tracking using kalman filter. In: IEEE. *2018 19th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*. 2018, s. 65–69.

[21] Karami, E.; Prasad, S. a Shehata, M. Image matching using SIFT, SURF, BRIEF and ORB: performance comparison for distorted images. *ArXiv preprint arXiv:1710.02726*, 2017.

[22] Kato, H. a Billinghurst, M. Marker tracking and hmd calibration for a video-based augmented reality conferencing system. In: IEEE. *Proceedings 2nd IEEE and ACM International Workshop on Augmented Reality (IWAR'99)*. 1999, s. 85–94.

[23] Ke, T. a Roumeliotis, S. I. An efficient algebraic solution to the perspective-three-point problem. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2017, s. 7225–7233.

[24] Khazetdinov, A.; Zakiev, A.; Tsoy, T.; Svinin, M. a Magid, E. Embedded ArUco: a novel approach for high precision UAV landing. In: IEEE. *2021 International Siberian Conference on Control and Communications (SIBCON)*. 2021, s. 1–6.

[25] Lepetit, V.; Moreno Noguer, F. a Fua, P. EP n P: An accurate O (n) solution to the P n P problem. *International journal of computer vision*. Springer, 2009, sv. 81, č. 2, s. 155–166.

[26] Madsen, K.; Nielsen, H. B. a Tingleff, O. Methods for non-linear least squares problems, 2004.

[27] Milgram, P.; Takemura, H.; Utsumi, A. a Kishino, F. Augmented reality: A class of displays on the reality-virtuality continuum. In: Spie. *Telemanipulator and telepresence technologies*. 1995, sv. 2351, s. 282–292.

[28] Müller, L.; Aslan, I. a Krüssen, L. GuideMe: A mobile augmented reality system to display user manuals for home appliances. In: Springer. *International Conference on Advances in Computer Entertainment Technology*. 2013, s. 152–167.

[29] Nee, A. Y.; Ong, S. K.; Chryssolouris, G. a Mourtzis, D. Augmented reality applications in design and manufacturing. *CIRP annals*. Elsevier, 2012, sv. 61, č. 2, s. 657–679.

[30] Olson, E. AprilTag: A robust and flexible visual fiducial system. In: IEEE. *2011 IEEE international conference on robotics and automation*. 2011, s. 3400–3407.

[31] Oufqir, Z.; El Abderrahmani, A. a Satori, K. From marker to markerless in augmented reality. In: *Embedded systems and artificial intelligence: Proceedings of ESAI 2019, Fez, Morocco*. Springer, 2020, s. 599–612.

[32] Oun, A.; Hagerdorn, N.; Scheideger, C. a Cheng, X. Mobile devices or head-mounted displays: a comparative review and analysis of augmented reality in healthcare. *IEEE Access*. IEEE, 2024, sv. 12, s. 21825–21839.

[33] Palm, F. a Källström, F. *Prototype of an Augmented Reality User Manual App*. 2014.

[34] Peng, S.; Liu, Y.; Huang, Q.; Zhou, X. a Bao, H. Pvnet: Pixel-wise voting network for 6dof pose estimation. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, s. 4561–4570.

[35] Ragland, K. a Tharcis, P. A survey on object detection, classification and tracking methods. *Int. J. Eng. Res. Technol*, 2014, sv. 3, č. 11, s. 622–628.

[36] Röltgen, D. a Dumitrescu, R. Classification of industrial Augmented Reality use cases. *Procedia CIRP*. Elsevier, 2020, sv. 91, s. 93–100.

[37] Romero Ramirez, F. J.; Muñoz Salinas, R. a Medina Carnicer, R. Speeded up detection of squared fiducial markers. *Image and vision Computing*. Elsevier, 2018, sv. 76, s. 38–47.

[38] Sanna, A. A survey on applications of augmented reality. *Advances in Computer Science: an International Journal*, 2016.

[39] Sanna, A.; Manuri, F.; Lamberti, F.; Paravati, G. a Pezzolla, P. Using handheld devices to sup port augmented reality-based maintenance and assembly tasks. In: IEEE. *2015 IEEE international conference on consumer electronics (ICCE)*. 2015, s. 178–179.

[40] Sergio Garrido, A. P. *ONLINE: OpenCV, Detection of ArUco Markers.* Dostupné z: https://docs.opencv.org/4.12.0/d5/dae/tutorial_aruco_detection.html. Citováno: 2026-3-26.

[41] Shah, M. M.; Arshad, H. a Sulaiman, R. Occlusion in augmented reality. In: IEEE. *2012 8th International Conference on Information Science and Digital Content Technology (ICIDT2012).* 2012, sv. 2, s. 372–378.

[42] Smink, A. R.; Reijmersdal, E. A. van a Noort, G. van. Consumers' use of augmented reality apps: prevalence, user characteristics, and gratifications. *Journal of Advertising.* Taylor & Francis, 2022, sv. 51, č. 1, s. 85–94.

[43] Smith, E.; Semple, G.; Evans, D.; McRae, K. a Blackwell, P. Augmented instructions: Analysis of performance and efficiency of assembly tasks. In: Springer. *International Conference on Human-Computer Interaction.* 2020, s. 166–177.

[44] Syed, T. A.; Siddiqui, M. S.; Abdullah, H. B.; Jan, S.; Namoun, A. et al. In-depth review of augmented reality: Tracking technologies, development tools, AR displays, collaborative AR, and security concerns. *Sensors.* MDPI, 2022, sv. 23, č. 1, s. 146.

[45] Taketomi, T.; Uchiyama, H. a Ikeda, S. Visual SLAM algorithms: A survey from 2010 to 2016. *IPSJ transactions on computer vision and applications.* Springer, 2017, sv. 9, č. 1, s. 16.

[46] Tekin, B.; Sinha, S. N. a Fua, P. Real-time seamless single shot 6d object pose prediction. In: *Proceedings of the IEEE conference on computer vision and pattern recognition.* 2018, s. 292–301.

[47] Uva, A. E.; Gattullo, M.; Manghisi, V. M.; Spagnulo, D.; Cascella, G. L. et al. Evaluating the effectiveness of spatial augmented reality in smart manufacturing: a solution for manual working stations. *The International Journal of Advanced Manufacturing Technology.* Springer, 2018, sv. 94, č. 1, s. 509–521.

[48] Van Noort, G. a Van Reijmersdal, E. A. Branded apps: explaining effects of brands' mobile phone applications on brand responses. *Journal of Interactive Marketing.* SAGE Publications Sage CA: Los Angeles, CA, 2019, sv. 45, č. 1, s. 16–26.

[49] Verhoeye, J. a De Wulf, R. Land cover mapping at sub-pixel scales using linear optimization techniques. *Remote Sensing of Environment.* Elsevier, 2002, sv. 79, č. 1, s. 96–104.

[50] Zollmann, S.; Langlotz, T.; Grasset, R.; Lo, W. H.; Mori, S. et al. Visualization techniques in augmented reality: A taxonomy, methods and patterns. *IEEE transactions on visualization and computer graphics.* IEEE, 2020, sv. 27, č. 9, s. 3808–3825.